SHOWSHELF LAB · FOR THE AFTERNOON TEACHER

Teacher guide

Students work through the steps in the Lab on their own. Each step has an explanation, a task, two hints and an answer button, so most of the time they can get unstuck alone. This guide gives you every task, a correct answer and the mistakes students usually make, so you can help quickly.

How to help: first ask them to read the "Not yet" message and the red Console line (if any), then open a hint. Only point at the answer after that. In the Lab, **Help › Teacher view** shows the answer and these notes on every step.

Day 1: Your first web page HTML

Today you build a web page about one TV show, **Friends**, using **HTML**: the language that tells the browser what each piece of a page *is*.

You start with an empty box and add one thing at a time: a heading, a paragraph, a picture, lists, a link and a table. The top bar and the footer are already there, so you can focus on the show itself.

Take your time with each step. There's no rush, and every step saves automatically.

Step 1. Your first HTML tag ~10 min

Teaches: An element is an opening tag, content and a closing tag. `<h1>` is the main heading.

TASK

In the dark code box, write a main heading that says **Friends**.

Watch the preview on the right while you type. Then press **Check**.

COMMON MISTAKES

- Forgetting the slash in the closing tag: `<h1>Friends<h1>`.
- A space inside the brackets: `< h1>`.
- Writing in the preview instead of the dark code box.

NOTE

The goal is only the idea of opening tag, content, closing tag. Let them try breaking it: delete the closing tag and see what happens (the browser guesses).

A CORRECT ANSWER

```
<h1>Friends</h1>
```

Step 2. Add a paragraph ~10 min

Teaches: `<p>` makes a paragraph. The browser ignores extra spaces and new lines in your code.

TASK

Under your heading, add a paragraph with this text (copy it):

Six friends in their twenties and thirties share apartments, jobs, break-ups and a lot of coffee in New York City.

COMMON MISTAKES

- Putting the paragraph inside the heading: `<h1>Friends <p>...</p></h1>`.
- Forgetting `</p>` (the browser often guesses, but the check asks for it).

A CORRECT ANSWER

```
<h1>Friends</h1>

<p>
  Six friends in their twenties and thirties share apartments, jobs,
  break-ups and a lot of coffee in New York City.
</p>
```

Step 3. Bold and italic ~10 min

Teaches: Elements can be nested. `<strong>` is important (bold), `<em>` is emphasis (italic).

TASK

In your paragraph:

1. make **New York City** bold with `<strong>`
2. make **a lot of coffee** italic with `<em>`

COMMON MISTAKES

- Closing in the wrong order: `<p>...<strong>...</p></strong>`.
- Using `<b>` and `<i>`. They also work, but `<strong>` and `<em>` say *why* the words are special.

A CORRECT ANSWER

```
<!-- ... unchanged ... -->
<p>
  Six friends in their twenties and thirties share apartments, jobs,
  break-ups and <em>a lot of coffee</em> in <strong>New York City</strong>.
</p>
```

Step 4. Attributes and a class ~15 min

Teaches: Attributes go in the opening tag as `name="value"`. A `class` is a label for CSS.

TASK

Between the heading and the paragraph, add a new paragraph with the class `meta` and this text:

NBC · 1994–2004 · about 22 minutes per episode

The dot `·` is hard to type on some keyboards. A normal dash `-` or a `|` is fine too.

COMMON MISTAKES

- No quotes: `class=meta` (works in browsers, but always use quotes).
- The attribute in the closing tag: `</p class="meta">`.

A CORRECT ANSWER

```
<h1>Friends</h1>
<p class="meta">NBC · 1994–2004 · about 22 minutes per episode</p>

<p>
<!-- ... unchanged ... -->
```

Step 5. Add the poster ~15 min

Teaches: `<img src="..." alt="...">` shows a picture. It has no closing tag, and `alt` describes the picture.

TASK

Under the heading, add the poster:

- `src` is `poster.svg`
- `alt` describes it, for example "Poster for Friends"
- also add `class="poster"`, for tomorrow's CSS

Then try this: change `src` to a name that doesn't exist, like `posterr.svg`. What do you see? Change it back before you press Check.

COMMON MISTAKES

- Writing `<img src="poster.svg"></img>`.
- Leaving out `alt`, or writing "image" as the alt text (describe what it shows instead).
- Typing `scr` instead of `src`.

A CORRECT ANSWER

```
<!-- ... unchanged ... -->
<p class="meta">NBC · 1994–2004 · about 22 minutes per episode</p>

<img class="poster" src="poster.svg" alt="Poster for Friends">

<p>
  Six friends in their twenties and thirties share apartments, jobs,
<!-- ... unchanged ... -->
```

Step 6. Subheadings and a list ~15 min

Teaches: `<h2>` for parts of the page. `<ul>` with `<li>` items makes a bulleted list.

TASK

Under your paragraph, add:

1. a subheading `<h2>` that says **Genres**
2. a bulleted list with two items: **Comedy** and **Romance**

COMMON MISTAKES

- `<li>` items without a `<ul>` around them.
- Two separate lists, one per item, instead of one list with two items.

A CORRECT ANSWER

```
<!-- ... unchanged ... -->
  break-ups and <em>a lot of coffee</em> in <strong>New York City</strong>.
</p>

<h2>Genres</h2>
<ul>
  <li>Comedy</li>
  <li>Romance</li>
</ul>
```

Step 7. A longer list: the cast ~10 min

Teaches: Long lists: one `<li>` per line, indented inside the `<ul>`.

TASK

Under the genres list, add an `<h2>` that says **Main cast** and a list with **at least three** of these actors:

- Jennifer Aniston as Rachel Green
- Courteney Cox as Monica Geller
- Lisa Kudrow as Phoebe Buffay
- Matt LeBlanc as Joey Tribbiani
- Matthew Perry as Chandler Bing
- David Schwimmer as Ross Geller

COMMON MISTAKES

- Putting all actors in one `<li>` separated by commas.
- Forgetting the new `<h2>`.

A CORRECT ANSWER

```
<!-- ... unchanged ... -->
  <li>Romance</li>
</ul>

<h2>Main cast</h2>
<ul>
  <li>Jennifer Aniston as Rachel Green</li>
  <li>Courteney Cox as Monica Geller</li>
  <li>Lisa Kudrow as Phoebe Buffay</li>
  <li>Matt LeBlanc as Joey Tribbiani</li>
  <li>Matthew Perry as Chandler Bing</li>
  <li>David Schwimmer as Ross Geller</li>
</ul>
```

Step 8. A numbered list ~10 min

Teaches: `<ol>` is a numbered list, for when the order matters.

TASK

Add an `<h2>` that says **My favourite characters** and an ordered list of your top three. Your opinion, your order.

COMMON MISTAKES

- Typing "1." inside each item (you get numbers twice).

A CORRECT ANSWER

```
<!-- ... unchanged ... -->
  <li>David Schwimmer as Ross Geller</li>
</ul>

<h2>My favourite characters</h2>
<ol>
  <li>Phoebe</li>
  <li>Chandler</li>
  <li>Joey</li>
</ol>
```

Step 9. A link to another website ~15 min

Teaches: `<a href="https://...">text</a>` is a link to another page.

TASK

At the bottom, add a paragraph with a link to the show's page on TVmaze, a free website about TV shows:

`https://www.tvmaze.com/shows/431/friends`

Use any link text you like, for example "Read more on TVmaze". Click the link in the preview when you're done: it opens in a new tab.

COMMON MISTAKES

- Leaving out `https://`, so the link points to a missing file.
- Putting the address as the link text but forgetting `href`.

A CORRECT ANSWER

```
<!-- ... unchanged ... -->
  <li>Joey</li>
</ol>

<p>
  Want more? <a href="https://www.tvmaze.com/shows/431/friends">Read about it on TVmaze</a>.
</p>
```

Step 10. A table of seasons ~20 min

Teaches: Tables: `<table>`, rows `<tr>`, heading cells `<th>`, data cells `<td>`.

TASK

Add two more rows to the table:

- season **2**: **24** episodes, **1995**
- season **3**: **25** episodes, **1996**

Each new row is a `<tr>` with three `<td>` cells, just like the season 1 row.

COMMON MISTAKES

- Pasting the new row after `</table>` instead of before it.
- A row with only two `<td>`: the columns shift.
- Missing `</tr>` between rows.

A CORRECT ANSWER

```
<!-- ... unchanged ... -->
</ol>

<h2>Seasons</h2>
<table>
  <tr>
    <th>Season</th>
    <th>Episodes</th>
    <th>Year</th>
  </tr>
  <tr>
    <td>1</td>
    <td>24</td>
    <td>1994</td>
  </tr>
  <tr>
    <td>2</td>
    <td>24</td>
    <td>1995</td>
  </tr>
  <tr>
    <td>3</td>
    <td>25</td>
    <td>1996</td>
  </tr>
</table>

<p>
  Want more? <a href="https://www.tvmaze.com/shows/431/friends">Read about it on TVmaze</a>.
<!-- ... unchanged ... -->
```

Step 11. Group it with sections ~15 min

Teaches: Semantic elements (`header` , `main` , `article` , `section` , `footer`) describe the parts of a page.

TASK

Wrap **Genres** (the `<h2>` and its list) in a `<section>`. Then do the same for **Main cast**.

If you like, wrap the other two parts as well. Indent the lines inside each section by two spaces.

COMMON MISTAKES

- One big section around everything.
- `</section>` placed between the heading and its list.

A CORRECT ANSWER

```
<h1>Friends</h1>
<p class="meta">NBC · 1994–2004 · about 22 minutes per episode</p>

<img class="poster" src="poster.svg" alt="Poster for Friends">

<p>
  Six friends in their twenties and thirties share apartments, jobs,
  break-ups and <em>a lot of coffee</em> in <strong>New York City</strong>.
</p>

<section>
  <h2>Genres</h2>
  <ul>
    <li>Comedy</li>
    <li>Romance</li>
  </ul>
</section>

<section>
  <h2>Main cast</h2>
  <ul>
    <li>Jennifer Aniston as Rachel Green</li>
    <li>Courteney Cox as Monica Geller</li>
    <li>Lisa Kudrow as Phoebe Buffay</li>
    <li>Matt LeBlanc as Joey Tribbiani</li>
    <li>Matthew Perry as Chandler Bing</li>
    <li>David Schwimmer as Ross Geller</li>
  </ul>
</section>

<section>
  <h2>My favourite characters</h2>
  <ol>
    <li>Phoebe</li>
    <li>Chandler</li>
    <li>Joey</li>
  </ol>
</section>

<section>
  <h2>Seasons</h2>
  <table>
    <tr>
      <th>Season</th>
      <th>Episodes</th>
      <th>Year</th>
    </tr>
    <tr>
```

```
      <td>1</td>
      <td>24</td>
      <td>1994</td>
    </tr>
    <tr>
      <td>2</td>
      <td>24</td>
      <td>1995</td>
    </tr>
    <tr>
      <td>3</td>
      <td>25</td>
      <td>1996</td>
    </tr>
  </table>
</section>

<p>
  Want more? <a href="https://www.tvmaze.com/shows/431/friends">Read about it on TVmaze</a>.
</p>
```

Step 12. Notes for humans ~10 min

Teaches: `<!-- comments -->` are notes the browser ignores.

TASK

1. Add a comment above the seasons section that says what it is.
2. Try hiding the "My favourite characters" section with a comment, look at the preview, then remove the comment marks again so it comes back.

COMMON MISTAKES

- A comment that never closes (no `-->`): everything after it disappears.

A CORRECT ANSWER

```
<!-- ... unchanged ... -->
</section>

<!-- Seasons: one row per season -->
<section>
  <h2>Seasons</h2>
<!-- ... unchanged ... -->
```

Extra 1. A spoiler you have to click ~10 min

Teaches: `<details>` with `<summary>` makes a box that opens on click.

TASK

Add a `<details>` element with a `<summary>` like "Spoiler: how does the show end?" and a paragraph inside with your answer (or "Watch it and find out!").

Click it in the preview to open and close it.

A CORRECT ANSWER

```
<!-- ... unchanged ... -->
  Want more? <a href="https://www.tvmaze.com/shows/431/friends">Read about it on TVmaze</a>.
</p>

<details>
  <summary>Spoiler: how does the show end?</summary>
  <p>Watch it and find out!</p>
</details>
```

Extra 2. A poster with a caption ~10 min

Teaches: `<figure>` + `<figcaption>`: a picture with a caption.

TASK

Wrap your poster `<img>` in a `<figure>` and add a `<figcaption>` under it, for example "The official poster".

A CORRECT ANSWER

```
<!-- ... unchanged ... -->
<p class="meta">NBC · 1994–2004 · about 22 minutes per episode</p>

<figure>
  <img class="poster" src="poster.svg" alt="Poster for Friends">
  <figcaption>The official poster</figcaption>
</figure>

<p>
<!-- ... unchanged ... -->
```

Extra 3. Add a second show ~20 min

Teaches: You can build a page section from scratch: heading, text, list, link.

TASK

At the bottom, add:

1. an `<hr>`
2. an `<h2>` with the name of a show you like
3. a paragraph about it
4. a list of at least two things (genres, characters, anything)
5. optional: a link to its page on TVmaze (search for it on tvmaze.com)

A CORRECT ANSWER

```
<!-- ... unchanged ... -->
  Want more? <a href="https://www.tvmaze.com/shows/431/friends">Read about it on TVmaze</a>.
</p>

<hr>
<h2>The Office</h2>
<p>A camera crew follows the everyday chaos of a small paper company office.</p>
<ul>
  <li>Comedy</li>
  <li>Michael Scott is the boss</li>
</ul>
```

Day 2: Style it CSS

Yesterday's page works, but it looks plain. Today you give it colours, fonts, spacing and a card design with **CSS**, the language that says how things *look*.

The HTML is the same Friends page you built yesterday (our tidy copy of it). You only write CSS today, in a file called **style.css**. To see which classes and elements you can style, open the **show.html (whole page)** tab above the code box.

At the end you choose your own colours. They'll follow you through the rest of the course.

Step 1. Your first CSS rule ~15 min

Teaches: A CSS rule: `selector { property: value; }`.

TASK

Give the page a light background colour of your choice, using the `body` selector.

Some ideas: `#f6f5fb` (light violet), `#fdf6e3` (cream), `#eef6f3` (mint), `lavender`.

COMMON MISTAKES

- British spelling: `background-colour` doesn't work in CSS.
- Forgetting the `;` or the closing `}`.
- A space inside the hex code: `# f6f5fb`.

A CORRECT ANSWER

```
/* style.css: how ShowShelf looks */

body {
  background-color: #f6f5fb;
}
```

Step 2. Text colour and font ~15 min

Teaches: `color`, `font-family` and `line-height` style text. Elements inherit them from `body`.

TASK

In your `body` rule, add:

- a dark text `color` (not pure black: try `#1c1b29`)
- a `font-family` without little "feet", for example `system-ui, Arial, sans-serif`
- a `line-height` of about `1.6`

COMMON MISTAKES

- A second `body { }` rule for each property (works, but keep them together).
- Missing commas between fonts: `system-ui Arial sans-serif`.
- Writing `font: Arial;` instead of `font-family`.

A CORRECT ANSWER

```
/* ... unchanged ... */
body {
  background-color: #f6f5fb;
  color: #1c1b29;
  font-family: system-ui, Arial, sans-serif;
  line-height: 1.6;
}
```

Step 3. Colour the headings ~10 min

Teaches: Element selectors like `h1` style every element of that kind. Commas group selectors.

TASK

Give all `h1` and `h2` headings an accent colour of your choice, in one rule. Try a strong colour, like violet `#5b3fd6`.

COMMON MISTAKES

- A space instead of a comma: `h1 h2` means "an h2 inside an h1", which doesn't exist here.

A CORRECT ANSWER

```
/* ... unchanged ... */
  line-height: 1.6;
}

h1, h2 {
  color: #5b3fd6;
}
```

Step 4. Style by class ~15 min

Teaches: `.name` selects elements with `class="name"`. Sizes need units like `px`.

TASK

Make the facts line (`class="meta"`) smaller and grey: a grey `color` like `#676580` and a `font-size` of `14px`.

COMMON MISTAKES

- No dot: `meta { }` looks for a `<meta>` element.
- A number without a unit: `font-size: 14;` is ignored.
- A dot in the HTML: `class=".meta"`.

A CORRECT ANSWER

```
/* ... unchanged ... */
  color: #5b3fd6;
}

.meta {
  color: #676580;
  font-size: 14px;
}
```

Step 5. The box model: make a card ~20 min

Teaches: Box model: content, padding (inside), border, margin (outside).

TASK

The whole show is inside `<article class="show">`. Turn it into a white card:

- `background-color: #ffffff;`
- `padding` of `24px`
- a thin `border` , like `1px solid #e3e0f0`
- rounded corners with `border-radius: 12px;`

COMMON MISTAKES

- `border: 1px #e3e0f0;` without a style (`solid`): no border appears.
- Mixing up padding and margin.

A CORRECT ANSWER

```
/* ... unchanged ... */
  font-size: 14px;
}

.show {
  background-color: #ffffff;
  padding: 24px;
  border: 1px solid #e3e0f0;
  border-radius: 12px;
}
```

Step 6. Width and centring ~15 min

Teaches: `max-width` limits the width; `margin: 0 auto` centres a box.

TASK

In your `.show` rule, add a `max-width` of `640px` and a `margin` of `32px auto`.

COMMON MISTAKES

- `width: 640px` instead of `max-width`: the card is too wide for phones.
- `margin: auto 32px` (values swapped): no space above, and it isn't centred.

A CORRECT ANSWER

```
/* ... unchanged ... */

.show {
  max-width: 640px;
  margin: 32px auto;
  background-color: #ffffff;
  padding: 24px;
/* ... unchanged ... */
```

Step 7. Size the poster ~10 min

Teaches: Size images with `width`; the height follows.

TASK

Style the poster (`class="poster"`): a `width` of `200px` and slightly rounded corners, `border-radius: 8px`.

A CORRECT ANSWER

```
/* ... unchanged ... */
  border-radius: 12px;
}

.poster {
  width: 200px;
  border-radius: 8px;
}
```

Step 8. Header and footer ~20 min

Teaches: Style parts of the page by their class: header, logo, footer.

TASK

1. `.site-header`: a white `background-color`, `padding: 16px 24px;` and a `border-bottom` like `1px solid #e3e0f0`
2. `.logo`: `font-weight: bold;`, a bigger `font-size` (like `22px`) and `text-decoration: none;`
3. `.site-footer`: `text-align: center;` and a grey `color`

A CORRECT ANSWER

```
/* ... unchanged ... */
  border-radius: 8px;
}

.site-header {
  background-color: #ffffff;
  padding: 16px 24px;
  border-bottom: 1px solid #e3e0f0;
}

.logo {
  font-weight: bold;
  font-size: 22px;
  text-decoration: none;
}

.site-footer {
  text-align: center;
  color: #676580;
}
```

Step 9. Links and hover ~15 min

Teaches: `a:hover` styles a link while the mouse is over it.

TASK

1. Give all links (`a`) your accent colour, the same as your headings.
2. Add an `a:hover` rule that changes the colour to a darker shade, like `#3d2a99`.

COMMON MISTAKES

- A space: `a :hover` means something different and doesn't work.

A CORRECT ANSWER

```
/* ... unchanged ... */
  color: #676580;
}

a {
  color: #5b3fd6;
}

a:hover {
  color: #3d2a99;
}
```

Step 10. A tidy table ~15 min

Teaches: `border-collapse: collapse` and cell padding make tables readable.

TASK

Style the seasons table: `border-collapse: collapse;` on `table`, and on `th, td` some `padding`, a `border-bottom` and `text-align: left;`.

A CORRECT ANSWER

```
/* ... unchanged ... */
  color: #3d2a99;
}

table {
  border-collapse: collapse;
}

th, td {
  padding: 6px 16px;
  border-bottom: 1px solid #e3e0f0;
  text-align: left;
}
```

Step 11. Your own colours with variables ~25 min

Teaches: CSS variables: define `--name: value;` in `:root`, use it with `var(--name)`.

TASK

1. At the top of your CSS, add a `:root` rule with two variables: `--color-accent` (your favourite strong colour) and `--color-bg` (a light background colour).
2. Use `var(--color-accent)` for the headings (`h1, h2`) and the links (`a`).
3. Use `var(--color-bg)` for the body's background.

Then change `--color-accent` to a different colour and watch the headings and links change together.

COMMON MISTAKES

- One dash instead of two: `-color-accent`.
- `var(color-accent)` without the dashes.
- `root { }` without the colon.

NOTE

The student's `:root` block is carried to Days 3 onwards automatically. If a student picks a dark `--color-bg`, later pages will be hard to read: suggest a light one.

A CORRECT ANSWER

```
/* style.css: how ShowShelf looks */

:root {
  --color-accent: #5b3fd6;
  --color-bg: #f6f5fb;
}

body {
  background-color: var(--color-bg);
  color: #1c1b29;
  font-family: system-ui, Arial, sans-serif;
/* ... unchanged ... */

h1, h2 {
  color: var(--color-accent);
}

/* ... unchanged ... */

a {
  color: var(--color-accent);
}

/* ... unchanged ... */
```

Extra 1. Dark mode ~20 min

Teaches: `@media (prefers-color-scheme: dark)` for a dark theme.

TASK

Add a `@media (prefers-color-scheme: dark)` block that sets new, dark values for `--color-bg`, plus a light text colour. Tip: add a `--color-text` variable and use it in `body` for this.

A CORRECT ANSWER

```
/* ... unchanged ... */
  --color-accent: #5b3fd6;
  --color-bg: #f6f5fb;
  --color-text: #1c1b29;
}

@media (prefers-color-scheme: dark) {
  :root {
    --color-bg: #12111a;
    --color-text: #f1f0f7;
    --color-accent: #a18bff;
  }
}

body {
  background-color: var(--color-bg);
  color: var(--color-text);
  font-family: system-ui, Arial, sans-serif;
  line-height: 1.6;
/* ... unchanged ... */
```

Extra 2. A poster that moves ~15 min

Teaches: `transform` and `transition` for small animations.

TASK

Make the poster grow a little and tilt when the mouse is over it, smoothly.

A CORRECT ANSWER

```
/* ... unchanged ... */
  width: 200px;
  border-radius: 8px;
  transition: transform 0.2s;
}

.poster:hover {
  transform: scale(1.05) rotate(-2deg);
}

.site-header {
  background-color: #ffffff;
/* ... unchanged ... */
```

Extra 3. Small, spaced headings ~10 min

Teaches: `text-transform` and `letter-spacing` style headings.

TASK

Make the `h2` headings uppercase, a bit smaller (`16px`) and spaced out (`letter-spacing: 1px`). Keep the `h1` as it is.

A CORRECT ANSWER

```
/* ... unchanged ... */
  text-align: left;
}

h2 {
  font-size: 16px;
  text-transform: uppercase;
  letter-spacing: 1px;
}
```

Day 3: Lay out the home page Flexbox, Grid and phones

Yesterday you gave a page colours and fonts. Today you decide **where things go**. You build the layout of the ShowShelf **home page**: a top bar, a search bar and a grid of show cards.

The HTML is ready, and so are the colours, fonts and the look of each card: they're in **base.css** (open its tab above the code box to see it). But base.css has no layout at all, so right now everything is stacked in one long column. Your job is **layout.css**.

You'll use three tools that almost every website uses:

- **Flexbox** to put things in a row: the top bar and the search bar
- **Grid** to arrange the cards in columns
- **Media queries** to show more columns on bigger screens

Your colours from Day 2 are already in the page. Open the **index.html (whole page)** tab any time to see the class names you can use.

Step 1. A centred container ~10 min

Teaches: A `.container` with `max-width`, `margin: 0 auto` and side `padding` centres the page.

TASK

In layout.css, write a `.container` rule with:

1. `max-width: 1200px;`
2. `margin: 0 auto;`
3. `padding: 0 16px;`

COMMON MISTAKES

- `container { }` without the dot.
- `padding: 16px 0` (values swapped): space above and below, but the text still touches the sides.
- `width: 1200px` instead of `max-width`: the page scrolls sideways on a phone.

NOTE

A quick warm-up from Day 2. The preview is narrow, so the centring only shows in the **Bigger** view; the padding is visible straight away.

A CORRECT ANSWER

```
/* layout.css: where things go on the page */

.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 16px;
}
```

Step 2. Flexbox: a row ~15 min

Teaches: `display: flex` on a parent puts its direct children in a row.

TASK

Put the logo and the menu side by side: give `.header-inner` the declaration `display: flex;`.

COMMON MISTAKES

- Putting `display: flex` on the children (`.logo`, `.nav`) instead of the parent.
- Using `.site-header`: its only child is the container, so nothing moves.
- `display: flexbox`.

NOTE

If a student is lost, open the whole page tab together and draw the boxes: header, then container, then logo and nav. "Which box holds the two things we want in a row?"

A CORRECT ANSWER

```
/* ... unchanged ... */
  padding: 0 16px;
}

.header-inner {
  display: flex;
}
```

Step 3. Logo left, menu right ~15 min

Teaches: `justify-content` (along the row) and `align-items` (across it) place flex items.

TASK

In your `.header-inner` rule, add:

1. `justify-content: space-between;` so the logo is on the left and the menu on the right
2. `align-items: center;` so the small menu text lines up with the middle of the bigger logo

COMMON MISTAKES

- `justify-content: centre` or `space between` (with a space).
- Putting the two properties on `.nav` or `.logo` instead of the flex container.
- Mixing up the two: `align-items: space-between`.

A CORRECT ANSWER

```
/* ... unchanged ... */
.header-inner {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

Step 4. Space between the menu links ~10 min

Teaches: `gap` sets the space between flex items. A box can be a flex item and a flex container.

TASK

Write a `.nav` rule that:

1. makes the nav a flex container
2. puts `16px` of `gap` between the links

COMMON MISTAKES

- `gap` on `.nav` without `display: flex`: nothing happens.
- `gap` on `.header-inner`: it only changes the space between the logo and the nav.
- `margin` on every link, which also adds space after the last one.

A CORRECT ANSWER

```
/* ... unchanged ... */
  align-items: center;
}

.nav {
  display: flex;
  gap: 16px;
}
```

Step 5. The hero: centred text ~10 min

Teaches: `text-align: center` on a parent centres all the text inside it.

TASK

Write a `.hero` rule with:

1. `text-align: center;`
2. `padding: 40px 0 28px;` (40px above, 28px below)

COMMON MISTAKES

- `text-align: centre`.
- Centring `h1` and `p` separately (works, but one rule on `.hero` is enough).

A CORRECT ANSWER

```
/* ... unchanged ... */
  gap: 16px;
}

.hero {
  text-align: center;
  padding: 40px 0 28px;
}
```

Step 6. A search bar that stretches ~15 min

Teaches: `flex: 1` on a flex item makes it grow to fill the free space.

TASK

1. Write a `.search` rule with `display: flex;` and `gap: 8px;`.
2. Write a `.search input` rule with `flex: 1;`.
3. In `.search`, add `max-width: 520px;` and `margin: 0 auto;` so the search bar stays a sensible size and centred on big screens.

COMMON MISTAKES

- `flex: 1` on `.search` (the form) instead of on the input.
- `.searchinput` or `.search.input` instead of `.search input` (with a space).
- Forgetting `display: flex` on the form, so `flex: 1` has no effect.

NOTE

Press **Bigger** with them and resize the new window: only the input grows. That picture makes `flex: 1` click.

A CORRECT ANSWER

```
/* ... unchanged ... */
  padding: 40px 0 28px;
}

.search {
  display: flex;
  gap: 8px;
  max-width: 520px;
  margin: 0 auto;
}

.search input {
  flex: 1;
}
```

Step 7. Grid: cards in columns ~20 min

Teaches: `display: grid` with `grid-template-columns: repeat(2, 1fr)` makes two equal columns; `gap` spaces them.

TASK

The six cards are inside `<div class="grid">`. Make it a grid with **2 equal columns** (the right number for a phone) and `16px` of gap.

COMMON MISTAKES

- `repeat(2 1fr)` without the comma, or `repeat(2, 1 fr)` with a space: the line is ignored.
- `grid-template-column` (no s).
- `grid-template-columns: 2;` (a number alone isn't a column size).

NOTE

Good moment to ask: "What would `1fr 2fr` do?" Let them try it, then go back to `repeat(2, 1fr)`.

A CORRECT ANSWER

```
/* ... unchanged ... */
  flex: 1;
}

.grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 16px;
}
```

Step 8. Posters that fill the card ~15 min

Teaches: `width: 100%`, `aspect-ratio: 2 / 3` and `object-fit: cover` give every poster the same shape.

TASK

Write a `.poster` rule with `width: 100%;`, `aspect-ratio: 2 / 3;` and `object-fit: cover;`.

COMMON MISTAKES

- `aspect-ratio: 3 / 2` (a wide picture instead of a tall one).
- `height: 300px` instead of `aspect-ratio`: the shape changes when the column width changes.
- `object-fit: fill` or `contain` (stretched, or empty bars on the sides).

NOTE

The placeholder poster already has a 2:3 shape, so `object-fit` shows no visible change today. Later in the course, with real pictures from TVmaze, it matters. To demo it, temporarily set `aspect-ratio: 1 / 1` and switch `object-fit` between `cover` and `fill`.

A CORRECT ANSWER

```
/* ... unchanged ... */
  gap: 16px;
}

.poster {
  width: 100%;
  aspect-ratio: 2 / 3;
  object-fit: cover;
}
```

Step 9. Phones first, then bigger screens ~20 min

Teaches: Mobile first: phone CSS first, then `@media (min-width: 600px) { ... }` for bigger screens.

TASK

At the bottom of layout.css, add a media query for screens **at least 600px** wide, with a `.grid` rule inside it that makes **3 columns**.

Then press **Bigger** and try it.

COMMON MISTAKES

- `max-width` instead of `min-width`.
- Missing the second `}`, so the rest of the file ends up inside the media query.
- `@media min-width: 600px` without round brackets, or `600` without `px`.
- A property directly inside `@media` without a selector: `@media (...) { grid-template-columns: ...; }`.

NOTE

Students often think their code is broken because the preview doesn't change. It's working: the preview is narrower than 600px. Show them the **Bigger** tab and resizing the window. (If the Lab window itself is small, the preview may be wider and show 3 columns directly.)

A CORRECT ANSWER

```
/* ... unchanged ... */
  object-fit: cover;
}

@media (min-width: 600px) {
  .grid {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Step 10. A second breakpoint for laptops ~15 min

Teaches: Several breakpoints, from small to big; one media query can hold several rules.

TASK

Under your 600px media query, add one for **900px and wider** with two rules:

1. `.grid` with **4 columns**
2. `.hero h1` with a bigger `font-size`: `2.6rem`

Press **Bigger** and make the window wide to see it.

COMMON MISTAKES

- The 900px query above the 600px one: on a laptop the 600px rule wins, and you get 3 columns.
- The `.hero h1` rule placed after the media query's last `}` (then it applies on phones too).

A CORRECT ANSWER

```
/* ... unchanged ... */
  }
}

@media (min-width: 900px) {
  .grid {
    grid-template-columns: repeat(4, 1fr);
  }
  .hero h1 {
    font-size: 2.6rem;
  }
}
```

Step 11. Cards that lift ~15 min

Teaches: `transform`, `box-shadow` and `transition` on `:hover` make cards lift smoothly.

TASK

1. Write a `.card` rule with `transition: transform 0.15s, box-shadow 0.15s;`.
2. Write a `.card:hover` rule with `transform: translateY(-4px);` and a `box-shadow` like `0 8px 20px rgba(0, 0, 0, 0.12)`.

Move your mouse over the cards in the preview.

COMMON MISTAKES

- `transition` only in `.card:hover`: smooth on the way up, but it jumps back.
- `translateY(4px)` (positive): the card moves down.
- `.card :hover` with a space.

A CORRECT ANSWER

```
/* ... unchanged ... */
  }
}

.card {
  transition: transform 0.15s, box-shadow 0.15s;
}

.card:hover {
  transform: translateY(-4px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.12);
}
```

Extra 1. A top bar that stays ~10 min

Teaches: `position: sticky; top: 0;` keeps an element in view while scrolling.

TASK

Make the whole top bar (`.site-header`) sticky at the top. Then scroll the preview to try it.

A CORRECT ANSWER

```
/* ... unchanged ... */
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.12);
}

.site-header {
  position: sticky;
  top: 0;
  z-index: 10;
}
```

Extra 2. Six columns on big screens ~10 min

Teaches: A 1200px breakpoint for big screens, placed last.

TASK

Add a media query for `min-width: 1200px` that gives the grid **6 columns**. Check it with **Bigger** on a wide window.

A CORRECT ANSWER

```
/* ... unchanged ... */
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.12);
}

@media (min-width: 1200px) {
  .grid {
    grid-template-columns: repeat(6, 1fr);
  }
}
```

Extra 3. A grid without media queries ~15 min

Teaches: `repeat(auto-fill, minmax(160px, 1fr))` fits as many columns as there's room for.

TASK

1. In your normal `.grid` rule, replace `repeat(2, 1fr)` with `repeat(auto-fill, minmax(160px, 1fr))`.
2. Delete the `.grid` rules inside your media queries (they would overrule it). Keep the `.hero h1` rule.

Try it with **Bigger** and resize the window: the columns appear one by one.

A CORRECT ANSWER

```
/* ... unchanged ... */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(160px, 1fr));
  gap: 16px;
}
/* ... unchanged ... */



@media (min-width: 900px) {
  .hero h1 {
/* ... unchanged ... */
```

Day 4: JavaScript: first steps

Values, variables, maths

Today you write your first **JavaScript**, the language that makes a page *do* things. HTML says what things are, CSS says how they look, and JavaScript gives the browser instructions.

You work on the Friends page again. It's styled with the ShowShelf look (and your colours from Day 2), but some parts are empty: grey notes say what's missing. By the end of the day your JavaScript fills them in: the title, the facts, how long it takes to watch every episode, and a message that changes depending on the numbers.

Your code goes in a file called **app.js**. Keep an eye on the **Console** under the preview: it's where JavaScript talks back to you.

Go slowly. Every step is small on purpose, and every step saves automatically.

Step 1. Hello, Console

~10 min

Teaches: `console.log("...")` writes a message in the Console. Code runs from top to bottom.

TASK

1. Under the comments in the code box, write a `console.log` that says **Hello, ShowShelf!**
2. On the next line, write a second `console.log` with a sentence about a show you like, for example "My favourite show is Dark."

Look at the Console: two lines, in the same order as your code.

COMMON MISTAKES

- `Console.log` with a capital C, or `console.Log`.
- Forgetting the quotes: `console.log(Hello)`.
- Looking for the message on the page instead of in the Console.

NOTE

If the Console panel is hidden, click its title bar under the preview. Show students that the order of the lines in the Console follows the order in the code.

A CORRECT ANSWER

```
// app.js: the JavaScript for the Friends page.
// Lines that start with // are comments. JavaScript skips them.

console.log("Hello, ShowShelf!");
console.log("My favourite show is Dark.");
```

Step 2. Text and numbers ~15 min

Teaches: Strings are text in quotes, numbers have no quotes. `+` adds numbers but joins strings.

TASK

Under your two hello lines:

1. Log `236 + 62` (the episodes of Friends plus the episodes of Breaking Bad). No quotes.
2. Log `"236" + 62`, with quotes around the first number only. What do you get, and why?

COMMON MISTAKES

- Quotes around the whole sum: `console.log("236 + 62")` prints the text itself.
- A comma in decimals: `8,7` instead of `8.7`.

A CORRECT ANSWER

```
// ... unchanged ...
console.log("Hello, ShowShelf!");
console.log("My favourite show is Dark.");

console.log(236 + 62);
console.log("236" + 62);
```

Step 3. Variables with const ~15 min

Teaches: `const name = value;` creates a variable. Use its name without quotes.

TASK

Under your other lines, create three variables about Friends:

1. `showName` with the text `"Friends"`
2. `episodes` with the number `236`
3. `runtime` with the number `30` (minutes per episode)

Then log `showName` and log `episodes`.

COMMON MISTAKES

- Quotes around the variable when using it: `console.log("showName")`.
- Quotes around numbers: `const episodes = "236";`.
- A space in the name: `const show name`.

A CORRECT ANSWER

```
// ... unchanged ...
console.log(236 + 62);
console.log("236" + 62);

const showName = "Friends";
const episodes = 236;
const runtime = 30;

console.log(showName);
console.log(episodes);
```

Step 4. Values that change: let ~15 min

Teaches: `let` for values that change. Change them with `name = new value;`.

TASK

1. Create `episodesWatched` with `let`, starting at `10`, and log it.
2. On a new line, add 1 to it. Then log it again. The Console should show 10, then 11.
3. Try this: change `let` to `const` and read the red error. Then change it back.

COMMON MISTAKES

- Writing `let` again on the second line: "has already been declared".
- Writing `episodesWatched + 1;` alone (it calculates 11 but doesn't store it).
- Using `const` and getting "Assignment to constant variable".

A CORRECT ANSWER

```
// ... unchanged ...
console.log(showName);
console.log(episodes);

let episodesWatched = 10;
console.log(episodesWatched);

episodesWatched = episodesWatched + 1;
console.log(episodesWatched);
```

Step 5. Maths with variables ~20 min

Teaches: Maths: `+ - * /`, results in variables. `Math.round()` rounds to a whole number.

TASK

Under your code, using your variables `episodes` and `runtime`:

1. Create `totalMinutes`: episodes times runtime. Log it.
2. Create `hours`: totalMinutes divided by 60. Log it.
3. Create `days`: hours divided by 24. Log it. (Lots of decimals!)
4. Log `Math.round(days)`.

COMMON MISTAKES

- `math.round` with a small m.
- `x` instead of `*` for times.
- Typing the numbers (`236 * 30`) instead of the variable names.

A CORRECT ANSWER

```
// ... unchanged ...
episodesWatched = episodesWatched + 1;
console.log(episodesWatched);

const totalMinutes = episodes * runtime;
console.log(totalMinutes);

const hours = totalMinutes / 60;
console.log(hours);

const days = hours / 24;
console.log(days);
console.log(Math.round(days));
```

Step 6. Sentences with backticks ~15 min

Teaches: Template literals: text between backticks, with `${variable}` for the values.

TASK

Create a variable `sentence` with a template literal that makes this text from your variables:

Watching all 236 episodes of Friends takes 118 hours.

Use `${episodes}`, `${showName}` and `${hours}` for the parts that come from variables. Then log `sentence`.

COMMON MISTAKES

- Normal quotes instead of backticks: the `${ }` is printed as it is.
- `$(hours)` or `{hours}` instead of `${hours}`.
- Forgetting spaces around the `${ }` parts, so words stick together.

A CORRECT ANSWER

```
// ... unchanged ...
console.log(days);
console.log(Math.round(days));

const sentence = `Watching all ${episodes} episodes of ${showName} takes ${hours} hours.`;
console.log(sentence);
```

Step 7. Put text on the page ~20 min

Teaches: `document.querySelector("#id").textContent = value;` puts text on the page.

TASK

At the bottom of your code, fill three elements:

1. `#title` gets `showName`
2. `#meta` gets a template literal: **236 episodes · 30 minutes each** (use `${episodes}` and `${runtime}`)
3. `#watch-time` gets your `sentence`

Watch the grey notes disappear.

COMMON MISTAKES

- Forgetting the `#` in the selector: `querySelector("title")` changes the browser tab's title instead.
- `textcontent` with a small c: no error, but nothing happens.
- Quotes around the variable: the page shows the word *showName*.

A CORRECT ANSWER

```
// ... unchanged ...
const sentence = `Watching all ${episodes} episodes of ${showName} takes ${hours} hours.`;
console.log(sentence);

document.querySelector("#title").textContent = showName;
document.querySelector("#meta").textContent = `${episodes} episodes · ${runtime} minutes each`;
document.querySelector("#watch-time").textContent = sentence;
```

Step 8. True or false ~15 min

Teaches: Booleans are `true` or `false`. Compare with `> < >= <= === !==`.

TASK

1. Create `isLong`: whether `episodes` is greater than 100. Log it.
2. Log `showName === "Friends"`.
3. Log `runtime === "30"`. Why is the answer `false`?

COMMON MISTAKES

- One `=` in a comparison: `showName = "Friends"` tries to change the variable.
- Quotes around `true`: `"true"` is a string, not a boolean.

A CORRECT ANSWER

```
// ... unchanged ...
document.querySelector("#meta").textContent = `${episodes} episodes · ${runtime} minutes each`;
document.querySelector("#watch-time").textContent = sentence;

const isLong = episodes > 100;
console.log(isLong);
console.log(showName === "Friends");
console.log(runtime === "30");
```

Step 9. Choose with if and else ~20 min

Teaches: `if (condition) { ... } else { ... }` runs one of two blocks.

TASK

At the bottom of your code, write an `if / else`:

- If the show is long (`isLong`), put **A long one: get comfortable on the sofa!** in `#verdict`.
- Else, put **A short one: perfect for a weekend.** in `#verdict`.

Then change `episodes` to `10` at the top and watch the message change. Change it back to `236` when you're done.

COMMON MISTAKES

- A semicolon after the condition: `if (isLong); {` (the block then always runs).
- Missing `}` before `else`.
- Putting both messages outside the blocks, so the second one always wins.

NOTE

The check runs the student's code once more with `episodes` set to 5 to test the `else` block, so a hard-coded condition like `if (true)` won't pass.

A CORRECT ANSWER

```
// ... unchanged ...
console.log(showName === "Friends");
console.log(runtime === "30");

if (isLong) {
  document.querySelector("#verdict").textContent = "A long one: get comfortable on the sofa!";
} else {
  document.querySelector("#verdict").textContent = "A short one: perfect for a weekend.";
}
```

Step 10. Read the error, fix the code ~15 min

Teaches: Red errors name the problem and the line. A syntax error stops the whole file.

TASK

We added two broken lines at the bottom of your code. Look at the page: the grey notes are back, because one mistake stopped the whole file.

1. Read the red error and its line number. Fix that mistake.
2. A different error appears. Read it, and fix that one too.

When both are fixed, the badge above the title says **NBC**.

COMMON MISTAKES

- Fixing the line number in the message but not looking at the line above.
- Deleting the broken lines instead of fixing them.

NOTE

Encourage students to read the error out loud. The grey explanation under each error is written for beginners.

A CORRECT ANSWER

```
// ... unchanged ...
  document.querySelector("#verdict").textContent = "A short one: perfect for a weekend.";
}

// Step 10: two mistakes are hiding below. The Console will help you find them.
const network = "NBC";
document.querySelector("#network").textContent = network;
```

Extra 1. Text that knows tricks ~10 min

Teaches: `.length` counts characters; `.toUpperCase()` and `.toLowerCase()` change the case.

TASK

1. Log the number of characters in `showName`.
2. At the very bottom, set `#title` to `showName` in capital letters.

A CORRECT ANSWER

```
// ... unchanged ...
const network = "NBC";
document.querySelector("#network").textContent = network;

console.log(showName.length);
document.querySelector("#title").textContent = showName.toUpperCase();
```

Extra 2. A random episode ~15 min

Teaches: `Math.floor(Math.random() * n) + 1` is a random whole number from 1 to n.

TASK

1. Create `randomEpisode`: a random whole number from 1 to `episodes`.
2. Put **Tonight's random pick: episode 137** (with your random number) in `#random`.

Press **Run again** above the preview a few times: the number changes.

A CORRECT ANSWER

```
// ... unchanged ...
const network = "NBC";
document.querySelector("#network").textContent = network;

const randomEpisode = Math.floor(Math.random() * episodes) + 1;
document.querySelector("#random").textContent = `Tonight's random pick: episode ${randomEpisode}`;
```

Extra 3. Three choices with else if ~15 min

Teaches: `if / else if / else` chooses between several cases, top to bottom.

TASK

1. Create `const rating = 8.8;`.
2. Show a label in `#rating`: ★ **Excellent** if rating is 9 or more, ★ **Very good** if it's 8 or more, and ★ **Worth a try** otherwise.

Try changing the rating to 9.5 and to 7 to see all three labels.

A CORRECT ANSWER

```
// ... unchanged ...
const network = "NBC";
document.querySelector("#network").textContent = network;

const rating = 8.8;

if (rating >= 9) {
  document.querySelector("#rating").textContent = "★ Excellent";
} else if (rating >= 8) {
  document.querySelector("#rating").textContent = "★ Very good";
} else {
  document.querySelector("#rating").textContent = "★ Worth a try";
}
```

Day 5: Functions and decisions Reusable code, if and else

Yesterday you wrote instructions that run once, from top to bottom. Today you package them into **functions**: named pieces of code you can run again and again, with different numbers each time.

The page shows four show cards. Every card has grey notes like *(write watchTime)*. A helper file, **page.js**, is already written: it runs after your code and uses your functions to fill in the cards. So each function you finish makes all four cards light up at once.

You'll also learn the most important difference in today's lesson: a function can *print* a value, or *give it back*. Only one of them fills the cards.

When the cards are full, keep going with the **practice exercises** (P1 to P36): small functions of their own, from one line to a few decisions. Do them before the extra challenges.

Step 1. Your first function ~15 min

Teaches: `function name() { ... }` defines a function; `name();` calls it.

TASK

1. Define a function called `sayHello` that logs **Hello from a function!**
2. Under it, call it twice.

COMMON MISTAKES

- Defining the function but never calling it (nothing appears).
- Calling it without brackets: `sayHello;` does nothing.
- Putting the calls inside the function's own `{ }`.

A CORRECT ANSWER

```
// app.js: your functions. page.js uses them to fill in the cards.

function sayHello() {
  console.log("Hello from a function!");
}

sayHello();
sayHello();
```

Step 2. Parameters and arguments ~15 min

Teaches: Parameters are named in the definition; arguments are the values in the call.

TASK

1. Define a function `greet` with one parameter, `name`. It logs a greeting with the name, like **Hello, Ana!**
2. Call it twice, with two different names.

COMMON MISTAKES

- Quotes around the parameter inside the function: `"Hello, name!"`.
- Calling with no argument: `greet()` prints *Hello, undefined!*

NOTE

If a student asks about `undefined`: it's JavaScript's word for "no value yet". It becomes important in the next two steps.

A CORRECT ANSWER

```
// ... unchanged ...
sayHello();
sayHello();

function greet(name) {
  console.log(`Hello, ${name}!`);
}

greet("Ana");
greet("Ben");
```

Step 3. Two parameters ~15 min

Teaches: Several parameters are separated by commas; arguments go in the same order.

TASK

1. Define `watchTime` with two parameters, `episodes` and `runtime`. It logs the hours: episodes times runtime, divided by 60.
2. Call it for Friends, `watchTime(236, 30)`, and for Breaking Bad, `watchTime(62, 60)`.

Then look closely:

- The Console shows more numbers than you asked for. Where do the others come from?
- The cards say *Watch it all: undefined hours*. Why? The next step explains.

COMMON MISTAKES

- Arguments in the wrong order (the result is the same here, but not in general).
- Using the numbers inside the function instead of the parameters: `console.log(236 * 30 / 60)`.

A CORRECT ANSWER

```
// ... unchanged ...
greet("Ana");
greet("Ben");

function watchTime(episodes, runtime) {
  console.log(episodes * runtime / 60);
}

watchTime(236, 30);
watchTime(62, 60);
```

Step 4. Give the answer back with return ~15 min

Teaches: `return` gives a value back to the caller; `console.log` only shows it to you.

TASK

1. Change `watchTime` so that it **returns** the hours, rounded with `Math.round`, instead of logging them.
2. Your two calls now print nothing. Replace them with one line that logs the answer: `console.log(watchTime(62, 60));`.

Look at the cards: the watch times appear.

COMMON MISTAKES

- Keeping `console.log` inside and adding `return` after it (fine, but remove the log).
- `return` on its own line with the value on the next line: JavaScript returns `undefined`.
- Writing lines after `return` and wondering why they never run.

NOTE

This is the key idea of the day. If a student is unsure, ask: "Who needs the answer, you or page.js?"

A CORRECT ANSWER

```
// ... unchanged ...

function watchTime(episodes, runtime) {
  return Math.round(episodes * runtime / 60);
}

console.log(watchTime(62, 60));
```

Step 5. Decisions inside a function ~20 min

Teaches: Functions can use `if / else if / else` and return a different value in each block.

TASK

Write `ratingLabel(rating)` that returns:

- **Must watch** for 9 or more
- **Good** for 8 or more
- **OK** for 6 or more
- **Skip** for anything lower

Test it with `console.log(ratingLabel(6.5));` and `console.log(ratingLabel(3));`. All our shows are rated 8 or more, so only the Console shows the last two labels.

COMMON MISTAKES

- The conditions in the wrong order (`>= 6` first catches everything).
- `>` instead of `>=`: a rating of exactly 9 becomes Good.
- `console.log` in each block instead of `return`.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(watchTime(62, 60));

function ratingLabel(rating) {
  if (rating >= 9) {
    return "Must watch";
  } else if (rating >= 8) {
    return "Good";
  } else if (rating >= 6) {
    return "OK";
  } else {
    return "Skip";
  }
}

console.log(ratingLabel(6.5));
console.log(ratingLabel(3));
```

Step 6. Compare two parameters ~15 min

Teaches: Conditions can compare parameters with each other: `if (premiered === ended)`.

TASK

Write `yearsLabel(premiered, ended)`:

- when both years are the same, return only one year, like **2019**
- otherwise, return both with a dash between them, like **1994–2004**

The long dash `–` can be hard to type: a normal `-` is fine too.

COMMON MISTAKES

- One `=` in the condition: it changes `premiered` and the result is always the second year.
- Returning `"premiered–ended"` in quotes instead of a template literal.

A CORRECT ANSWER

```
// ... unchanged ...
console.log(ratingLabel(6.5));
console.log(ratingLabel(3));

function yearsLabel(premiered, ended) {
  if (premiered === ended) {
    return `${premiered}`;
  } else {
    return `${premiered}–${ended}`;
  }
}
```

Step 7. Hours and minutes ~20 min

Teaches: `Math.floor()` rounds down; `%` is the remainder of a division.

TASK

In `formatRuntime`, replace the three `TODO` comments with an `if / else if / else` that returns:

1. `"45 min"` style when `hours` is 0
2. `"1 h"` style when `rest` is 0
3. `"1 h 30 min"` style otherwise

Use template literals with `${hours}` and `${rest}`. Test it: `console.log(formatRuntime(90));`.

COMMON MISTAKES

- Checking `minutes === 0` instead of `hours === 0`.
- Forgetting `return` in one of the blocks.

NOTE

If `%` is confusing, use a clock: 90 minutes is 1 hour and 30 left over. `90 % 60` gives the left-over part.

A CORRECT ANSWER

```
// ... unchanged ...
  }
}

function formatRuntime(minutes) {
  const hours = Math.floor(minutes / 60);
  const rest = minutes % 60;
  if (hours === 0) {
    return `${rest} min`;
  } else if (rest === 0) {
    return `${hours} h`;
  } else {
    return `${hours} h ${rest} min`;
  }
}

console.log(formatRuntime(90));
```

Step 8. Functions that use functions ~15 min

Teaches: Functions can call other functions, and can return `true` or `false`.

TASK

Write `isQuickWatch(episodes, runtime)`. It returns `true` when the whole show takes **10 hours or less**, and `false` otherwise.

Use your `watchTime` function inside it. Only one of the four shows is a quick watch. Can you guess which one before you look?

COMMON MISTAKES

- Returning the strings `"true"` / `"false"` instead of booleans.
- Copying the maths from `watchTime` instead of calling it.
- `< 10` instead of `<= 10`.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(formatRuntime(90));

function isQuickWatch(episodes, runtime) {
  return watchTime(episodes, runtime) <= 10;
}
```

Step 9. Arrow functions ~15 min

Teaches: Arrow functions: `const name = (a, b) => value;`

TASK

Rewrite `watchTime` as an arrow function, on one line:

1. Write `const watchTime = (episodes, runtime) => ...` with the calculation (and `Math.round`) after the arrow.
2. Delete the old `function watchTime ...` block, from `function` to its `}`. Two functions with the same name cause an error.

The cards and the Console should look exactly the same as before.

COMMON MISTAKES

- Keeping both versions: "Identifier 'watchTime' has already been declared".
- `{ }` around the body without `return`: the function returns `undefined`.
- Putting the arrow function below the line that calls it.

A CORRECT ANSWER

```
// ... unchanged ...
greet("Ben");

const watchTime = (episodes, runtime) => Math.round(episodes * runtime / 60);

console.log(watchTime(62, 60));
// ... unchanged ...
```

Step 10. A default value ~20 min

Teaches: Default values: `function f(a, b = "x")` uses `"x"` when `b` is left out.

TASK

Write `plural(count, word = "episode")`:

- when `count` is 1, return the count and the word: **1 episode**
- otherwise, return the count, the word and an **s**: **236 episodes**

page.js calls `plural(episodes)` with one argument, so the default is used. Test the other word in the Console: `console.log(plural(3, "season"));` should print **3 seasons**.

COMMON MISTAKES

- `word == "episode"` or `word: "episode"` in the brackets instead of `word = "episode"`.
- Writing `episodes` in the text instead of `${word}s`.

NOTE

All six grey notes should now be filled. Students who finish now go on to the practice exercises P1 to P36 (the Next button leads there), and then the extra challenges.

A CORRECT ANSWER

```
// ... unchanged ...
  return watchTime(episodes, runtime) <= 10;
}

function plural(count, word = "episode") {
  if (count === 1) {
    return `${count} ${word}`;
  } else {
    return `${count} ${word}s`;
  }
}

console.log(plural(3, "season"));
```

Practice 1. Episodes left to watch ~5 min

Teaches: Practice: return a calculation made from two parameters.

TASK

Write `episodesLeft(total, watched)`. It returns how many episodes are still left to watch.

Friends has 236 episodes, so after 10 of them, `episodesLeft(236, 10)` gives **226**.

Write your line where it says `// your code here`, then watch the ticks in the preview.

COMMON MISTAKES

- `watched - total`, which gives negative numbers.
- `console.log` instead of `return` (the gym asks "Did you forget return?").

NOTE

Practice exercises have their own code box on a separate page, the practice gym, so they never change the cards from the guided steps. Students can skip any of them. Every `tryIt` line becomes a row with a tick or a cross; the Check button tests more values than the rows show.

A CORRECT ANSWER

```
// Practice 1: how many episodes are left to watch?
// episodesLeft(236, 10) should give 226.
function episodesLeft(total, watched) {
  return total - watched;
}

tryIt(episodesLeft, [236, 10], 226);
tryIt(episodesLeft, [62, 62], 0);
tryIt(episodesLeft, [5, 0], 5);
```

Practice 2. Shout the title ~5 min

Teaches: Practice: string methods inside an arrow function.

TASK

Finish the arrow function `shout(title)`. It returns the title in capital letters: `shout("Dark")` gives **"DARK"**.

Keep the curly brackets and write `return` inside, or, if you like, rewrite it on one line without `{ }` and `return`.

COMMON MISTAKES

- Forgetting the brackets: `title.toUpperCase` gives the method itself, not the result.
- Quotes around the parameter: `"title".toUpperCase()` always gives `"TITLE"`.
- Curly brackets without `return`.

A CORRECT ANSWER

```
// Practice 2: the title in capital letters.
// shout("Dark") should give "DARK".
const shout = (title) => {
  return title.toUpperCase();
};

tryIt(shout, ["Dark"], "DARK");
tryIt(shout, ["Breaking Bad"], "BREAKING BAD");
tryIt(shout, [""], "");
```

Practice 3. How old is the show? ~6 min

Teaches: Practice: a default value in an arrow function.

TASK

Write the arrow function `showAge(premiered, thisYear)`. It returns how many years ago the show started.

1. Give `thisYear` a default value of `2026`.
2. Return the difference between the two years.

Friends premiered in 1994, so `showAge(1994)` gives **32**, and `showAge(1994, 2004)` gives **10**.

COMMON MISTAKES

- No default, so `showAge(1994)` gives `NaN` (2026 is missing: undefined minus 1994).
- `premiered - thisYear`, which gives negative numbers.

NOTE

The default year 2026 is written into the task and the check. If you teach this course in a later year, students may ask why: it's only practice for default values.

A CORRECT ANSWER

```
// Practice 3: how many years ago did the show start?
// showAge(1994) should give 32 (this year is 2026).
const showAge = (premiered, thisYear = 2026) => {
  return thisYear - premiered;
};

tryIt(showAge, [1994], 32);
tryIt(showAge, [1994, 2004], 10);
tryIt(showAge, [2019, 2019], 0);
```

Practice 4. Does the title fit on the card? ~6 min

Teaches: Practice: `.length` in a comparison that returns `true` or `false`.

TASK

A card has room for 12 characters on one line. Write `fitsOnCard(title)`: it returns `true` when the title has **12 characters or fewer**, and `false` otherwise.

"Breaking Bad" has exactly 12 characters (count the space), so it fits.

COMMON MISTAKES

- `title.length()` with brackets: "title.length is not a function".
- `< 12` instead of `<= 12`, so "Breaking Bad" doesn't fit.
- Returning `"true"` in quotes, which is text, not a boolean.

A CORRECT ANSWER

```
// Practice 4: does the title fit in 12 characters?
// fitsOnCard("Dark") should give true.
function fitsOnCard(title) {
  return title.length <= 12;
}

tryIt(fitsOnCard, ["Dark"], true);
tryIt(fitsOnCard, ["Breaking Bad"], true);
tryIt(fitsOnCard, ["Game of Thrones"], false);
tryIt(fitsOnCard, [""], true);
```

Practice 5. One sentence about a show ~7 min

Teaches: Practice: a template literal built from four parameters.

TASK

Write `aboutShow(name, episodes, runtime, network)`. It returns one sentence like this:

Fleabag has 12 episodes of 30 minutes on BBC Three.

Use all four parameters, so the same function works for every show.

COMMON MISTAKES

- Normal quotes instead of backticks: the `${ }` parts appear as they are.
- Typing "Fleabag" into the sentence instead of `${name}`.
- A missing space around a `${ }` part, so words stick together.

A CORRECT ANSWER

```
// Practice 5: one sentence about a show, from four parameters.
// aboutShow("Fleabag", 12, 30, "BBC Three") should give
// "Fleabag has 12 episodes of 30 minutes on BBC Three."
function aboutShow(name, episodes, runtime, network) {
  return `${name} has ${episodes} episodes of ${runtime} minutes on ${network}.`;
}

tryIt(aboutShow, ["Fleabag", 12, 30, "BBC Three"], "Fleabag has 12 episodes of 30 minutes on BBC
Three.");
tryIt(aboutShow, ["Dark", 26, 60, "Netflix"], "Dark has 26 episodes of 60 minutes on Netflix.");
```

Practice 6. Stars out of five ~6 min

Teaches: Practice: `Math.round` around a calculation.

TASK

Write the arrow function `stars(rating)`. It returns the rating divided by 2 and rounded to a whole number.

Friends is rated 8.8, so `stars(8.8)` gives **4**. Breaking Bad's 9.2 gives **5**.

COMMON MISTAKES

- `Math.round(rating) / 2`: rounds first, divides second, and gives 4.5.
- `Math.floor` instead of `Math.round`: 9.0 becomes 4 stars.

A CORRECT ANSWER

```
// Practice 6: a rating out of 10 as whole stars out of 5.
// stars(8.8) should give 4.
const stars = (rating) => Math.round(rating / 2);

tryIt(stars, [8.8], 4);
tryIt(stars, [9.2], 5);
tryIt(stars, [0], 0);
```

Practice 7. How much have I watched? ~8 min

Teaches: Practice: percentages with `/`, `*` and `Math.round`.

TASK

Write `percentWatched(watched, total)`. It returns the percentage of the show you've watched, rounded to a whole number.

Breaking Bad has 62 episodes, so `percentWatched(31, 62)` gives **50**. Friends has 236, so 10 of them is **4**.

COMMON MISTAKES

- Forgetting `* 100`, so the answer is 0.5 instead of 50.
- `total / watched`, the wrong way round.
- No rounding, so 10 of 236 gives 4.237...

A CORRECT ANSWER

```
// Practice 7: what percentage of the show have you watched?
// percentWatched(31, 62) should give 50.
function percentWatched(watched, total) {
  return Math.round(watched / total * 100);
}

tryIt(percentWatched, [31, 62], 50);
tryIt(percentWatched, [10, 236], 4);
tryIt(percentWatched, [0, 26], 0);
tryIt(percentWatched, [12, 12], 100);
```

Practice 8. Classic or modern? ~6 min

Teaches: Practice: `if / else` with a boundary value.

TASK

Write `era(year)`. It returns **"Classic"** for shows that premiered before 2000, and **"Modern"** for 2000 and later.

So Friends (1994) is Classic, and Band of Brothers (2001) is Modern. A show from exactly 2000 is Modern.

COMMON MISTAKES

- `<= 2000`, so 2000 becomes Classic.
- `console.log("Classic")` instead of `return "Classic"`.

A CORRECT ANSWER

```
// Practice 8: "Classic" before 2000, "Modern" from 2000 on.
// era(1994) should give "Classic".
function era(year) {
  if (year < 2000) {
    return "Classic";
  } else {
    return "Modern";
  }
}

tryIt(era, [1994], "Classic");
tryIt(era, [2008], "Modern");
tryIt(era, [2000], "Modern");
```

Practice 9. Can I finish it tonight? ~8 min

Teaches: Practice: return a comparison that has a calculation on one side.

TASK

Write `canFinishTonight(episodes, runtime, minutesFree)`. It returns `true` when all the episodes, one after another, fit into the minutes you have free, and `false` otherwise.

Chernobyl has 5 episodes of 60 minutes, so 300 free minutes are exactly enough: `canFinishTonight(5, 60, 300)` gives **true**.

COMMON MISTAKES

- `<` instead of `<=`: 5 episodes of 60 minutes don't "fit" into 300 minutes.
- Comparing only `runtime` with `minutesFree`, forgetting the number of episodes.

A CORRECT ANSWER

```
// Practice 9: do all the episodes fit into your free evening?
// canFinishTonight(5, 60, 300) should give true.
function canFinishTonight(episodes, runtime, minutesFree) {
  return episodes * runtime <= minutesFree;
}

tryIt(canFinishTonight, [5, 60, 300], true);
tryIt(canFinishTonight, [4, 30, 90], false);
tryIt(canFinishTonight, [0, 60, 0], true);
```

Practice 10. Minutes as hours and minutes ~8 min

Teaches: Practice: `Math.floor` and `%` to split minutes.

TASK

Write `hoursAndMinutes(minutes)`. It **always** returns both parts, like **"2 h 15 min"**, even when one of them is 0: `hoursAndMinutes(45)` gives **"0 h 45 min"**.

Watching all of The Office takes 6030 minutes. What does your function make of that?

COMMON MISTAKES

- `Math.round(minutes / 60)`: 6030 minutes becomes 101 hours.
- No `Math.floor` at all, so the hours have decimals: "2.25 h".
- Normal quotes instead of backticks.

A CORRECT ANSWER

```
// Practice 10: minutes as "2 h 15 min", always with both parts.
// hoursAndMinutes(135) should give "2 h 15 min".
function hoursAndMinutes(minutes) {
  const hours = Math.floor(minutes / 60);
  const rest = minutes % 60;
  return `${hours} h ${rest} min`;
}

tryIt(hoursAndMinutes, [135], "2 h 15 min");
tryIt(hoursAndMinutes, [60], "1 h 0 min");
tryIt(hoursAndMinutes, [45], "0 h 45 min");
tryIt(hoursAndMinutes, [6030], "100 h 30 min");
```

Practice 11. Round a rating to one decimal ~8 min

Teaches: Practice: round to one decimal with `Math.round(x * 10) / 10`.

TASK

A rating worked out from many votes can look like 8.7634. Write the arrow function `roundRating(rating)` that rounds it to one decimal: `roundRating(8.76)` gives **8.8**.

COMMON MISTAKES

- `Math.round(rating) / 10` or `Math.round(rating / 10)`: the brackets in the wrong place.
- Forgetting `/ 10` at the end, so 8.76 becomes 88.
- `rating.toFixed(1)`, which gives the text "8.8".

A CORRECT ANSWER

```
// Practice 11: round a rating to one decimal.
// roundRating(8.76) should give 8.8.
const roundRating = (rating) => Math.round(rating * 10) / 10;

tryIt(roundRating, [8.76], 8.8);
tryIt(roundRating, [9.04], 9);
tryIt(roundRating, [8.333], 8.3);
```

Practice 12. Not started, watching or finished? ~12 min

Teaches: Practice: four cases with `if / else if / else`, in the right order.

TASK

Write `progressLabel(watched, total)` with four cases:

1. **"Not started"** when `watched` is 0
2. **"Finished"** when `watched` is `total`
3. **"Last episode"** when exactly one episode is left
4. **"Watching"** otherwise

Think about the order. A one-episode show you haven't started has 0 watched *and* one episode left: it should say "Not started".

COMMON MISTAKES

- Testing "Last episode" before "Not started", so `progressLabel(0, 1)` says "Last episode".
- One `=` instead of `===` in a condition.
- `total - 1 === watched` written as `total - 1 = watched` (an error).

A CORRECT ANSWER

```
// Practice 12: a label for your progress through a show.
// progressLabel(0, 236) should give "Not started".
function progressLabel(watched, total) {
  if (watched === 0) {
    return "Not started";
  } else if (watched === total) {
    return "Finished";
  } else if (watched === total - 1) {
    return "Last episode";
  } else {
    return "Watching";
  }
}

tryIt(progressLabel, [0, 236], "Not started");
tryIt(progressLabel, [1, 236], "Watching");
tryIt(progressLabel, [235, 236], "Last episode");
tryIt(progressLabel, [236, 236], "Finished");
tryIt(progressLabel, [0, 1], "Not started");
```

Practice 13. Price per episode ~10 min

Teaches: Practice: an early `return` for 0, and rounding to two decimals.

TASK

Imagine a box set that costs `price`. Write `pricePerEpisode(price, episodes)`. It returns the price of one episode, rounded to two decimals.

- When `episodes` is 0, return 0.
- A 30-euro Fleabag box set (12 episodes) gives **2.5**. 20 euros for 3 episodes gives **6.67**.

COMMON MISTAKES

- No check for 0: `pricePerEpisode(15, 0)` gives `Infinity`.
- Rounding with 10 instead of 100, so 6.67 becomes 6.7.
- `toFixed(2)`, which gives text.

A CORRECT ANSWER

```
// Practice 13: the price of one episode, rounded to cents.
// pricePerEpisode(30, 12) should give 2.5. With 0 episodes, give 0.
function pricePerEpisode(price, episodes) {
  if (episodes === 0) {
    return 0;
  }
  return Math.round(price / episodes * 100) / 100;
}

tryIt(pricePerEpisode, [30, 12], 2.5);
tryIt(pricePerEpisode, [20, 3], 6.67);
tryIt(pricePerEpisode, [15, 0], 0);
```

Practice 14. Shorten long titles ~10 min

Teaches: Practice: `.slice(0, n)`, `.length` and a default value together.

TASK

Write `shortTitle(title, max)`:

1. Give `max` a default value of `10`.
2. When the title is **longer** than `max`, return its first `max` characters plus `"..."`.
3. Otherwise, return the title as it is.

`shortTitle("Game of Thrones")` gives **"Game of Th..."**. "The Office" has exactly 10 characters, so it stays as it is.

COMMON MISTAKES

- `>=` instead of `>`, so "The Office" (exactly 10) gets dots.
- Writing `10` inside the function instead of `max`, so the second argument is ignored.
- No default, so `shortTitle("Game of Thrones")` keeps the whole title.

A CORRECT ANSWER

```
// Practice 14: cut titles that are longer than max, and add "...".
// shortTitle("Game of Thrones") should give "Game of Th...".
function shortTitle(title, max = 10) {
  if (title.length > max) {
    return title.slice(0, max) + "...";
  } else {
    return title;
  }
}

tryIt(shortTitle, ["Game of Thrones"], "Game of Th...");
tryIt(shortTitle, ["Dark"], "Dark");
tryIt(shortTitle, ["The Office"], "The Office");
tryIt(shortTitle, ["Chernobyl", 5], "Chern...");
```

Practice 15. Old enough, and enough time? ~8 min

Teaches: Practice: `&&` to require two conditions at once.

TASK

Write `canWatchTonight(age, ageLimit, runtime, minutesFree)`. It returns `true` only when **both** are true:

- `age` is at least `ageLimit` (exactly the limit is fine)
- one episode (`runtime`) fits into `minutesFree` (exactly is fine too)

`canWatchTonight(16, 16, 60, 60)` gives **true**.

COMMON MISTAKES

- Only one of the two comparisons, so a too-young viewer gets `true`.
- `>` instead of `>=`, so exactly the age limit isn't allowed.
- Writing `age >= ageLimit && <= minutesFree`: each side needs its own complete comparison.

A CORRECT ANSWER

```
// Practice 15: old enough AND enough time for one episode?
// canWatchTonight(16, 16, 60, 60) should give true.
function canWatchTonight(age, ageLimit, runtime, minutesFree) {
  return age >= ageLimit && runtime <= minutesFree;
}

tryIt(canWatchTonight, [16, 16, 60, 60], true);
tryIt(canWatchTonight, [15, 16, 60, 90], false);
tryIt(canWatchTonight, [30, 18, 60, 45], false);
```

Practice 16. How many evenings? (harder) ~12 min

Teaches: Practice: round up with `Math.floor`, `%` and an `if`.

TASK

You watch `perEvening` episodes every evening. Write `eveningsNeeded(episodes, perEvening)`. It returns how many evenings it takes to watch all the episodes. A last evening with fewer episodes still counts.

Fleabag has 12 episodes: at 3 a night that's **4** evenings. Chernobyl's 5 episodes at 2 a night take **3**. And 0 episodes take **0** evenings.

COMMON MISTAKES

- Always adding 1, so 12 episodes at 3 a night become 5 evenings.
- `Math.round(episodes / perEvening)`: 10 episodes at 3 a night is 3.33, which rounds to 3, one evening short.

NOTE

Students who did the extra challenge *How many days to binge it?* may know `Math.ceil`. `return Math.ceil(episodes / perEvening);` is also correct and passes.

A CORRECT ANSWER

```
// Practice 16 (harder): how many evenings to watch every episode?
// eveningsNeeded(12, 3) should give 4, eveningsNeeded(5, 2) should give 3.
function eveningsNeeded(episodes, perEvening) {
  const full = Math.floor(episodes / perEvening);
  if (episodes % perEvening === 0) {
    return full;
  } else {
    return full + 1;
  }
}

tryIt(eveningsNeeded, [12, 3], 4);
tryIt(eveningsNeeded, [5, 2], 3);
tryIt(eveningsNeeded, [62, 4], 16);
tryIt(eveningsNeeded, [0, 3], 0);
```

Practice 17. Time left, from two helpers (harder) ~15 min

Teaches: Practice: a function that calls two other functions and chooses between three answers.

TASK

Write `timeLeft(watched, total, runtime)`. Use `episodesLeft` to work out how many episodes are left, then return:

1. **"Finished!"** when 0 or fewer are left (yes, people type the wrong numbers)
2. **"Last one! 30 min to go"** when exactly one is left, with `formatRuntime(runtime)` for the time
3. **"2 episodes left: 1 h to go"** otherwise, with `formatRuntime` of the minutes for **all** the episodes left

COMMON MISTAKES

- `formatRuntime(runtime)` in the last case, so the time is for one episode, not all of them.
- `left === 0` instead of `left <= 0`, so watching "too many" episodes gives "-8 episodes left".
- Arguments to `episodesLeft` in the wrong order: it wants `(total, watched)`.

NOTE

The order of the arguments is the trap here: `timeLeft` gets `(watched, total, ...)`, but `episodesLeft` wants `(total, watched)`. Swapped, every show looks finished.

A CORRECT ANSWER

```
// Practice 17 (harder): how much is left? Use the two helpers.
// timeLeft(10, 12, 30) should give "2 episodes left: 1 h to go".

// Helpers, already written (don't change them):
function episodesLeft(total, watched) {
  return total - watched;
}

function formatRuntime(minutes) {
  const hours = Math.floor(minutes / 60);
  const rest = minutes % 60;
  if (hours === 0) {
    return `${rest} min`;
  } else if (rest === 0) {
    return `${hours} h`;
  } else {
    return `${hours} h ${rest} min`;
  }
}

// Your function:
function timeLeft(watched, total, runtime) {
  const left = episodesLeft(total, watched);
  if (left <= 0) {
    return "Finished!";
  } else if (left === 1) {
    return `Last one! ${formatRuntime(runtime)} to go`;
  } else {
    return `${left} episodes left: ${formatRuntime(left * runtime)} to go`;
  }
}

tryIt(timeLeft, [10, 12, 30], "2 episodes left: 1 h to go");
tryIt(timeLeft, [200, 201, 30], "Last one! 30 min to go");
tryIt(timeLeft, [100, 201, 30], "101 episodes left: 50 h 30 min to go");
tryIt(timeLeft, [62, 62, 60], "Finished!");
```

Practice 18. What's next? (harder) ~15 min

Teaches: Practice: three cases from four parameters, with calculations inside `${ }`.

TASK

Write `nextEpisode(season, episode, perSeason, seasons)`. It gets the episode you watched last, the number of episodes per season and the number of seasons, and returns:

1. **"Season 1, episode 4"** when the season has more episodes
2. **"Season 2, episode 1"** when the season is over but there's another season
3. **"That was the finale!"** when it was the last episode of the last season

COMMON MISTAKES

- `${episode} + 1` instead of `${episode + 1}`: the page shows "3 + 1".
- `episode <= perSeason`, so after episode 6 comes "episode 7".
- Testing `season < seasons` first, so the middle of season 1 jumps to season 2.

A CORRECT ANSWER

```
// Practice 18 (harder): which episode comes next?
// Fleabag: 2 seasons of 6 episodes.
// nextEpisode(1, 6, 6, 2) should give "Season 2, episode 1".
function nextEpisode(season, episode, perSeason, seasons) {
  if (episode < perSeason) {
    return `Season ${season}, episode ${episode + 1}`;
  } else if (season < seasons) {
    return `Season ${season + 1}, episode 1`;
  } else {
    return "That was the finale!";
  }
}

tryIt(nextEpisode, [1, 3, 6, 2], "Season 1, episode 4");
tryIt(nextEpisode, [1, 6, 6, 2], "Season 2, episode 1");
tryIt(nextEpisode, [2, 6, 6, 2], "That was the finale!");
tryIt(nextEpisode, [1, 5, 5, 1], "That was the finale!");
```

Practice 19. Minutes to seconds ~5 min

Teaches: Practice: a one-line converter with an arrow function.

TASK

Write the arrow function `toSeconds(minutes)`. It returns the same time in seconds.

A Friends episode lasts 30 minutes, so `toSeconds(30)` gives **1800**.

COMMON MISTAKES

- Dividing by 60 instead of multiplying.
- Curly brackets without `return`.

A CORRECT ANSWER

```
// Practice 19: a converter from minutes to seconds.
// toSeconds(30) should give 1800.
const toSeconds = (minutes) => minutes * 60;

tryIt(toSeconds, [30], 1800);
tryIt(toSeconds, [1], 60);
tryIt(toSeconds, [0], 0);
```

Practice 20. Hours to whole days ~5 min

Teaches: Practice: divide and round down for whole units.

TASK

Write the arrow function `fullDays(hours)`. It returns how many **whole** days fit into the hours.

Watching all of Friends takes 118 hours, so `fullDays(118)` gives **4**: four full days and some hours left over.

COMMON MISTAKES

- `Math.round`, so 23 hours counts as a whole day.
- Multiplying by 24 instead of dividing.

A CORRECT ANSWER

```
// Practice 20: how many whole days fit into some hours?
// fullDays(118) should give 4.
const fullDays = (hours) => Math.floor(hours / 24);

tryIt(fullDays, [118], 4);
tryIt(fullDays, [24], 1);
tryIt(fullDays, [23], 0);
```

Practice 21. Title with its year ~6 min

Teaches: Practice: a string builder with a check for the empty string.

TASK

Write `titleWithYear(name, year)`. It returns the name and the year in brackets: `titleWithYear("Dark", 2017)` gives **"Dark (2017)"**.

When `name` is an empty string, use **Untitled** instead: `titleWithYear("", 2005)` gives **"Untitled (2005)"**.

COMMON MISTAKES

- `${(year)}` or `$(year)` instead of `(${year})`.
- Forgetting the space before the bracket: "Dark(2017)".
- Checking `name === " "` (a space) instead of `""`.

A CORRECT ANSWER

```
// Practice 21: a show's name with its year in brackets.
// titleWithYear("Dark", 2017) should give "Dark (2017)".
function titleWithYear(name, year) {
  if (name === "") {
    return `Untitled (${year})`;
  }
  return `${name} (${year})`;
}

tryIt(titleWithYear, ["Dark", 2017], "Dark (2017)");
tryIt(titleWithYear, ["The Wire", 2002], "The Wire (2002)");
tryIt(titleWithYear, ["", 2005], "Untitled (2005)");
```

Practice 22. Which show came first? ~7 min

Teaches: Practice: compare two shows passed as name and number.

TASK

Write `olderShow(nameA, yearA, nameB, yearB)`. It returns the name of the show that premiered **first**, or **"Same year"** when both started in the same year.

Friends (1994) came before The Office (2005), so `olderShow("Friends", 1994, "The Office", 2005)` gives **"Friends"**.

COMMON MISTAKES

- `>` instead of `<`: the newer show wins.
- Returning `"nameA"` in quotes, which gives the word nameA.
- No draw case, so same-year shows return the second name.

A CORRECT ANSWER

```
// Practice 22: which of two shows premiered first?
// olderShow("Friends", 1994, "The Office", 2005) should give "Friends".
function olderShow(nameA, yearA, nameB, yearB) {
  if (yearA < yearB) {
    return nameA;
  } else if (yearB < yearA) {
    return nameB;
  } else {
    return "Same year";
  }
}

tryIt(olderShow, ["Friends", 1994, "The Office", 2005], "Friends");
tryIt(olderShow, ["Dark", 2017, "The Wire", 2002], "The Wire");
tryIt(olderShow, ["Fleabag", 2016, "The Crown", 2016], "Same year");
```

Practice 23. Who picks tonight? Even or odd ~6 min

Teaches: Practice: even or odd with `% 2`.

TASK

You and a friend take turns choosing what to watch. Write `whoPicks(evening)`:

- odd evenings (1, 3, 5 ...): return **"You"**
- even evenings (2, 4, 6 ...): return **"Your friend"**

COMMON MISTAKES

- `evening / 2 === 0`, which is only true for 0.
- The two labels in the wrong blocks.

A CORRECT ANSWER

```
// Practice 23: odd evenings you pick, even evenings your friend picks.
// whoPicks(1) should give "You".
function whoPicks(evening) {
  if (evening % 2 === 0) {
    return "Your friend";
  } else {
    return "You";
  }
}

tryIt(whoPicks, [1], "You");
tryIt(whoPicks, [2], "Your friend");
tryIt(whoPicks, [61], "You");
```

Practice 24. Is the rating valid? ~6 min

Teaches: Practice: check a range with `>=`, `<=` and `&&`.

TASK

Write the arrow function `isValidRating(rating)`. It returns `true` when the rating is from 0 to 10 (both included), and `false` otherwise.

COMMON MISTAKES

- `0 <= rating <= 10`: JavaScript reads it differently and says `true` for 15.
- `>` and `<`, so 0 and 10 are rejected.

A CORRECT ANSWER

```
// Practice 24: is the rating from 0 to 10?
// isValidRating(8.8) should give true, isValidRating(11) false.
const isValidRating = (rating) => rating >= 0 && rating <= 10;

tryIt(isValidRating, [8.8], true);
tryIt(isValidRating, [11], false);
tryIt(isValidRating, [-1], false);
tryIt(isValidRating, [10], true);
```

Practice 25. Do the years make sense? ~8 min

Teaches: Practice: validation that names the problem.

TASK

Write `checkYears(premiered, ended)`. It returns:

1. **"Not out yet"** when `premiered` is after 2026
2. **"Ended before it started"** when `ended` is before `premiered`
3. **"OK"** otherwise

Chernobyl premiered and ended in 2019: that's **"OK"**.

COMMON MISTAKES

- `ended <= premiered`, so a one-year show like Chernobyl is rejected.
- `>= 2026`, so a show from this year is "Not out yet".

A CORRECT ANSWER

```
// Practice 25: do the two years make sense?
// checkYears(2013, 2008) should give "Ended before it started".
function checkYears(premiered, ended) {
  if (premiered > 2026) {
    return "Not out yet";
  } else if (ended < premiered) {
    return "Ended before it started";
  } else {
    return "OK";
  }
}

tryIt(checkYears, [1994, 2004], "OK");
tryIt(checkYears, [2013, 2008], "Ended before it started");
tryIt(checkYears, [2030, 2035], "Not out yet");
tryIt(checkYears, [2019, 2019], "OK");
```

Practice 26. Price tiers for screens ~8 min

Teaches: Practice: a price table as a chain of cases.

TASK

Imagine a streaming plan whose price depends on how many screens can watch at once. Write `planPrice(screens)`:

- 0 screens: **0**
- 1 screen: **8**
- 2 screens: **12**
- 3 or more screens: **16**

COMMON MISTAKES

- `else if (screens === 3)` for the last tier, so 4 screens gives `undefined`.
- Prices in quotes (`"8"`), which are text, not numbers.

A CORRECT ANSWER

```
// Practice 26: the price of a plan, by number of screens.
// planPrice(2) should give 12.
function planPrice(screens) {
  if (screens === 0) {
    return 0;
  } else if (screens === 1) {
    return 8;
  } else if (screens === 2) {
    return 12;
  } else {
    return 16;
  }
}

tryIt(planPrice, [0], 0);
tryIt(planPrice, [1], 8);
tryIt(planPrice, [2], 12);
tryIt(planPrice, [5], 16);
```

Practice 27. Age groups ~6 min

Teaches: Practice: sort a number into groups, smallest first.

TASK

ShowShelf sorts viewers into groups. Write `ageGroup(age)`:

- under 7: **"Little ones"**
- 7 to 12: **"Kids"**
- 13 to 17: **"Teens"**
- 18 and older: **"Adults"**

COMMON MISTAKES

- `<= 13` for Kids, so a 13-year-old is a Kid.
- Testing `age < 18` first, so everyone under 18 is a Teen.

A CORRECT ANSWER

```
// Practice 27: sort a viewer's age into a group.
// ageGroup(13) should give "Teens".
function ageGroup(age) {
  if (age < 7) {
    return "Little ones";
  } else if (age < 13) {
    return "Kids";
  } else if (age < 18) {
    return "Teens";
  } else {
    return "Adults";
  }
}

tryIt(ageGroup, [4], "Little ones");
tryIt(ageGroup, [7], "Kids");
tryIt(ageGroup, [13], "Teens");
tryIt(ageGroup, [18], "Adults");
```

Practice 28. A show for your mood ~8 min

Teaches: Practice: compare text with `===` after `.toLowerCase()`.

TASK

Write `recommend(mood)`, which picks a show from our table by genre:

- **"happy"**: a comedy, **"Friends"**
- **"curious"**: a mystery, **"Dark"**
- **"brave"**: an adventure, **"Avatar: The Last Airbender"**
- anything else: **"The Crown"**

It must work whatever capitals are used: `recommend("Happy")` and `recommend("HAPPY")` both give "Friends".

COMMON MISTAKES

- Comparing `mood === "happy"` without `.toLowerCase()`, so "Happy" gives The Crown.
- Comparing with `"Happy"` (capital H) after making the text small.
- `mood.toLowerCase` without brackets.

A CORRECT ANSWER

```
// Practice 28: a show for each mood, whatever capitals are typed.
// recommend("Happy") should give "Friends".
function recommend(mood) {
  const m = mood.toLowerCase();
  if (m === "happy") {
    return "Friends";
  } else if (m === "curious") {
    return "Dark";
  } else if (m === "brave") {
    return "Avatar: The Last Airbender";
  } else {
    return "The Crown";
  }
}

tryIt(recommend, ["happy"], "Friends");
tryIt(recommend, ["Curious"], "Dark");
tryIt(recommend, ["BRAVE"], "Avatar: The Last Airbender");
tryIt(recommend, ["sleepy"], "The Crown");
```

Practice 29. Which season is episode 7 in? ~10 min

Teaches: Practice: put numbered items into groups with `Math.floor`.

TASK

Write `seasonOf(episode, perSeason)`. Episodes are numbered from 1 across the whole show, and every season has `perSeason` episodes. Return the season the episode is in.

Fleabag has 6 episodes per season: episode 6 is in season **1**, episode 7 in season **2**.

COMMON MISTAKES

- `Math.floor(episode / perSeason) + 1` without `- 1`: episode 6 lands in season 2.
- Brackets missing around `episode - 1`, so only the 1 is divided.

A CORRECT ANSWER

```
// Practice 29: which season is an episode in?
// Fleabag, 6 per season: seasonOf(7, 6) should give 2.
function seasonOf(episode, perSeason) {
  return Math.floor((episode - 1) / perSeason) + 1;
}

tryIt(seasonOf, [1, 6], 1);
tryIt(seasonOf, [6, 6], 1);
tryIt(seasonOf, [7, 6], 2);
tryIt(seasonOf, [12, 6], 2);
```

Practice 30. Episode number within its season ~10 min

Teaches: Practice: the position inside a group with `%`.

TASK

Write `episodeInSeason(episode, perSeason)`. It returns the episode's number **within its season**.

Fleabag has 6 episodes per season. Episode 8 of the whole show is episode **2** of season 2, and episode 6 is episode **6** of season 1.

COMMON MISTAKES

- `episode % perSeason`: the last episode of a season gives 0.
- `episode - 1 % perSeason + 1` without brackets: `%` happens first and the answer is the episode itself.

A CORRECT ANSWER

```
// Practice 30: the episode's number within its own season.
// Fleabag, 6 per season: episodeInSeason(8, 6) should give 2.
function episodeInSeason(episode, perSeason) {
  return (episode - 1) % perSeason + 1;
}

tryIt(episodeInSeason, [8, 6], 2);
tryIt(episodeInSeason, [6, 6], 6);
tryIt(episodeInSeason, [7, 6], 1);
tryIt(episodeInSeason, [1, 6], 1);
```

Practice 31. Episode 8 as a full label (harder) ~10 min

Teaches: Practice: guard against bad input, then build a label from two helpers.

TASK

Write `fullLabel(episode, perSeason, total)`. It returns:

1. **"No such episode"** when `episode` is less than 1, or more than `total`
2. otherwise a label like **"Season 2, episode 2"**, made with `seasonOf` and `episodeInSeason`

Fleabag has 12 episodes, 6 per season: `fullLabel(8, 6, 12)` gives **"Season 2, episode 2"**.

COMMON MISTAKES

- Only one guard, so episode 0 becomes "Season 0, episode 0" or episode 13 becomes "Season 3, episode 1".
- `episode >= total`, so the last episode is rejected.
- Writing the helper names without the arguments: `${seasonOf}`.

A CORRECT ANSWER

```
// Practice 31 (harder): "Season 2, episode 2" from an episode number.

// Helpers, already written (don't change them):
function seasonOf(episode, perSeason) {
  return Math.floor((episode - 1) / perSeason) + 1;
}

function episodeInSeason(episode, perSeason) {
  return (episode - 1) % perSeason + 1;
}

// Your function:
function fullLabel(episode, perSeason, total) {
  if (episode < 1) {
    return "No such episode";
  } else if (episode > total) {
    return "No such episode";
  } else {
    return `Season ${seasonOf(episode, perSeason)}, episode ${episodeInSeason(episode, perSeason)}`;
  }
}

tryIt(fullLabel, [8, 6, 12], "Season 2, episode 2");
tryIt(fullLabel, [6, 6, 12], "Season 1, episode 6");
tryIt(fullLabel, [13, 6, 12], "No such episode");
tryIt(fullLabel, [0, 6, 12], "No such episode");
```

Practice 32. Which of two shows? (harder) ~10 min

Teaches: Practice: a main comparison with a tie-breaker.

TASK

Write `betterPick(ratingA, runtimeA, ratingB, runtimeB)`. It returns **"First"** or **"Second"**:

1. The show with the **higher rating** wins.
2. With the same rating, the **shorter runtime** wins.
3. When both are the same, return **"Either one"**.

The Wire (9.0, 60 min) against Avatar: The Last Airbender (9.0, 30 min): same rating, so the shorter Avatar wins: **"Second"**.

COMMON MISTAKES

- Comparing runtimes before ratings, so a short, weaker show wins.
- `runtimeA > runtimeB` for "First": the longer show wins the tie.
- No final `else`, so a perfect tie gives `undefined`.

A CORRECT ANSWER

```
// Practice 32 (harder): higher rating wins; same rating, shorter wins.
// betterPick(9.0, 60, 9.0, 30) should give "Second".
function betterPick(ratingA, runtimeA, ratingB, runtimeB) {
  if (ratingA > ratingB) {
    return "First";
  } else if (ratingB > ratingA) {
    return "Second";
  } else if (runtimeA < runtimeB) {
    return "First";
  } else if (runtimeB < runtimeA) {
    return "Second";
  } else {
    return "Either one";
  }
}

tryIt(betterPick, [9.2, 60, 8.8, 30], "First");
tryIt(betterPick, [9.0, 60, 9.0, 30], "Second");
tryIt(betterPick, [8.0, 30, 8.0, 60], "First");
tryIt(betterPick, [9.1, 60, 9.1, 60], "Either one");
```

Practice 33. A summary from three helpers (harder) ~12 min

Teaches: Practice: combine three helpers, one call inside another.

TASK

Write `showSummary(name, year, episodes, runtime)`, using all three helpers. It returns a line like this:

Chernobyl (2019): 5 episodes, 5 hours

and for a one-episode, one-hour special: **Pilot (2020): 1 episode, 1 hour**.

COMMON MISTAKES

- `plural(episodes * runtime / 60, "hour")`: the maths is repeated and not rounded.
- Forgetting the second argument: `plural(hours)` says "118 episodes".
- `${plural(hoursOf, "hour")}`: the helper's name without calling it.

A CORRECT ANSWER

```
// Practice 33 (harder): one line about a show, from three helpers.
// showSummary("Chernobyl", 2019, 5, 60) should give
// "Chernobyl (2019): 5 episodes, 5 hours"

// Helpers, already written (don't change them):
function titleWithYear(name, year) {
  return `${name} (${year})`;
}

function plural(count, word = "episode") {
  if (count === 1) {
    return `${count} ${word}`;
  } else {
    return `${count} ${word}s`;
  }
}

const hoursOf = (episodes, runtime) => Math.round(episodes * runtime / 60);

// Your function:
function showSummary(name, year, episodes, runtime) {
  return `${titleWithYear(name, year)}: ${plural(episodes, "episode")}, ${plural(hoursOf(episodes,
runtime), "hour")}`;
}

tryIt(showSummary, ["Chernobyl", 2019, 5, 60], "Chernobyl (2019): 5 episodes, 5 hours");
tryIt(showSummary, ["Friends", 1994, 236, 30], "Friends (1994): 236 episodes, 118 hours");
tryIt(showSummary, ["Pilot", 2020, 1, 60], "Pilot (2020): 1 episode, 1 hour");
```

Practice 34. When does it end? (harder) ~12 min

Teaches: Practice: clock times with `Math.floor`, `%` and a leading zero.

TASK

Write `endTime(hour, minute, runtime)`. It gets the start time and the runtime in minutes, and returns the end time as text, like **"22:15"**. The hours have no extra 0 (**"0:30"** after midnight), but the minutes always have two digits.

Start Friends at 19:00 and watch all 7080 minutes without a break: `endTime(19, 0, 7080)` gives **"17:00"**, some days later.

COMMON MISTAKES

- No `% 24`, so a late show ends at "24:30".
- Forgetting the leading zero: "23:5".
- `hour + runtime / 60` instead of counting everything in minutes.

A CORRECT ANSWER

```
// Practice 34 (harder): start at hour:minute, watch for runtime minutes.
// endTime(21, 30, 45) should give "22:15".
function endTime(hour, minute, runtime) {
  const total = hour * 60 + minute + runtime;
  const hours = Math.floor(total / 60) % 24;
  const minutes = total % 60;
  if (minutes < 10) {
    return `${hours}:0${minutes}`;
  } else {
    return `${hours}:${minutes}`;
  }
}

tryIt(endTime, [21, 30, 45], "22:15");
tryIt(endTime, [20, 0, 60], "21:00");
tryIt(endTime, [22, 55, 10], "23:05");
tryIt(endTime, [23, 30, 60], "0:30");
```

Practice 35. Premiere, finale or regular? (harder) ~12 min

Teaches: Practice: five overlapping cases with `%`, strictest first.

TASK

Write `episodeStatus(episode, perSeason, seasons)`. The show has `perSeason * seasons` episodes in total. Return:

1. **"Not a real episode"** when `episode` is more than the total
2. **"Series finale"** for the very last episode
3. **"Season finale"** for the last episode of a season
4. **"Season premiere"** for the first episode of a season
5. **"Regular episode"** otherwise

COMMON MISTAKES

- "Season finale" tested before "Series finale", so the last episode is only a season finale.
- `episode / perSeason === 0` instead of `%`.
- Forgetting the total check, so episode 13 is called a premiere.

A CORRECT ANSWER

```
// Practice 35 (harder): what kind of episode is it?
// Fleabag, 6 per season, 2 seasons: episodeStatus(6, 6, 2) should give "Season finale".
function episodeStatus(episode, perSeason, seasons) {
  const total = perSeason * seasons;
  if (episode > total) {
    return "Not a real episode";
  } else if (episode === total) {
    return "Series finale";
  } else if (episode % perSeason === 0) {
    return "Season finale";
  } else if (episode % perSeason === 1) {
    return "Season premiere";
  } else {
    return "Regular episode";
  }
}

tryIt(episodeStatus, [1, 6, 2], "Season premiere");
tryIt(episodeStatus, [3, 6, 2], "Regular episode");
tryIt(episodeStatus, [6, 6, 2], "Season finale");
tryIt(episodeStatus, [12, 6, 2], "Series finale");
tryIt(episodeStatus, [13, 6, 2], "Not a real episode");
```

Practice 36. How much longer? (harder) ~12 min

Teaches: Practice: a sentence that changes with a comparison, always with positive numbers.

TASK

Write `compareTime(nameA, hoursA, nameB, hoursB)`. It returns one of three sentences:

- **"Friends takes 56 hours more than Breaking Bad"** (118 against 62)
- **"Chernobyl takes 5 hours less than Band of Brothers"** (5 against 10)
- **"The Wire and The Crown take the same time"** (60 and 60)

The numbers in the sentences are always positive.

COMMON MISTAKES

- `${hoursA - hoursB}` in every case, so the sentence says "-5 hours less".
- The words "more" and "less" in the wrong blocks.
- No case for equal times, so it says "0 hours less".

A CORRECT ANSWER

```
// Practice 36 (harder): compare how long two shows take, in a sentence.
// compareTime("Friends", 118, "Breaking Bad", 62) should give
// "Friends takes 56 hours more than Breaking Bad"
function compareTime(nameA, hoursA, nameB, hoursB) {
  if (hoursA > hoursB) {
    return `${nameA} takes ${hoursA - hoursB} hours more than ${nameB}`;
  } else if (hoursA < hoursB) {
    return `${nameA} takes ${hoursB - hoursA} hours less than ${nameB}`;
  } else {
    return `${nameA} and ${nameB} take the same time`;
  }
}

tryIt(compareTime, ["Friends", 118, "Breaking Bad", 62], "Friends takes 56 hours more than Breaking
Bad");
tryIt(compareTime, ["Chernobyl", 5, "Band of Brothers", 10], "Chernobyl takes 5 hours less than Band
of Brothers");
tryIt(compareTime, ["The Wire", 60, "The Crown", 60], "The Wire and The Crown take the same time");
```

Extra 1. Episode codes like S01E05 ~15 min

Teaches: `String(n).padStart(2, "0")` gives two-digit numbers like 05.

TASK

Write `episodeCode(season, episode)` that returns a code like **S01E05**. Each card gets a new line: *First episode: S01E01*.

Test it: `console.log(episodeCode(10, 18));` should print **S10E18**.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(plural(3, "season"));

function episodeCode(season, episode) {
  const s = String(season).padStart(2, "0");
  const e = String(episode).padStart(2, "0");
  return `S${s}E${e}`;
}

console.log(episodeCode(10, 18));
```

Extra 2. How many days to binge it? ~15 min

Teaches: `Math.ceil()` rounds up to the next whole number.

TASK

Write `binge(hours, hoursPerDay)` that returns how many days it takes to watch `hours` of TV at `hoursPerDay` a day. A started day counts as a whole day, so round up.

Every card gets a new line: *At 2 hours a day: 59 days*.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(plural(3, "season"));

function binge(hours, hoursPerDay) {
  return Math.ceil(hours / hoursPerDay);
}
```

Day 6: Lists of shows

Arrays, objects, loops

A real app doesn't show one show: it shows lists. Today you learn how JavaScript keeps many values together (**arrays**), how it describes one thing with named values (**objects**), and how to go through a whole list with a **loop**.

The file **data.js** (a read-only tab above the code box) holds twelve shows, each one an object with a name, genres, a rating and more. You start in the Console, exploring the data. Then you write small functions that pick shows from the list: the best ones, one by its id, the ones that match a search, the ones in a genre.

As before, a helper file, **page.js**, uses your functions to fill in the page. It also scrolls the preview to the newest part you've made.

After the ten steps come thirty-two short **practice exercises** (P1 to P32) in a practice gym: small functions that use the same ideas again, from very easy to harder.

Step 1. A list: arrays

~15 min

Teaches: Arrays: `[a, b, c]`, `list[0]` is the first item, `.length`, `.push()`.

TASK

1. Create `favourites`: an array with the names of three shows you like.
2. Log the first one, and log how many there are.
3. Add a fourth show with `push`. Then log the whole array.

COMMON MISTAKES

- `favourites[1]` for the first item (counting starts at 0).
- Forgetting the commas between items.
- Round brackets instead of square ones: `("Dark", "Fleabag")`.

A CORRECT ANSWER

```
// app.js: lists of shows.
// data.js (a read-only tab) gives you `shows`: an array of 12 show objects.

const favourites = ["Dark", "Fleabag", "The Wire"];
console.log(favourites[0]);
console.log(favourites.length);

favourites.push("Chernobyl");
console.log(favourites);
```

Step 2. One show: objects ~15 min

Teaches: Objects: `{ key: value, ... }`. Read with `obj.key`, change with `obj.key = value`.

TASK

1. Create an object `myShow` for a show you like, with three properties: `name` (text), `year` (the year it started, a number) and `rating` (a number).
2. Log its name.
3. Change the rating to your own score, with a dot and `=`.
4. Log the whole object.

COMMON MISTAKES

- `=` instead of `:` inside the object: `name = "Fleabag"`.
- A missing comma between properties.
- Quotes around numbers: `year: "2016"`.

A CORRECT ANSWER

```
// ... unchanged ...
favourites.push("Chernobyl");
console.log(favourites);

const myShow = {
  name: "Fleabag",
  year: 2016,
  rating: 8.4,
};
console.log(myShow.name);

myShow.rating = 10;
console.log(myShow);
```

Step 3. The twelve shows ~15 min

Teaches: Arrays of objects: `shows[0].name`. The last item is `list[list.length - 1]`.

TASK

Log four things about the data:

1. how many shows there are
2. the name of the first show
3. the rating of the second show
4. the name of the last show, using `shows.length - 1`

COMMON MISTAKES

- `shows[12]` for the last show (it's `shows[11]`).
- `shows.name` (the array has no name; each show has).

NOTE

Encourage students to open the data.js tab and read one show object in full before they start.

A CORRECT ANSWER

```
// ... unchanged ...
myShow.rating = 10;
console.log(myShow);

console.log(shows.length);
console.log(shows[0].name);
console.log(shows[1].rating);
console.log(shows[shows.length - 1].name);
```

Step 4. Go through every show ~15 min

Teaches: `for (const item of list) { ... }` runs the block once per item.

TASK

Write a `for...of` loop through `shows` that logs the name of every show. You'll see twelve new lines in the Console.

If you like, log the rating too: `console.log(show.name, show.rating);`.

COMMON MISTAKES

- `shows.name` inside the loop instead of `show.name`.
- `for (const show in shows)`: `in` gives the index numbers, not the shows.

A CORRECT ANSWER

```
// ... unchanged ...
console.log(shows[1].rating);
console.log(shows[shows.length - 1].name);

for (const show of shows) {
  console.log(show.name);
}
```

Step 5. Keep only the best ~20 min

Teaches: Pick items: start with `[]`, loop, and `push` the ones that pass an `if`.

TASK

Replace the `TODO` comment: if a show's rating is **9 or more**, push the show into `best`.

The Console shows `best.length`. How many shows are rated 9 or more?

Then try it with `8.8` instead of `9`, and change it back.

COMMON MISTAKES

- The `push` outside the `if` (every show is added).
- `const best = [];` inside the loop (it starts empty every round).
- `>` instead of `>=` (the 9.0 shows are left out).

A CORRECT ANSWER

```
// ... unchanged ...
  console.log(show.name);
}

const best = [];
for (const show of shows) {
  if (show.rating >= 9) {
    best.push(show);
  }
}
console.log(best.length);
```

Step 6. filter: the short way ~20 min

Teaches: `list.filter((item) => test)` gives a new array with the items that pass the test.

TASK

Under your code, write `topRated(list)`. It returns the shows in `list` with a rating of **9 or more**, using `filter`.

The Top rated section on the page fills with five cards.

COMMON MISTAKES

- Forgetting `return` before `list.filter`.
- Using `shows` inside the function instead of the parameter `list`.
- Curly brackets after the arrow without a `return`: `(show) => { show.rating >= 9 }` keeps nothing.

A CORRECT ANSWER

```
// ... unchanged ...
}
console.log(best.length);

function topRated(list) {
  return list.filter((show) => show.rating >= 9);
}
```

Step 7. map: one thing from each ~15 min

Teaches: `list.map((item) => newValue)` gives a new array with one result per item.

TASK

1. Write `names(list)` that returns an array with the name of every show in `list`, using `map`.
2. Log `names(shows)` to see the result in the Console.

COMMON MISTAKES

- Using `filter` instead of `map` (you get whole shows back).
- `shows.name` instead of `show.name` inside the arrow function.

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.rating >= 9);
}

function names(list) {
  return list.map((show) => show.name);
}

console.log(names(shows));
```

Step 8. find: one show by its id ~15 min

Teaches: `list.find((item) => test)` gives back the first matching item, or `undefined`.

TASK

1. Write `getById(list, id)` that returns the show in `list` with that `id`, using `find`.
2. Test it: `console.log(getById(shows, 2).name);` should print Breaking Bad. (Without `.name`, you'd see the whole show object.)

page.js uses it for **Pick of the day**: `getById(shows, 7)`.

COMMON MISTAKES

- `show.id = id` (one equals sign) always gives the first show.
- Using `filter` and getting a list with one show in it.
- `getById(shows, "2")`: the id in data.js is a number, and `===` checks the type.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(names(shows));

function getById(list, id) {
  return list.find((show) => show.id === id);
}

console.log(getById(shows, 2).name);
```

Step 9. Search by name ~20 min

Teaches: Search: `trim()`, `toLowerCase()` on both sides, then `includes()`.

TASK

We added the start of `searchByName(list, term)`. Replace the two `TODO` comments:

1. Make `needle`: the `term`, trimmed and in lower case.
2. Return the shows in `list` whose name, in lower case, includes `needle`.

Then type in the search box on the page. Try `office`, `bad` and `THE`. Why does How I Met Your Mother appear for `THE`?

COMMON MISTAKES

- Making only the search word lower case, not the show's name.
- `show.name === needle` instead of `includes` (only exact names match).
- `toLowerCase` without brackets.

NOTE

On Day 7 students write the event code that page.js does here (listening to the search box). Today, the point is the function.

A CORRECT ANSWER

```
// ... unchanged ...

console.log(getById(shows, 2).name);

function searchByName(list, term) {
  const needle = term.trim().toLowerCase();
  return list.filter((show) => show.name.toLowerCase().includes(needle));
}
```

Step 10. Browse by genre ~15 min

Teaches: `array.includes(value)` checks whether a list contains a value.

TASK

Write `getByGenre(list, genre)`. It returns the shows in `list` whose `genres` include `genre`.

Then click the genre buttons on the page. What happens with **Western**?

COMMON MISTAKES

- `show.genre` instead of `show.genres` (the key has an s).
- `show.genres === genre` (an array is never equal to one string).

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.name.toLowerCase().includes(needle));
}

function getByGenre(list, genre) {
  return list.filter((show) => show.genres.includes(genre));
}
```

Practice 1. How many shows? ~5 min

Teaches: Practice: `.length` counts the items in any list.

TASK

Make `countShows(list)` return the number of shows in `list`.

When every row in the preview is green, press **Check**.

COMMON MISTAKES

- `return shows.length` instead of `list.length` (right for `shows`, wrong for the other lists).
- `list.length - 1` (that's the index of the last item, not the count).
- Forgetting `return`: the rows say "Did you forget return?".

NOTE

Practice exercises are independent: a student can skip one and come back to it later. Each one has its own Hint and Answer buttons, like the steps.

A CORRECT ANSWER

```
// P1. How many shows are in a list?
// countShows(list) gives back how many shows the list has.

function countShows(list) {
  return list.length;
}

// Each tryIt line calls your function and shows the result in the preview.
const two = [{ name: "Dark" }, { name: "Fleabag" }];
const none = [];

tryIt(countShows, [shows], 12);
tryIt(countShows, [two], 2);
tryIt(countShows, [none], 0);
```

Practice 2. The first name ~5 min

Teaches: Practice: `list[0].name` is the first item's name.

TASK

Make `firstName(list)` return the **name** of the first show in `list`.

COMMON MISTAKES

- `list[1]` for the first item (counting starts at 0).
- `list.name` (the list has no name; each show has).
- Returning `list[0]`, the whole show, without `.name`.

A CORRECT ANSWER

```
// P2. The name of the first show
// firstName(list) gives back the name of the first show in the list.

function firstName(list) {
  return list[0].name;
}

const trio = [{ name: "Chernobyl" }, { name: "Dark" }, { name: "The Crown" }];

tryIt(firstName, [shows], "Friends");
tryIt(firstName, [trio], "Chernobyl");
```

Practice 3. The last name ~8 min

Teaches: Practice: the last item is `list[list.length - 1]`.

TASK

Make `lastName(list)` return the **name** of the last show in `list`. It must work for lists of any length, so don't write a fixed number like 11.

COMMON MISTAKES

- `list[list.length]`: one past the end, so the show is `undefined` and reading `.name` crashes.
- `list[11]`: right for `shows`, but crashes for the shorter lists.
- `list.length - 1` without the square brackets around it.

A CORRECT ANSWER

```
// P3. The name of the last show
// lastName(list) gives back the name of the last show, for a list of any length.

function lastName(list) {
  return list[list.length - 1].name;
}

const trio = [{ name: "Chernobyl" }, { name: "Dark" }, { name: "The Crown" }];
const one = [{ name: "Fleabag" }];

tryIt(lastName, [shows], "Avatar: The Last Airbender");
tryIt(lastName, [trio], "The Crown");
tryIt(lastName, [one], "Fleabag");
```

Practice 4. A sentence about one show ~8 min

Teaches: Practice: `${show.name}` puts a property into a template literal.

TASK

Make `aboutShow(show)` return a sentence built from the show's `name`, `premiered` and `network`, like this: **Friends started in 1994 on NBC.**

COMMON MISTAKES

- `shows.name` (the list) instead of `show.name` (the parameter): the sentence says *undefined*.
- Quotes instead of backticks, so `${...}` appears as it is.
- A wrong key such as `show.year` (the key is `premiered`).

A CORRECT ANSWER

```
// P4. A sentence about one show
// aboutShow(show) gets ONE show object and gives back a sentence like:
// "Friends started in 1994 on NBC."

function aboutShow(show) {
  return `${show.name} started in ${show.premiered} on ${show.network}.`;
}

tryIt(aboutShow, [shows[0]], "Friends started in 1994 on NBC.");
tryIt(aboutShow, [shows[6]], "Chernobyl started in 2019 on HBO.");
tryIt(aboutShow, [{ name: "Dark", premiered: 2017, network: "Netflix" }], "Dark started in 2017 on
Netflix.");
```

Practice 5. Shows from one network ~8 min

Teaches: Practice: `filter` with a parameter in the test, `show.network === network`.

TASK

Make `fromNetwork(list, network)` return the shows in `list` whose `network` is the same as the parameter `network`. Give back whole shows, not names.

COMMON MISTAKES

- One equals sign, `show.network = network`: every show is kept.
- Curly brackets after the arrow without `return`: nothing is kept.
- `"network"` in quotes, which compares with the word network instead of the parameter.

A CORRECT ANSWER

```
// P5. Shows from one network
// fromNetwork(list, network) gives back the shows (whole objects) from that network.

function fromNetwork(list, network) {
  return list.filter((show) => show.network === network);
}

const mini = [
  { name: "Chernobyl", network: "HBO" },
  { name: "Friends", network: "NBC" },
  { name: "The Wire", network: "HBO" },
];
const none = [];

tryIt(fromNetwork, [mini, "HBO"], [{ name: "Chernobyl", network: "HBO" }, { name: "The Wire",
network: "HBO" }]);
tryIt(fromNetwork, [mini, "NBC"], [{ name: "Friends", network: "NBC" }]);
tryIt(fromNetwork, [mini, "AMC"], []);
tryIt(fromNetwork, [none, "HBO"], []);
```

Practice 6. Names above a rating ~8 min

Teaches: Practice: `filter` to pick, then `map` to take the names.

TASK

Make `topNames(list, minRating)` return the **names** of the shows in `list` with a rating of `minRating` **or more**.

COMMON MISTAKES

- `map` before `filter`: the names have no rating, so nothing is kept.
- `>` instead of `>=`: a show rated exactly `minRating` is left out.
- Returning the filtered shows without the `map` step.

A CORRECT ANSWER

```
// P6. Names of the shows with at least a given rating
// topNames(list, minRating) gives back the NAMES of the shows rated minRating or more.

function topNames(list, minRating) {
  const good = list.filter((show) => show.rating >= minRating);
  return good.map((show) => show.name);
}

const mini = [
  { name: "Fleabag", rating: 8.4 },
  { name: "Chernobyl", rating: 9.1 },
  { name: "Dark", rating: 8.7 },
];
const none = [];

tryIt(topNames, [mini, 8.5], ["Chernobyl", "Dark"]);
tryIt(topNames, [mini, 9.1], ["Chernobyl"]);
tryIt(topNames, [mini, 9.5], []);
tryIt(topNames, [none, 8], []);
tryIt(topNames, [shows, 9], ["Breaking Bad", "Chernobyl", "The Wire", "Band of Brothers", "Avatar:
The Last Airbender"]);
```

Practice 7. The comedies, by name ~8 min

Teaches: Practice: `show.genres.includes("Comedy")` tests a value inside an array.

TASK

Make `comedyNames(list)` return the **names** of the shows in `list` whose genres include `"Comedy"`.

COMMON MISTAKES

- `show.genres === "Comedy"` (an array is never equal to one string).
- `show.genre` without the s: "Cannot read properties of undefined".
- `"comedy"` in small letters: array `includes` checks capitals too.

A CORRECT ANSWER

```
// P7. The names of the comedies
// comedyNames(list) gives back the NAMES of the shows that have "Comedy" in their genres.

function comedyNames(list) {
  const comedies = list.filter((show) => show.genres.includes("Comedy"));
  return comedies.map((show) => show.name);
}

const mini = [
  { name: "Fleabag", genres: ["Comedy", "Drama"] },
  { name: "Dark", genres: ["Drama", "Mystery", "Science-Fiction"] },
];
const none = [];

tryIt(comedyNames, [mini], ["Fleabag"]);
tryIt(comedyNames, [none], []);
tryIt(comedyNames, [shows], ["Friends", "The Office", "How I Met Your Mother", "Fleabag"]);
```

Practice 8. Started before a year ~10 min

Teaches: Practice: pick names with a loop: empty array, `if`, `push(show.name)`, return after the loop.

TASK

Make `startedBefore(list, year)` return the **names** of the shows that premiered **before** `year` (a show that started in that year itself doesn't count). Try the loop way this time.

COMMON MISTAKES

- `<=` instead of `<`: a show from that very year is included.
- `const found = [];` inside the loop: "found is not defined" after the loop.
- `return found;` inside the loop: only the first show is checked.

A CORRECT ANSWER

```
// P8. Shows that started before a year
// startedBefore(list, year) gives back the NAMES of the shows that premiered before that year.
// Try it with a loop, an if and push.

function startedBefore(list, year) {
  const found = [];
  for (const show of list) {
    if (show.premiered < year) {
      found.push(show.name);
    }
  }
  return found;
}

const mini = [
  { name: "Band of Brothers", premiered: 2001 },
  { name: "Dark", premiered: 2017 },
  { name: "Friends", premiered: 1994 },
];
const none = [];

tryIt(startedBefore, [mini, 2010], ["Band of Brothers", "Friends"]);
tryIt(startedBefore, [mini, 2001], ["Friends"]);
tryIt(startedBefore, [mini, 1990], []);
tryIt(startedBefore, [none, 2000], []);
```

Practice 9. Is it on the list? ~10 min

Teaches: Practice: `list.find(...) !== undefined` is true when something matches.

TASK

Make `hasShow(list, title)` return `true` if `list` has a show whose name is `title`, ignoring capital letters, and `false` otherwise.

`"the office"` should find The Office, but `"Office"` should not: no show is called only Office.

COMMON MISTAKES

- Returning the show itself instead of `true` / `false`.
- Making only one side lower case.
- Using `includes` on the name, so "Office" finds The Office.

A CORRECT ANSWER

```
// P9. Is a show on the list?
// hasShow(list, title) gives back true if a show in the list has exactly that name
// (capital letters don't matter), and false if not.

function hasShow(list, title) {
  const wanted = title.toLowerCase();
  const found = list.find((show) => show.name.toLowerCase() === wanted);
  return found !== undefined;
}

const none = [];

tryIt(hasShow, [shows, "Dark"], true);
tryIt(hasShow, [shows, "the office"], true);
tryIt(hasShow, [shows, "BREAKING BAD"], true);
tryIt(hasShow, [shows, "Office"], false);
tryIt(hasShow, [shows, "Lost"], false);
tryIt(hasShow, [none, "Dark"], false);
```

Practice 10. Count the short shows ~10 min

Teaches: Practice: count with `let count = 0;` before the loop and `count = count + 1;` inside the `if`.

TASK

Make `countShort(list)` return how many shows in `list` have episodes of **30 minutes or less** (the key is `runtime`). Use a counter.

COMMON MISTAKES

- `const count = 0;`: "Assignment to constant variable" on the first `+ 1`.
- `let count = 0;` inside the loop, or `return count;` inside the loop.
- `=== 30` instead of `<= 30`: a 25-minute show isn't counted.

NOTE

This is the first exercise with a counter. If a student is lost, act it out: "Start at zero. For each show, if it's short, add one. At the end, say the number."

A CORRECT ANSWER

```
// P10. Count the short shows
// countShort(list) gives back HOW MANY shows have episodes of 30 minutes or less.
// Use a counter: let count = 0; before the loop, add 1 inside the if.

function countShort(list) {
  let count = 0;
  for (const show of list) {
    if (show.runtime <= 30) {
      count = count + 1;
    }
  }
  return count;
}

const mini = [
  { name: "Fleabag", runtime: 30 },
  { name: "Dark", runtime: 60 },
  { name: "Quick Show", runtime: 25 },
];
const none = [];

tryIt(countShort, [mini], 2);
tryIt(countShort, [none], 0);
tryIt(countShort, [shows], 5);
```

Practice 11. Total episodes ~8 min

Teaches: Practice: add up with `let total = 0;` before the loop and `total = total + ...` inside.

TASK

Make `totalEpisodes(list)` return the total number of episodes of all the shows in `list`.

COMMON MISTAKES

- `total = show.episodes` without `total +`: only the last show is left.
- `return total;` inside the loop: only the first show is added.
- `const total`: "Assignment to constant variable".

A CORRECT ANSWER

```
// P11. Total episodes
// totalEpisodes(list) gives back the number of episodes of all the shows together.

function totalEpisodes(list) {
  let total = 0;
  for (const show of list) {
    total = total + show.episodes;
  }
  return total;
}

const mini = [
  { name: "Chernobyl", episodes: 5 },
  { name: "Band of Brothers", episodes: 10 },
  { name: "Fleabag", episodes: 12 },
];
const none = [];

tryIt(totalEpisodes, [mini], 27);
tryIt(totalEpisodes, [none], 0);
tryIt(totalEpisodes, [shows], 1014);
```

Practice 12. The average rating ~12 min

Teaches: Practice: average = total / `list.length`; `Math.round(x * 10) / 10` keeps one decimal.

TASK

Make `averageRating(list)`:

1. return `0` if `list` is empty
2. otherwise, return the average `rating` of the shows, rounded to one decimal

COMMON MISTAKES

- Dividing by 12 instead of `list.length`.
- `Math.round(average)` without the times-10 trick: 9 instead of 8.7.
- No check for an empty list: the function gives `NaN`.

A CORRECT ANSWER

```
// P12. The average rating
// averageRating(list) gives back the average rating, rounded to one decimal.
// An empty list gives 0.

function averageRating(list) {
  if (list.length === 0) {
    return 0;
  }
  let total = 0;
  for (const show of list) {
    total = total + show.rating;
  }
  return Math.round(total / list.length * 10) / 10;
}

const mini = [
  { name: "Chernobyl", rating: 9.1 },
  { name: "Friends", rating: 8.8 },
  { name: "The Crown", rating: 8 },
];
const two = [{ name: "Dark", rating: 8.7 }, { name: "Game of Thrones", rating: 8.9 }];
const none = [];

tryIt(averageRating, [mini], 8.6);
tryIt(averageRating, [two], 8.8);
tryIt(averageRating, [none], 0);
tryIt(averageRating, [shows], 8.7);
```

Practice 13. Every network once ~12 min

Teaches: Practice: no duplicates: push only when `result.includes(value) === false`.

TASK

Make `uniqueNetworks(list)` return a list of the networks of the shows in `list`, each network only once, in the order they first appear.

COMMON MISTAKES

- Pushing the whole show instead of `show.network`.
- Asking `list.includes(...)` (the list you loop through) instead of the new list.
- `=== true` instead of `=== false`: nothing is ever pushed.

NOTE

Students who have seen `!` elsewhere may write `!networks.includes(show.network)`. That's the same as `=== false` and passes.

A CORRECT ANSWER

```
// P13. Every network once
// uniqueNetworks(list) gives back the networks of the shows, without duplicates,
// in the order they first appear.

function uniqueNetworks(list) {
  const networks = [];
  for (const show of list) {
    if (networks.includes(show.network) === false) {
      networks.push(show.network);
    }
  }
  return networks;
}

const mini = [
  { name: "Chernobyl", network: "HBO" },
  { name: "Dark", network: "Netflix" },
  { name: "The Wire", network: "HBO" },
];
const none = [];

tryIt(uniqueNetworks, [mini], ["HBO", "Netflix"]);
tryIt(uniqueNetworks, [none], []);
tryIt(uniqueNetworks, [shows], ["NBC", "AMC", "HBO", "Netflix", "CBS", "BBC Three", "Nickelodeon"]);
```

Practice 14. The most episodes ~12 min

Teaches: Practice: best so far: `let best = list[0];`, replace it inside the loop when an item beats it.

TASK

Make `mostEpisodes(list)` return the **whole show** with the most episodes. For an empty list it gives back `undefined`.

COMMON MISTAKES

- `best = show.episodes` (a number) instead of `best = show` (the whole show).
- Comparing `show.episodes > best` instead of `> best.episodes`.
- `const best`: "Assignment to constant variable".

NOTE

If the last row says `shows[0]`: that's Friends. The starter compares with the first show of data.js because Friends happens to have the most episodes.

A CORRECT ANSWER

```
// P14. The show with the most episodes
// mostEpisodes(list) gives back the WHOLE SHOW with the most episodes.
// An empty list gives undefined.

function mostEpisodes(list) {
  let best = list[0];
  for (const show of list) {
    if (show.episodes > best.episodes) {
      best = show;
    }
  }
  return best;
}

const mini = [
  { name: "Chernobyl", episodes: 5 },
  { name: "The Wire", episodes: 60 },
  { name: "Fleabag", episodes: 12 },
];
const none = [];

tryIt(mostEpisodes, [mini], { name: "The Wire", episodes: 60 });
tryIt(mostEpisodes, [none], undefined);
tryIt(mostEpisodes, [shows], shows[0]);
```

Practice 15. The longest-running show ~12 min

Teaches: Practice: best so far with a calculation, and an early `return` for the empty list.

TASK

Make `longestRunning(list)` return the **name** of the show that ran the most years (`ended` minus `premiered`). For an empty list, return `""`.

COMMON MISTAKES

- Comparing only `show.ended` or only `show.premiered` instead of the difference.
- No check for an empty list: "Cannot read properties of undefined (reading 'name')".
- Returning `best` (the whole show) instead of `best.name`.

A CORRECT ANSWER

```
// P15. The longest-running show
// longestRunning(list) gives back the NAME of the show that ran the most years
// (ended minus premiered). An empty list gives "".

function longestRunning(list) {
  if (list.length === 0) {
    return "";
  }
  let best = list[0];
  for (const show of list) {
    if (show.ended - show.premiered > best.ended - best.premiered) {
      best = show;
    }
  }
  return best.name;
}

const mini = [
  { name: "Band of Brothers", premiered: 2001, ended: 2001 },
  { name: "The Wire", premiered: 2002, ended: 2008 },
  { name: "Dark", premiered: 2017, ended: 2020 },
];
const none = [];

tryIt(longestRunning, [mini], "The Wire");
tryIt(longestRunning, [none], "");
tryIt(longestRunning, [shows], "Friends");
```

Practice 16. Shows per network ~15 min

Teaches: Practice: `obj[key]` reads or sets a property whose name is in a variable.

TASK

Make `countByNetwork(list)` return an object with one key per network and the number of shows on it, like `{ HBO: 2, Netflix: 1 }`. An empty list gives `{}`.

COMMON MISTAKES

- `counts.network` or `counts.show.network` instead of `counts[show.network]`.
- Adding 1 to a key that doesn't exist yet: the count becomes `NaN`.
- Setting every count to 1, without the `else` that adds 1.

NOTE

Square brackets with a variable (`counts[show.network]`) are new here; the steps only used the dot. The extra challenge *Count the genres* uses the same idea, so students who finish this one are ready for it.

A CORRECT ANSWER

```
// P16. How many shows per network?
// countByNetwork(list) gives back an object like { HBO: 2, Netflix: 1 }:
// one key per network, and the number of shows on it.

function countByNetwork(list) {
  const counts = {};
  for (const show of list) {
    if (counts[show.network] === undefined) {
      counts[show.network] = 1;
    } else {
      counts[show.network] = counts[show.network] + 1;
    }
  }
  return counts;
}

const mini = [
  { name: "Chernobyl", network: "HBO" },
  { name: "Dark", network: "Netflix" },
  { name: "The Wire", network: "HBO" },
];
const none = [];

tryIt(countByNetwork, [mini], { HBO: 2, Netflix: 1 });
tryIt(countByNetwork, [none], {});
tryIt(countByNetwork, [shows], { NBC: 2, AMC: 1, HBO: 4, Netflix: 2, CBS: 1, "BBC Three": 1,
Nickelodeon: 1 });
```

Practice 17. One label per show ~8 min

Teaches: Practice: `map` with a template literal makes one text per item.

TASK

Make `labels(list)` return a list with one label per show, like **Friends (NBC, 1994)**: the name, then the network and the year it premiered in round brackets.

COMMON MISTAKES

- Quotes instead of backticks, so `${...}` appears as it is.
- `filter` instead of `map`: you get shows back, not labels.
- `shows.name` inside the arrow instead of `show.name`.

A CORRECT ANSWER

```
// P17. One label per show
// labels(list) gives back a list of labels like "Friends (NBC, 1994)", one per show.

function labels(list) {
  return list.map((show) => `${show.name} (${show.network}, ${show.premiered})`);
}

const mini = [
  { name: "Dark", network: "Netflix", premiered: 2017 },
  { name: "The Wire", network: "HBO", premiered: 2002 },
];
const none = [];

tryIt(labels, [mini], ["Dark (Netflix, 2017)", "The Wire (HBO, 2002)"]);
tryIt(labels, [none], []);
```

Practice 18. How many ended before? ~5 min

Teaches: Practice: `list.filter(...).length` counts the matching items.

TASK

Make `countEndedBefore(list, year)` return **how many** shows in `list` ended **before** `year` (a show that ended in that year itself doesn't count).

COMMON MISTAKES

- Returning the filtered list without `.length`.
- `show.premiered` instead of `show.ended`.
- `<=` instead of `<`.

A CORRECT ANSWER

```
// P18. How many shows ended before a year?
// countEndedBefore(list, year) gives back HOW MANY shows ended before that year.

function countEndedBefore(list, year) {
  return list.filter((show) => show.ended < year).length;
}

const mini = [
  { name: "Friends", ended: 2004 },
  { name: "Dark", ended: 2020 },
  { name: "The Wire", ended: 2008 },
];
const none = [];

tryIt(countEndedBefore, [mini, 2010], 2);
tryIt(countEndedBefore, [mini, 2008], 1);
tryIt(countEndedBefore, [none, 2010], 0);
tryIt(countEndedBefore, [shows, 2010], 4);
```

Practice 19. Which decade? ~8 min

Teaches: Practice: `Math.floor(year / 10) * 10` gives the start of the decade.

TASK

Make `decade(show)` return the decade the show premiered in, as text: **1990s** for Friends (1994), **2010s** for Chernobyl (2019).

COMMON MISTAKES

- `Math.round` instead of `Math.floor`: 2017 becomes the 2020s.
- Forgetting `* 10`: "201s".
- Returning the number 1990 without the `s`.

A CORRECT ANSWER

```
// P19. Which decade did a show start in?
// decade(show) gets ONE show and gives back text like "1990s".

function decade(show) {
  const start = Math.floor(show.premiered / 10) * 10;
  return `${start}s`;
}

tryIt(decade, [shows[0]], "1990s");
tryIt(decade, [shows[6]], "2010s");
tryIt(decade, [{ name: "Band of Brothers", premiered: 2001 }], "2000s");
tryIt(decade, [{ name: "Dark", premiered: 2017 }], "2010s");
```

Practice 20. The rating of a show ~8 min

Teaches: Practice: `if (found === undefined)` before reading `found.rating`.

TASK

Make `ratingOf(list, title)` return the `rating` of the show whose name is `title` (capital letters don't matter). If there's no such show, return `0`.

COMMON MISTAKES

- No `if`: "Cannot read properties of undefined (reading 'rating')" for an unknown name.
- Returning the whole show instead of `found.rating`.
- Making only one side lower case.

A CORRECT ANSWER

```
// P20. The rating of a show, by its name
// ratingOf(list, title) gives back the rating of the show called title
// (capital letters don't matter), or 0 if there is no such show.

function ratingOf(list, title) {
  const wanted = title.toLowerCase();
  const found = list.find((show) => show.name.toLowerCase() === wanted);
  if (found === undefined) {
    return 0;
  }
  return found.rating;
}

const none = [];

tryIt(ratingOf, [shows, "Dark"], 8.7);
tryIt(ratingOf, [shows, "the office"], 8.6);
tryIt(ratingOf, [shows, "CHERNOBYL"], 9.1);
tryIt(ratingOf, [shows, "Lost"], 0);
tryIt(ratingOf, [none, "Dark"], 0);
```

Practice 21. Small objects with map ~10 min

Teaches: Practice: `list.map((x) => ({ key: value }))` makes a list of new objects.

TASK

Make `nameAndYears(list)` return one small object per show, with two properties: `name` (the show's name) and `years` (`ended` minus `premiered`). Friends becomes `{ name: "Friends", years: 10 }`.

COMMON MISTAKES

- No round brackets around the object: every item is `undefined`.
- `premiered - ended` (a negative number of years).
- Keeping the whole show instead of a new object with two properties.

NOTE

The round brackets around an object after `=>` are a real JavaScript quirk. If a student is stuck, show them the longer version with `{ return { ... }; }`, which also passes.

A CORRECT ANSWER

```
// P21. Small objects with map
// nameAndYears(list) gives back one object per show: { name: ..., years: ... },
// where years is ended minus premiered.

function nameAndYears(list) {
  return list.map((show) => ({ name: show.name, years: show.ended - show.premiered }));
}

const mini = [
  { name: "The Wire", premiered: 2002, ended: 2008, network: "HBO" },
  { name: "Chernobyl", premiered: 2019, ended: 2019, network: "HBO" },
];
const none = [];

tryIt(nameAndYears, [mini], [{ name: "The Wire", years: 6 }, { name: "Chernobyl", years: 0 }]);
tryIt(nameAndYears, [none], []);
```

Practice 22. Longer than N years ~8 min

Teaches: Practice: a calculation as the test: `filter((show) => show.ended - show.premiered > n)`.

TASK

Make `longRunners(list, minYears)` return the **names** of the shows that ran **longer than** `minYears` years (`ended` minus `premiered`).

COMMON MISTAKES

- `>=` instead of `>`: shows of exactly `minYears` are included.
- `premiered - ended` (always negative, so nothing is kept).
- Returning the shows without the `map` to names.

A CORRECT ANSWER

```
// P22. Shows that ran longer than N years
// longRunners(list, minYears) gives back the NAMES of the shows with
// (ended minus premiered) greater than minYears.

function longRunners(list, minYears) {
  const long = list.filter((show) => show.ended - show.premiered > minYears);
  return long.map((show) => show.name);
}

const mini = [
  { name: "Friends", premiered: 1994, ended: 2004 },
  { name: "Dark", premiered: 2017, ended: 2020 },
  { name: "The Office", premiered: 2005, ended: 2013 },
];
const none = [];

tryIt(longRunners, [mini, 5], ["Friends", "The Office"]);
tryIt(longRunners, [mini, 8], ["Friends"]);
tryIt(longRunners, [mini, 20], []);
tryIt(longRunners, [none, 1], []);
```

Practice 23. Short and good: two conditions ~10 min

Teaches: Practice: `a && b` is true only when both are true.

TASK

Make `shortAndGood(list, minRating)` return the **names** of the shows with episodes of **30 minutes or less and** a rating of `minRating` **or more**.

COMMON MISTAKES

- Only one of the two tests.
- `show.runtime <= 30 && >= minRating` (the second test needs its own `show.rating`).
- A single `&` instead of `&&`.

NOTE

`&&` is new in this exercise; the explanation introduces it. "And" in plain English is the right way to read it aloud.

A CORRECT ANSWER

```
// P23. Short AND good
// shortAndGood(list, minRating) gives back the NAMES of the shows with
// episodes of 30 minutes or less AND a rating of minRating or more.

function shortAndGood(list, minRating) {
  const picked = list.filter((show) => show.runtime <= 30 && show.rating >= minRating);
  return picked.map((show) => show.name);
}

const mini = [
  { name: "Fleabag", runtime: 30, rating: 8.4 },
  { name: "Dark", runtime: 60, rating: 8.7 },
  { name: "Friends", runtime: 30, rating: 8.8 },
];
const none = [];

tryIt(shortAndGood, [mini, 8.5], ["Friends"]);
tryIt(shortAndGood, [mini, 8], ["Fleabag", "Friends"]);
tryIt(shortAndGood, [none, 8], []);
tryIt(shortAndGood, [shows, 8.5], ["Friends", "The Office", "Avatar: The Last Airbender"]);
```

Practice 24. Started between two years ~10 min

Teaches: Practice: a range: `from <= x && x <= to`.

TASK

Make `startedBetween(list, from, to)` return the **names** of the shows that premiered from the year `from` to the year `to`, **both years included**. Use a loop, an `if` and `push`.

COMMON MISTAKES

- `from <= show.premiered <= to`, written like maths: every show is kept.
- `<` instead of `<=`: the first or last year is left out.
- `const found = [];` inside the loop.

A CORRECT ANSWER

```
// P24. Shows that started between two years
// startedBetween(list, from, to) gives back the NAMES of the shows that
// premiered from the year "from" to the year "to" (both included).
// Use a loop, an if and push.

function startedBetween(list, from, to) {
  const found = [];
  for (const show of list) {
    if (from <= show.premiered && show.premiered <= to) {
      found.push(show.name);
    }
  }
  return found;
}

const mini = [
  { name: "The Office", premiered: 2005 },
  { name: "Breaking Bad", premiered: 2008 },
  { name: "Game of Thrones", premiered: 2011 },
];
const none = [];

tryIt(startedBetween, [mini, 2005, 2010], ["The Office", "Breaking Bad"]);
tryIt(startedBetween, [mini, 2008, 2008], ["Breaking Bad"]);
tryIt(startedBetween, [mini, 1990, 2000], []);
tryIt(startedBetween, [none, 2000, 2020], []);
```

Practice 25. Does every show have a rating? ~8 min

Teaches: Practice: a flag: `let ok = true;`, set it to `false` in the loop when an item fails.

TASK

Make `allRated(list)` return `true` when every show in `list` has a rating, and `false` when at least one has `rating: null`. Use a flag.

COMMON MISTAKES

- An `else` that sets the flag back to `true`, so only the last show counts.
- `return` inside the loop, so only the first show is checked.
- `show.rating === "null"` in quotes (that's the word null, not the value).

A CORRECT ANSWER

```
// P25. Does every show have a rating?
// allRated(list) gives back true if every show has a rating,
// and false if at least one has rating: null.

function allRated(list) {
  let allHave = true;
  for (const show of list) {
    if (show.rating === null) {
      allHave = false;
    }
  }
  return allHave;
}

const oneMissing = [
  { name: "Dark", rating: 8.7 },
  { name: "Show Without Rating", rating: null },
];
const none = [];

tryIt(allRated, [shows], true);
tryIt(allRated, [oneMissing], false);
tryIt(allRated, [none], true);
```

Practice 26. The shortest name ~10 min

Teaches: Practice: smallest so far: replace `best` when `show.name.length < best.name.length`.

TASK

Make `shortestName(list)` return the shortest show name in `list`. If two are equally short, the first one wins. For an empty list, return `""`.

COMMON MISTAKES

- Comparing `show.name < best.name` (alphabetical order, not length).
- `>` instead of `<` (the longest name).
- `<=` instead of `<`: on a tie the last one wins.

A CORRECT ANSWER

```
// P26. The shortest name
// shortestName(list) gives back the shortest show name.
// On a tie the first one wins. An empty list gives "".

function shortestName(list) {
  if (list.length === 0) {
    return "";
  }
  let best = list[0];
  for (const show of list) {
    if (show.name.length < best.name.length) {
      best = show;
    }
  }
  return best.name;
}

const mini = [{ name: "The Crown" }, { name: "Fleabag" }, { name: "Chernobyl" }];
const tie = [{ name: "Friends" }, { name: "Fleabag" }];
const none = [];

tryIt(shortestName, [mini], "Fleabag");
tryIt(shortestName, [tie], "Friends");
tryIt(shortestName, [none], "");
tryIt(shortestName, [shows], "Dark");
```

Practice 27. The lowest rating ~10 min

Teaches: Practice: lowest so far: replace `best` when `show.rating < best.rating`.

TASK

Make `lowestRated(list)` return the **name** of the show with the lowest rating. On a tie, the first one wins. For an empty list, return `""`.

COMMON MISTAKES

- `>` instead of `<` (the highest rating).
- `<=` instead of `<`: on the tie, How I Met Your Mother wins instead of The Crown.
- No empty-list check: a crash on `best.name`.

A CORRECT ANSWER

```
// P27. The lowest rating
// lowestRated(list) gives back the NAME of the show with the lowest rating.
// On a tie the first one wins. An empty list gives "".

function lowestRated(list) {
  if (list.length === 0) {
    return "";
  }
  let best = list[0];
  for (const show of list) {
    if (show.rating < best.rating) {
      best = show;
    }
  }
  return best.name;
}

const mini = [
  { name: "Dark", rating: 8.7 },
  { name: "Fleabag", rating: 8.4 },
  { name: "Chernobyl", rating: 9.1 },
];
const none = [];

tryIt(lowestRated, [mini], "Fleabag");
tryIt(lowestRated, [none], "");
tryIt(lowestRated, [shows], "The Crown");
```

Practice 28. A watchlist from ids (harder) ~12 min

Teaches: Practice: loop through ids, `find` each show, push it if found.

TASK

Make `watchlist(list, ids)` return the **names** of the shows whose ids are in `ids`, in the order of `ids`. Skip ids that no show has.

COMMON MISTAKES

- Looping through `list` instead of `ids`: the names come in the wrong order.
- No `if`: id 99 crashes with "Cannot read properties of undefined".
- `show.id === ids` (the whole list) instead of `show.id === id`.

A CORRECT ANSWER

```
// P28. A watchlist from ids
// watchlist(list, ids) gives back the NAMES of the shows with those ids,
// in the order of ids. Ids that no show has are skipped.

function watchlist(list, ids) {
  const names = [];
  for (const id of ids) {
    const found = list.find((show) => show.id === id);
    if (found !== undefined) {
      names.push(found.name);
    }
  }
  return names;
}

tryIt(watchlist, [shows, [7, 1]], ["Chernobyl", "Friends"]);
tryIt(watchlist, [shows, [2, 99, 4]], ["Breaking Bad", "Game of Thrones"]);
tryIt(watchlist, [shows, []], []);
```

Practice 29. Every genre once (harder) ~12 min

Teaches: Practice: a loop inside a loop: `for (const genre of show.genres)` inside `for (const show of list)`.

TASK

Make `allGenres(list)` return every genre of the shows in `list`, each only once, in the order they first appear.

COMMON MISTAKES

- Pushing `show.genres` (a whole array) instead of each `genre`.
- The `includes` test missing, so Comedy and Drama appear many times.
- `return` between the two closing `}` (after the first show only).

A CORRECT ANSWER

```
// P29. Every genre once
// allGenres(list) gives back every genre of the shows, without duplicates,
// in the order they first appear.

function allGenres(list) {
  const genres = [];
  for (const show of list) {
    for (const genre of show.genres) {
      if (genres.includes(genre) === false) {
        genres.push(genre);
      }
    }
  }
  return genres;
}

const mini = [
  { name: "Friends", genres: ["Comedy", "Romance"] },
  { name: "Fleabag", genres: ["Comedy", "Drama"] },
];
const none = [];

tryIt(allGenres, [mini], ["Comedy", "Romance", "Drama"]);
tryIt(allGenres, [none], []);
```

Practice 30. The best in a genre (harder) ~12 min

Teaches: Practice: `filter` to narrow the list, then "best so far" on the result.

TASK

Make `bestInGenre(list, genre)` return the **name** of the highest-rated show in `list` with that genre. If no show has it, return `""`.

COMMON MISTAKES

- Looping through `list` instead of the filtered list: the best show of any genre.
- No check for an empty result: a crash for Western.
- `show.genres === genre` instead of `show.genres.includes(genre)`.

A CORRECT ANSWER

```
// P30. The best show in a genre
// bestInGenre(list, genre) gives back the NAME of the highest-rated show
// with that genre, or "" if no show has it.

function bestInGenre(list, genre) {
  const inGenre = list.filter((show) => show.genres.includes(genre));
  if (inGenre.length === 0) {
    return "";
  }
  let best = inGenre[0];
  for (const show of inGenre) {
    if (show.rating > best.rating) {
      best = show;
    }
  }
  return best.name;
}

const mini = [
  { name: "Dark", rating: 8.7, genres: ["Drama", "Mystery"] },
  { name: "Fleabag", rating: 8.4, genres: ["Comedy", "Drama"] },
  { name: "Friends", rating: 8.8, genres: ["Comedy", "Romance"] },
];

tryIt(bestInGenre, [mini, "Comedy"], "Friends");
tryIt(bestInGenre, [mini, "Drama"], "Dark");
tryIt(bestInGenre, [mini, "Western"], "");
tryIt(bestInGenre, [shows, "Fantasy"], "Avatar: The Last Airbender");
```

Practice 31. Rating bands (harder) ~12 min

Teaches: Practice: counters in one object: `{ a: 0, b: 0 }`, then `obj.a = obj.a + 1` in the right branch.

TASK

Make `ratingBands(list)` return an object `{ mustWatch: ..., good: ..., ok: ... }` that counts the shows rated **9 or more** (`mustWatch`), **8 or more** but under 9 (`good`), and **under 8** (`ok`). An empty list gives all three at 0.

COMMON MISTAKES

- `>= 8` tested before `>= 9` : every show lands in `good` .
- `>` instead of `>=` : shows rated exactly 9.0 count as good.
- Creating the object inside the loop.

A CORRECT ANSWER

```
// P31. Rating bands
// ratingBands(list) counts the shows in three bands and gives back
// { mustWatch: ..., good: ..., ok: ... }:
// mustWatch = rated 9 or more, good = 8 or more, ok = under 8.

function ratingBands(list) {
  const bands = { mustWatch: 0, good: 0, ok: 0 };
  for (const show of list) {
    if (show.rating >= 9) {
      bands.mustWatch = bands.mustWatch + 1;
    } else if (show.rating >= 8) {
      bands.good = bands.good + 1;
    } else {
      bands.ok = bands.ok + 1;
    }
  }
  return bands;
}

const mini = [
  { name: "Chernobyl", rating: 9.1 },
  { name: "Dark", rating: 8.7 },
  { name: "Fleabag", rating: 8.4 },
];
const none = [];

tryIt(ratingBands, [mini], { mustWatch: 1, good: 2, ok: 0 });
tryIt(ratingBands, [none], { mustWatch: 0, good: 0, ok: 0 });
tryIt(ratingBands, [shows], { mustWatch: 5, good: 7, ok: 0 });
```

Practice 32. Names per network (harder) ~15 min

Teaches: Practice: group into an object of arrays: create `[]` the first time, then `push`.

TASK

Make `namesByNetwork(list)` return an object with one key per network, whose value is the list of the names of the shows on that network, in the order of `list`. An empty list gives `{}`.

COMMON MISTAKES

- `groups[show.network] = [show.name]` every time: only the last name per network is left.
- `push` before the empty list exists: "Cannot read properties of undefined (reading 'push')".
- `groups.network` instead of `groups[show.network]`.

A CORRECT ANSWER

```
// P32. Names per network
// namesByNetwork(list) gives back an object like
// { HBO: ["Chernobyl", "The Wire"], Netflix: ["Dark"] }

function namesByNetwork(list) {
  const groups = {};
  for (const show of list) {
    if (groups[show.network] === undefined) {
      groups[show.network] = [];
    }
    groups[show.network].push(show.name);
  }
  return groups;
}

const mini = [
  { name: "Chernobyl", network: "HBO" },
  { name: "Dark", network: "Netflix" },
  { name: "The Wire", network: "HBO" },
];
const none = [];

tryIt(namesByNetwork, [mini], { HBO: ["Chernobyl", "The Wire"], Netflix: ["Dark"] });
tryIt(namesByNetwork, [none], {});
```

Extra 1. Best first ~20 min

Teaches: `[...list].sort((a, b) => b.x - a.x)` sorts a copy, biggest first.

TASK

Write `sortByRating(list)` that returns a **copy** of the list, highest rating first. A new section, **Best first**, appears on the page.

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.genres.includes(genre));
}

function sortByRating(list) {
  return [...list].sort((a, b) => b.rating - a.rating);
}
```

Extra 2. Count the genres ~20 min

Teaches: `obj[key]` reads or sets a property whose name is in a variable.

TASK

Write `countByGenre(list)` that returns an object like `{ Comedy: 4, Drama: 8, ... }`. Use loops and an object, not a new array method.

When it works, the genre buttons on the page show their counts.

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.genres.includes(genre));
}

function countByGenre(list) {
  const counts = {};
  for (const show of list) {
    for (const genre of show.genres) {
      if (counts[genre]) {
        counts[genre] = counts[genre] + 1;
      } else {
        counts[genre] = 1;
      }
    }
  }
  return counts;
}
```

Extra 3. Watch everything ~15 min

Teaches: Add up a list: `let total = 0;` before the loop, `total = total + ...` inside.

TASK

Write `totalWatchTime(list)` that returns the total minutes needed to watch every episode of every show in the list (each show: episodes times runtime).

A new section, **Watch everything**, appears on the page.

A CORRECT ANSWER

```
// ... unchanged ...
  return list.filter((show) => show.genres.includes(genre));
}

function totalWatchTime(list) {
  let total = 0;
  for (const show of list) {
    total = total + show.episodes * show.runtime;
  }
  return total;
}
```

Day 7: Make the page react The DOM and events

Until now your JavaScript ran once and printed to the Console. Today the page **reacts**. You build the show cards with JavaScript, filter them while someone types, and add a watchlist called **My List** that is still there after a reload.

You work in one file, **app.js**. The page itself, the styles and two small files are ready. Open the read-only tabs above the code box to look at them:

- **data.js** holds `shows`, the 12 shows you know.
- **helpers.js** has functions you already know, like `ratingLabel`, `yearsLabel` and `searchByName`.

Your code grows from step to step: each step starts with your code from the step before.

Step 1. Find an element ~10 min

Teaches: `document.querySelector("#id")` finds an element; `textContent` changes its text.

TASK

Under the comment `Elements`:

1. Find the grid and keep it in a constant: `const grid = document.querySelector("#grid");`
2. As a test, put some text in it, for example "The shows will appear here."
3. Log the element with `console.log(grid);` and look at the Console under the preview.

COMMON MISTAKES

- Forgetting the `#`: `querySelector("grid")` looks for a `<grid>` tag and gives `null`.
- `queryselector` or `QuerySelector`: capital letters matter.
- Writing `grid.textContent` before the `const grid` line.

A CORRECT ANSWER

```
// ... unchanged ...

// ---------- Elements: the parts of the page we change ----------
const grid = document.querySelector("#grid");
grid.textContent = "The shows will appear here.";
console.log(grid);
```

Step 2. Build an element ~15 min

Teaches: `document.createElement("tag")` makes an element; `parent.append(child)` puts it on the page.

TASK

1. Delete the `grid.textContent = ...` line from step 1. The `console.log` can go too.
2. Under `const grid`, create an `article` element with the class `card`.
3. Create an `h3` whose text is the first show's name: `shows[0].name`.
4. Put the `h3` inside the article, and the article inside the grid.

A small white card with "Friends" appears in the preview.

COMMON MISTAKES

- `card.class = "card"`: the property is called `className`.
- Appending the card to the grid before putting the title in it (it works, but it's easier to read in the other order).
- Forgetting `grid.append(card)`, so the card never appears.

A CORRECT ANSWER

```
// ... unchanged ...
// ---------- Elements: the parts of the page we change ----------
const grid = document.querySelector("#grid");

// ---------- A first card ----------
const card = document.createElement("article");
card.className = "card";

const title = document.createElement("h3");
title.textContent = shows[0].name;

card.append(title);
grid.append(card);
```

Step 3. A function that builds a card ~20 min

Teaches: A function can build an element from an object and `return` it.

TASK

Fill in the three TODO lines inside `createCard`, using the helpers from **helpers.js**:

1. The badge shows the first genre: `show.genres[0]`.
2. The rating line reads like `★ 8.8 · Must watch`: the rating, then the label from `ratingLabel(show.rating)`. Copy the `★` and `·` from the comment, or use `*` and `-`.
3. The meta line shows the years: `yearsLabel(show.premiered, show.ended)`.

COMMON MISTAKES

- `badge.textContent = show.genres;` shows every genre with commas. Use `[0]` for the first one.
- Missing `+` between the pieces of the rating text, which gives a syntax error.
- Deleting `return card;`, so `createCard` gives back `undefined`.

NOTE

The code box starts fresh in this step: their step 2 lines were moved into the function. If a student asks where their code went, show them the `card` and `title` lines at the top of `createCard`.

A CORRECT ANSWER

```
// app.js: makes the ShowShelf page react (Day 7)
// Loaded before this file (open the read-only tabs to see them):
//   data.js    -> shows: the 12 shows
//   helpers.js -> ratingLabel, yearsLabel, searchByName, getByGenre, allGenres

// ---------- Elements: the parts of the page we change ----------
const grid = document.querySelector("#grid");

// ---------- One card for one show ----------
// createCard(show) builds a card and gives it back. It doesn't put it on the page.
function createCard(show) {
  const card = document.createElement("article");
  card.className = "card";

  const poster = document.createElement("img");
  poster.className = "poster";
  poster.src = show.image ?? "poster.svg";   // no picture (null)? use poster.svg
  poster.alt = "";                           // the name is under the poster, so no alt text

  const body = document.createElement("div");
  body.className = "card-body";

  const badge = document.createElement("span");
  badge.className = "badge";
  badge.textContent = show.genres[0];

  const title = document.createElement("h3");
  title.textContent = show.name;

  const rating = document.createElement("p");
  rating.className = "rating";
  rating.textContent = "★ " + show.rating + " · " + ratingLabel(show.rating);

  const meta = document.createElement("p");
  meta.className = "meta";
  meta.textContent = yearsLabel(show.premiered, show.ended);
```

```
  body.append(badge, title, rating, meta);
  card.append(poster, body);
  return card;
}

// Try it with the first show
grid.append(createCard(shows[0]));
```

Step 4. A card for every show ~10 min

Teaches: A loop plus a function: one element for every item in an array.

TASK

Replace the last line, `grid.append(createCard(shows[0]));`, with a `for...of` loop that adds a card for **every** show in `shows`.

COMMON MISTAKES

- Keeping the old line, so Friends appears twice.
- `createCard(shows)` (the whole array) instead of `createCard(show)`.

A CORRECT ANSWER

```
// ... unchanged ...
}

// One card for every show
for (const show of shows) {
  grid.append(createCard(show));
}
```

Step 5. Draw any list of shows ~15 min

Teaches: `renderGrid(list)` empties the grid with `replaceChildren()` and draws a card for each show.

TASK

1. Put your loop inside a new function `renderGrid(list)`. Make `grid.replaceChildren();` its first line, and loop over `list`.
2. Under the function, call it with all the shows: `renderGrid(shows);`

COMMON MISTAKES

- Forgetting `grid.replaceChildren()`, so cards pile up.
- Looping over `shows` inside the function, so the parameter is ignored.

A CORRECT ANSWER

```
// ... unchanged ...
}

// ---------- Draw a list of shows in the grid ----------
function renderGrid(list) {
  grid.replaceChildren();   // remove the old cards first
  for (const show of list) {
    grid.append(createCard(show));
  }
}

renderGrid(shows);
```

Step 6. React to typing ~15 min

Teaches: `element.addEventListener("input", () => { ... })` runs code on every change; `.value` is the text in a box.

TASK

1. At the top, under `const grid`, find the search box: `const searchInput = document.querySelector("#search");`
2. At the bottom, add an `"input"` listener to `searchInput`. Inside it, draw only the matching shows: `renderGrid(searchByName(shows, searchInput.value));`

Then type **office** in the search box in the preview, and delete it again.

COMMON MISTAKES

- `searchInput.value` read once outside the listener: it's empty at that moment and never updates.
- Using the `"change"` event, which waits until you leave the box.
- Calling the function instead of giving it: `addEventListener("input", renderGrid(...))`.

A CORRECT ANSWER

```
// ... unchanged ...
// ---------- Elements: the parts of the page we change ----------
const grid = document.querySelector("#grid");
const searchInput = document.querySelector("#search");

// ---------- One card for one show ----------
// ... unchanged ...

renderGrid(shows);

// ---------- Events: what happens when someone uses the page ----------
// Every time the text in the search box changes, show only the shows that match
searchInput.addEventListener("input", () => {
  renderGrid(searchByName(shows, searchInput.value));
});
```

Step 7. Stop the page reloading ~10 min

Teaches: `event.preventDefault()` stops the browser's default action, like a form reloading the page.

TASK

1. At the top, find the form: `const searchForm = document.querySelector("#search-form");`
2. At the bottom, add a `"submit"` listener to `searchForm` that calls `event.preventDefault();`.

Press Enter in the search box again: no notice this time.

COMMON MISTAKES

- Adding the listener to the button with `"click"`: Enter in the box still submits the form.
- `event.preventDefault` without brackets: nothing happens.
- Forgetting the `event` parameter: `event is not defined`.

A CORRECT ANSWER

```
// ... unchanged ...
const grid = document.querySelector("#grid");
const searchInput = document.querySelector("#search");
const searchForm = document.querySelector("#search-form");

// ---------- One card for one show ----------
// ... unchanged ...
  renderGrid(searchByName(shows, searchInput.value));
});

// Pressing Enter sends ("submits") the form. Stop the browser from reloading the page.
searchForm.addEventListener("submit", (event) => {
  event.preventDefault();
});
```

Step 8. When nothing matches ~10 min

Teaches: `element.hidden = condition;` shows or hides an element.

TASK

1. At the top, find the message: `const emptyMessage = document.querySelector("#empty-message");`
2. At the end of `renderGrid`, add one line that hides the message when the list has shows, and shows it when the list is empty.

Search for zzz, then for dark.

COMMON MISTAKES

- The condition the wrong way round: `list.length === 0` hides the message exactly when it's needed.
- Putting the line outside `renderGrid`, so it only runs once.
- `const emptyMessage` at the bottom of the file: "Cannot access before initialization".

A CORRECT ANSWER

```
// ... unchanged ...
const searchInput = document.querySelector("#search");
const searchForm = document.querySelector("#search-form");
const emptyMessage = document.querySelector("#empty-message");

// ---------- One card for one show ----------
// ... unchanged ...
    grid.append(createCard(show));
  }
  emptyMessage.hidden = list.length > 0;   // show the message only when there are no shows
}

// ... unchanged ...
```

Step 9. Genre buttons and one render() ~30 min

Teaches: Keep what can change in a `state` object; events change the state, then call `render()`.

TASK

1. **TODO 1** in `render()`: if `state.genre` is not `""`, keep only that genre, with `getByGenre(list, state.genre)`.
2. **TODO 2** in the chip's click listener: if `state.genre` is already this `genre`, set it to `""`; otherwise set it to `genre`. Then call `render()`.
3. In your search listener, replace the `renderGrid(...)` line with two lines: `state.search = searchInput.value;` and `render();`.
4. Delete the old line `renderGrid(shows);` under the `renderGrid` function. The `render();` at the bottom draws the page now.

Try it: click **Drama**, then type **the**. Click Drama again to see all genres.

COMMON MISTAKES

- Forgetting `render()` after changing `state.genre`: the state changes but the page doesn't.
- Leaving `renderGrid(searchByName(...))` in the search listener, so typing ignores the chosen genre.
- `if (state.genre = genre)` with one `=`: that sets the value instead of comparing.

NOTE

This step is longer than the others. The key idea is item 4: the search listener stops drawing and only changes `state`. If a student is lost, draw three boxes on paper: event, state, render, with arrows in that order.

A CORRECT ANSWER

```
// ... unchanged ...

// ---------- Events: what happens when someone uses the page ----------
// Every time the text in the search box changes: change the state, then draw again
searchInput.addEventListener("input", () => {
  state.search = searchInput.value;
  render();
});

// ... unchanged ...
  event.preventDefault();
});

// ---------- State: everything on the page that can change ----------
const state = {
  search: "",   // the text in the search box
  genre: "",    // the chosen genre ("" means all genres)
};

// ---------- Draw the page from the state ----------
function render() {
  let list = searchByName(shows, state.search);
  if (state.genre !== "") {
    list = getByGenre(list, state.genre);
  }

  renderGrid(list);
  renderGenres();
}

// One button (a "chip") per genre, in the box with the id "genres". The chosen one gets the class
```

```
"active".
const genresBox = document.querySelector("#genres");
function renderGenres() {
  genresBox.replaceChildren();
  for (const genre of allGenres(shows)) {
    const chip = document.createElement("button");
    chip.className = "chip";
    chip.textContent = genre;
    if (genre === state.genre) {
      chip.classList.add("active");
    }
    chip.addEventListener("click", () => {
      if (state.genre === genre) {
        state.genre = "";      // clicked the chosen genre again: all genres
      } else {
        state.genre = genre;
      }
      render();
    });
    genresBox.append(chip);
  }
}

// ---------- Start: draw the page for the first time ----------
render();
```

Step 10. My List ~25 min

Teaches: My List is an array of ids in the state: `push` adds, `filter` removes, `includes` checks.

TASK

1. In the `state` object, add `myList: []`.
2. At the top, find the counter in the header: `const listCount = document.querySelector("#list-count");`
3. Fill in **TODO 1** and **TODO 2** in `toggleMyList`.
4. In `createCard`, right before `return card;`, add the button: `card.append(createListButton(show));`
5. At the end of `render()`, show how many shows are in My List: `listCount.textContent = state.myList.length;`

Click + on a few cards and watch the counter at the top.

COMMON MISTAKES

- `state.myList.filter(...)` without `state.myList =` in front: `filter` makes a new array and doesn't change the old one.
- Adding the button after `return card;`, where the code never runs.
- `listCount.textContent` set outside `render()`, so it doesn't update on clicks.

A CORRECT ANSWER

```
// ... unchanged ...
const searchForm = document.querySelector("#search-form");
const emptyMessage = document.querySelector("#empty-message");
const listCount = document.querySelector("#list-count");

// ---------- One card for one show ----------
// ... unchanged ...
  body.append(badge, title, rating, meta);
  card.append(poster, body);
  card.append(createListButton(show));
  return card;
}
// ... unchanged ...
  search: "",   // the text in the search box
  genre: "",    // the chosen genre ("" means all genres)
  myList: [],   // the ids of the shows in My List, like [2, 5]
};

// ... unchanged ...
  renderGrid(list);
  renderGenres();
  listCount.textContent = state.myList.length;
}

// ... unchanged ...
// ---------- Start: draw the page for the first time ----------
render();

// ---------- My List ----------

// Is this show in My List? Gives true or false.
function isInMyList(id) {
  return state.myList.includes(id);
}
```

```
// Add the show to My List, or take it out if it's already there
function toggleMyList(show) {
  if (isInMyList(show.id)) {
    state.myList = state.myList.filter((id) => id !== show.id);
  } else {
    state.myList.push(show.id);
  }
  render();
}

// The round button on a card: "+" adds the show, "✓" means it's in My List
function createListButton(show) {
  const button = document.createElement("button");
  button.className = "list-btn";
  if (isInMyList(show.id)) {
    button.textContent = "✓";
  } else {
    button.textContent = "+";
  }
  button.setAttribute("aria-pressed", isInMyList(show.id));   // "pressed" look, and for screen
readers
  button.setAttribute("aria-label", "My List: " + show.name);
  button.addEventListener("click", () => {
    toggleMyList(show);
  });
  return button;
}
```

Step 11. Remember My List ~15 min

Teaches: `localStorage.setItem/getItem` with `JSON.stringify/parse` saves data across reloads.

TASK

1. Fill in the TODO in `saveMyList`, using the key `"myList"`.
2. In `toggleMyList`, call `saveMyList();` right before `render();`.
3. In `state`, start `myList` from the saved list: `JSON.parse(localStorage.getItem("myList")) ?? []`.

Add two shows, then press **Run again**: they're still in My List.

COMMON MISTAKES

- Saving the array without `JSON.stringify`: it's stored as `"2,5"`, which `JSON.parse` can't read back as an array.
- Forgetting `JSON.parse` when loading, so `myList` is a piece of text.
- Calling `saveMyList()` after `render()` in a different function, or not at all.

NOTE

If a student's page shows ticks from earlier experiments, they can click them away. The Lab keeps localStorage between reloads on purpose, like a real browser does.

A CORRECT ANSWER

```
// ... unchanged ...
  search: "",   // the text in the search box
  genre: "",    // the chosen genre ("" means all genres)
  myList: JSON.parse(localStorage.getItem("myList")) ?? [],   // saved ids, or an empty list
};

// ... unchanged ...
    state.myList.push(show.id);
  }
  saveMyList();
  render();
}
// ... unchanged ...
  return button;
}

// Save My List in the browser (localStorage keeps text only, so we turn it into JSON text)
function saveMyList() {
  localStorage.setItem("myList", JSON.stringify(state.myList));
}
```

Extra 1. Only My List ~15 min

Teaches: A checkbox's `checked` and `"change"` event can drive a filter in `render()`.

TASK

1. At the top: `const onlyListBox = document.querySelector("#only-list");`
2. Add `onlyMyList: false` to `state`.
3. Add a `"change"` listener to the checkbox that sets `state.onlyMyList` and calls `render()`.
4. In `render()`, before `renderGrid(list)`: if `state.onlyMyList` is true, keep only the shows that are in My List (`filter` with `isInMyList(show.id)`).

A CORRECT ANSWER

```
// ... unchanged ...
const emptyMessage = document.querySelector("#empty-message");
const listCount = document.querySelector("#list-count");
const onlyListBox = document.querySelector("#only-list");

// ---------- One card for one show ----------
// ... unchanged ...
  genre: "",    // the chosen genre ("" means all genres)
  myList: JSON.parse(localStorage.getItem("myList")) ?? [],   // saved ids, or an empty list
  onlyMyList: false,  // is "Only My List" ticked?
};

// ... unchanged ...
    list = getByGenre(list, state.genre);
  }
  if (state.onlyMyList) {
    list = list.filter((show) => isInMyList(show.id));
  }

  renderGrid(list);
// ... unchanged ...
  localStorage.setItem("myList", JSON.stringify(state.myList));
}

// Only My List: tick the box to see only your shows
onlyListBox.addEventListener("change", () => {
  state.onlyMyList = onlyListBox.checked;
  render();
});
```

Extra 2. Best rated first ~15 min

Teaches: `list.sort((a, b) => b.rating - a.rating)` sorts from highest to lowest.

TASK

1. At the top: `const sortSelect = document.querySelector("#sort");`
2. Add `sort: ""` to `state`.
3. Add a `"change"` listener to the select that sets `state.sort` to `sortSelect.value` and calls `render()`.
4. In `render()`, before `renderGrid(list)`: if `state.sort` is `"rating"`, sort `list` with the best rating first.

A CORRECT ANSWER

```
// ... unchanged ...
const emptyMessage = document.querySelector("#empty-message");
const listCount = document.querySelector("#list-count");
const sortSelect = document.querySelector("#sort");

// ---------- One card for one show ----------
// ... unchanged ...
  genre: "",    // the chosen genre ("" means all genres)
  myList: JSON.parse(localStorage.getItem("myList")) ?? [],   // saved ids, or an empty list
  sort: "",     // "" for normal order, "rating" for best rated first
};

// ... unchanged ...
    list = getByGenre(list, state.genre);
  }
  if (state.sort === "rating") {
    list.sort((a, b) => b.rating - a.rating);   // highest rating first
  }

  renderGrid(list);
// ... unchanged ...
  localStorage.setItem("myList", JSON.stringify(state.myList));
}

// The order drop-down
sortSelect.addEventListener("change", () => {
  state.sort = sortSelect.value;
  render();
});
```

Extra 3. A details box ~20 min

Teaches: `dialog.showModal()` opens a dialog; `event.stopPropagation()` stops a click bubbling up.

TASK

1. At the top, find the dialog: `const detailsDialog = document.querySelector("#details");`
2. Write `openDetails(show)`: set the text of `#details-title` (name), `#details-meta` (years) and `#details-summary` (`show.summary`), then call `detailsDialog.showModal()`.
3. In `createCard`, before `return card;`, add a click listener to the card that calls `openDetails(show)`.
4. The close button `#close-details` closes the dialog.
5. In `createListButton`, add `event.stopPropagation();` so the + button doesn't open the dialog.

A CORRECT ANSWER

```
// ... unchanged ...
const emptyMessage = document.querySelector("#empty-message");
const listCount = document.querySelector("#list-count");
const detailsDialog = document.querySelector("#details");

// ---------- One card for one show ----------
// ... unchanged ...
  card.append(poster, body);
  card.append(createListButton(show));
  card.addEventListener("click", () => {
    openDetails(show);
  });
  return card;
}
// ... unchanged ...
  button.setAttribute("aria-pressed", isInMyList(show.id));   // "pressed" look, and for screen
readers
  button.setAttribute("aria-label", "My List: " + show.name);
  button.addEventListener("click", (event) => {
    event.stopPropagation();   // don't also open the details box
    toggleMyList(show);
  });
// ... unchanged ...
  localStorage.setItem("myList", JSON.stringify(state.myList));
}

// ---------- Details box ----------
function openDetails(show) {
  document.querySelector("#details-title").textContent = show.name;
  document.querySelector("#details-meta").textContent = yearsLabel(show.premiered, show.ended);
  document.querySelector("#details-summary").textContent = show.summary;
  detailsDialog.showModal();
}

document.querySelector("#close-details").addEventListener("click", () => {
  detailsDialog.close();
});
```

Day 8: Real shows from the internet fetch and APIs

Yesterday's ShowShelf knew 12 shows. Today it talks to **TVmaze**, a free online database of TV shows. The search box asks TVmaze, the page starts with real shows, and it tells people when it's loading or when something went wrong.

Your code box starts with the finished app from Day 7, tidied into sections. A few lines are new or changed for today; they say **Day 8** in their comments:

- `state.shows` holds the shows on the page. For now it starts with the 12 sample shows from data.js.
- Search-as-you-type is gone: from today the search goes to TVmaze when the form is sent.
- Cards cope with real shows that have no genre or no rating yet.

No internet? The Lab answers with saved sample data from TVmaze, so every step works offline too.

Step 1. What is an API? ~15 min

Teaches: An API answers a web address with data. Template literals `` `...${x}...` `` `build text;` encodeURIComponent` makes it safe for a URL.

TASK

Under the Day 8 comment at the bottom:

1. Keep the start of every address in a constant: `const API = "https://api.tvmaze.com";`
2. Write a function `searchUrl(term)` that returns the search address for `term`, built with a template literal, `API` and `encodeURIComponent(term)`.
3. Log `searchUrl("the office")` and look at the Console.

COMMON MISTAKES

- Quotes instead of backticks: `"${API}/search..."` keeps the `${...}` as plain text.
- A space or a missing slash: `${API}search/shows`.
- Forgetting `return` in `searchUrl`.

A CORRECT ANSWER

```
// ... unchanged ...
// ---------- Day 8: talking to TVmaze ----------
// Your code for today goes below this line.

// The address of the TVmaze API. Every request starts with it.
const API = "https://api.tvmaze.com";

// The search address: searchUrl("the office") -> ".../search/shows?q=the%20office"
function searchUrl(term) {
  return `${API}/search/shows?q=${encodeURIComponent(term)}`;
}

console.log(searchUrl("the office"));
```

Step 2. Your first fetch ~15 min

Teaches: `await fetch(url)` then `await response.json()`, inside an `async function`.

TASK

Under your `searchUrl` code:

1. Write an `async function tryFetch()` that fetches `searchUrl("office")`, reads the JSON into a constant `results`, and logs `results`.
2. Call it: `tryFetch();`

Look at the Console: a long answer from TVmaze.

COMMON MISTAKES

- Forgetting `await`: you log a promise (`{}`) instead of the data.
- `await` in a function without `async`: "await is only valid in async functions".
- Writing the function but never calling it.

NOTE

If a student's Console says the Lab used sample data, that's fine: every check works offline. Online, the preview asks TVmaze again each time the code runs, so ask students not to hammer "Run again".

A CORRECT ANSWER

```
// ... unchanged ...

console.log(searchUrl("the office"));

// A first request to TVmaze
async function tryFetch() {
  const response = await fetch(searchUrl("office"));
  const results = await response.json();
  console.log(results);
}

tryFetch();
```

Step 3. The shape of the answer ~10 min

Teaches: Read the shape of JSON before using it: a search gives `[{ score, show }]`.

TASK

In `tryFetch`, replace `console.log(results);` with two logs:

1. how many results there are: `results.length`
2. the name of the first show found

COMMON MISTAKES

- `results.show.name`: `results` is an array, so it needs `[0]` first.
- `results[0].name`: the name is inside `show`.

A CORRECT ANSWER

```
// ... unchanged ...
  const response = await fetch(searchUrl("office"));
  const results = await response.json();
  console.log(results.length);
  console.log(results[0].show.name);
}

// ... unchanged ...
```

Step 4. From their shape to ours ~20 min

Teaches: An adapter (`toShow`) turns someone else's data shape into yours; `map` applies it to every item.

TASK

Turn `tryFetch` into a real search function:

1. Rename it to `searchShows` and give it a parameter: `async function searchShows(term)`.
2. Fetch `searchUrl(term)` instead of `searchUrl("office")`.
3. Replace the two logs with `return results.map((result) => toShow(result.show));`
4. Tidy up: delete the test lines `tryFetch();` and `console.log(searchUrl("the office"));`.

COMMON MISTAKES

- `results.map(toShow)`: here each item is `{ score, show }`, so it needs `toShow(result.show)`.
- Keeping `"office"` inside the function instead of `term`.
- Forgetting `return`: the function gives back `undefined`.

A CORRECT ANSWER

```
// ... unchanged ...
}

// Search TVmaze. Gives back a list of shows in OUR shape, ready for createCard.
async function searchShows(term) {
  const response = await fetch(searchUrl(term));
  const results = await response.json();
  return results.map((result) => toShow(result.show));
}
```

Step 5. Search TVmaze ~15 min

Teaches: An `async` function can `await` data, put it in the state and call `render()`.

TASK

1. Fill in the two TODOs in `loadShows`.
2. In the `"submit"` listener (in the Events section), under `event.preventDefault();`, call `loadShows(searchInput.value);`.

Type **friends** in the search box and press Enter.

COMMON MISTAKES

- `state.shows = searchShows(term);` without `await`: the state holds a promise.
- Calling `loadShows` outside the submit listener, so it runs once at start with an empty box.
- Using `searchInput` before it has a value, for example in a `const` at the top.

A CORRECT ANSWER

```
// ... unchanged ...
searchForm.addEventListener("submit", (event) => {
  event.preventDefault();   // don't reload the page
  loadShows(searchInput.value);
});

// ... unchanged ...
  return results.map((result) => toShow(result.show));
}

// Search TVmaze, then show the results on the page
async function loadShows(term) {
  state.shows = await searchShows(term);
  state.genre = "";   // the new shows may not have the chosen genre
  render();
}
```

Step 6. Loading... ~15 min

Teaches: Show a loading message before `await`, and hide it after.

TASK

1. At the top, in the Elements section, find the status line: `const statusMessage = document.querySelector("#status");`
2. At the start of `loadShows`, before the `await`: set its text to "Loading shows..." and show it (`hidden = false`). Also hide `emptyMessage`: an old "nothing found" doesn't belong to the new search.
3. After the `await`: hide the status line again.

Search for something and watch closely: the message flashes up while TVmaze answers.

COMMON MISTAKES

- Hiding the message before the `await`, so it never shows.
- `statusMessage.hidden = "false"` in quotes: any non-empty text counts as true.
- Forgetting the `const statusMessage` line at the top.

A CORRECT ANSWER

```
// ... unchanged ...
const genresBox = document.querySelector("#genres");
const listCount = document.querySelector("#list-count");
const statusMessage = document.querySelector("#status");   // Day 8: "Loading..." and errors

// ---------- State: everything on the page that can change ----------
// ... unchanged ...
// Search TVmaze, then show the results on the page
async function loadShows(term) {
  statusMessage.textContent = "Loading shows…";
  statusMessage.hidden = false;
  emptyMessage.hidden = true;   // an old "nothing found" doesn't belong to the new search
  state.shows = await searchShows(term);
  statusMessage.hidden = true;
  state.genre = "";   // the new shows may not have the chosen genre
  render();
// ... unchanged ...
```

Step 7. When things go wrong ~20 min

Teaches: Check `response.ok`, `throw new Error(...)`, and handle failures with `try / catch`.

TASK

1. In `searchShows`, straight after the `fetch` line: if `response.ok` is false, `throw new Error("TVmaze answered " + response.status);`
2. In `loadShows`, put the `await` line and the lines after it inside `try { }`. In `catch (error) { }`, set the status text to a friendly message and add the class `error`: `statusMessage.classList.add("error");`
3. At the start of `loadShows`, remove the class again, so the next "Loading shows…" isn't red: `statusMessage.classList.remove("error");`

COMMON MISTAKES

- Checking `response.ok` after `response.json()`: an error page may not be JSON, so it crashes first.
- A `catch` that does nothing: the page stays on "Loading…" forever.
- Forgetting to remove the `error` class, so the next loading message is red.

NOTE

Students can see a real error by switching off Wi-Fi and searching (the Lab then uses sample data, so they may need to search a word that isn't in it, like "zzz", to see the empty state instead). The check simulates errors itself.

A CORRECT ANSWER

```
// ... unchanged ...
async function searchShows(term) {
  const response = await fetch(searchUrl(term));
  if (!response.ok) {
    throw new Error("TVmaze answered " + response.status);   // stop here: we have no shows
  }
  const results = await response.json();
  return results.map((result) => toShow(result.show));
// ... unchanged ...
async function loadShows(term) {
  statusMessage.textContent = "Loading shows…";
  statusMessage.classList.remove("error");
  statusMessage.hidden = false;
  emptyMessage.hidden = true;   // an old "nothing found" doesn't belong to the new search
  try {
    state.shows = await searchShows(term);
    statusMessage.hidden = true;
    state.genre = "";   // the new shows may not have the chosen genre
    render();
  } catch (error) {
    // No internet, or TVmaze answered with an error: tell the person, in red
    statusMessage.textContent = "Sorry, the shows could not be loaded (" + error.message + "). Check
your internet connection and try again.";
    statusMessage.classList.add("error");
  }
}
```

Step 8. When nothing is found ~10 min

Teaches: Four states for loaded data: loading, error, empty, success.

TASK

In `loadShows`, inside `try`, before `render();`: set the text of `emptyMessage` to something like `No shows found for "zzz". Try another word.`, with the searched word (`term`) in it. Use a template literal.

Search for **zzz**, then for **dark**.

COMMON MISTAKES

- Quotes instead of backticks around a text with `${term}`.
- Setting the text after `render()` is fine too, but setting it outside `try` shows the old word after an error.

A CORRECT ANSWER

```
// ... unchanged ...
    statusMessage.hidden = true;
    state.genre = "";   // the new shows may not have the chosen genre
    emptyMessage.textContent = `No shows found for "${term}". Try another word.`;
    render();
  } catch (error) {
// ... unchanged ...
```

Step 9. Real shows from the start ~20 min

Teaches: `/shows?page=0` lists shows; `map(toShow)` converts them; `trim()` removes spaces.

TASK

1. **TODO 1** in `getShowsPage`: give back the shows in our shape: `return apiShows.map(toShow);`
2. In `loadShows`, if `term` is `""`, use `await getShowsPage()`; otherwise `await searchShows(term)` as before.
3. In the submit listener, send the trimmed text: `loadShows(searchInput.value.trim());`
4. In `state`, start with no shows: `shows: []`. The page doesn't need data.js any more.
5. Delete the old Start section (the comment and `render();`), and fill in **TODO 2** at the very bottom: `loadShows("");`

COMMON MISTAKES

- `apiShows.map((s) => toShow(s.show))`: page items have no `show` inside.
- Calling `loadShows("")` above the other code, so a `const` isn't ready yet ("before initialization").
- Keeping `shows: shows`, so the sample shows flash before the real ones.

NOTE

Online, page 0 has 250 shows, many without ratings or genres. That's why the Day 8 version of `createCard` checks for them.

A CORRECT ANSWER

```
// ... unchanged ...
// ---------- State: everything on the page that can change ----------
const state = {
  shows: [],      // the shows on the page, from TVmaze (in our shape)
  genre: "",      // the chosen genre ("" means all genres)
  myList: JSON.parse(localStorage.getItem("myList")) ?? [],   // saved ids, or an empty list
// ... unchanged ...
searchForm.addEventListener("submit", (event) => {
  event.preventDefault();   // don't reload the page
  loadShows(searchInput.value.trim());
});

// ... unchanged ...
  emptyMessage.hidden = true;   // an old "nothing found" doesn't belong to the new search
  try {
    if (term === "") {
      state.shows = await getShowsPage();   // empty search box: the first page of shows
    } else {
      state.shows = await searchShows(term);
    }
    statusMessage.hidden = true;
    state.genre = "";   // the new shows may not have the chosen genre
// ... unchanged ...
  }
}

// Page 0 of TVmaze's list of shows: the first 250 (the Lab's sample data has 12)
async function getShowsPage() {
  const response = await fetch(`${API}/shows?page=0`);
  if (!response.ok) {
    throw new Error("TVmaze answered " + response.status);
  }
  const apiShows = await response.json();
  return apiShows.map(toShow);
```

```
}

// ---------- Start: load the first shows from TVmaze ----------
loadShows("");
```

Step 10. Give credit to TVmaze ~10 min

Teaches: Credit your data source; create links with `createElement("a")` and `href`.

TASK

At the bottom of your code:

1. Create an `a` element with the `href` `https://www.tvmaze.com` and the text **Data from TVmaze**.
2. Add it to the paragraph in the footer, after a separator: `document.querySelector(".site-footer p").append(" · ", credit);`

Scroll down in the preview to see it.

COMMON MISTAKES

- Setting `credit.src` instead of `credit.href`.
- Appending to `.site-footer` instead of the paragraph inside it (works, but lands on its own line).

A CORRECT ANSWER

```
// ... unchanged ...
// ---------- Start: load the first shows from TVmaze ----------
loadShows("");

// ---------- Credit: TVmaze's data is free to use if we say where it comes from ----------
const credit = document.createElement("a");
credit.href = "https://www.tvmaze.com";
credit.textContent = "Data from TVmaze";
document.querySelector(".site-footer p").append(" · ", credit);
```

Step 11. My List with TVmaze ids ~15 min

Teaches: When saved data changes meaning, save it under a new key.

TASK

Use the key `"myListIds"` instead of `"myList"` in both places:

1. where `state.myList` is loaded, at the top
2. in `saveMyList`

Add a show, press **Run again**, and check that the counter remembers it.

COMMON MISTAKES

- Changing only one of the two keys: the list saves but never loads (or the other way round).
- A typo in one of the keys, like `"myListID"`.

A CORRECT ANSWER

```
// ... unchanged ...
  shows: [],      // the shows on the page, from TVmaze (in our shape)
  genre: "",      // the chosen genre ("" means all genres)
  myList: JSON.parse(localStorage.getItem("myListIds")) ?? [],   // saved TVmaze ids, or an empty
list
};

// ... unchanged ...
// Save My List in the browser (localStorage keeps text only, so we turn it into JSON text)
function saveMyList() {
  localStorage.setItem("myListIds", JSON.stringify(state.myList));
}

// ... unchanged ...
```

Extra 1. Search as you type, politely ~20 min

Teaches: Debounce: `clearTimeout` then `setTimeout` on every input.

TASK

At the bottom, add an `"input"` listener to `searchInput` that loads the shows for the trimmed text 400 milliseconds after the last letter, with `clearTimeout` and `setTimeout`. Keep the submit listener as it is.

A CORRECT ANSWER

```
// ... unchanged ...
credit.textContent = "Data from TVmaze";
document.querySelector(".site-footer p").append(" · ", credit);

// ---------- Search as you type, but only after a short pause ----------
let searchTimer;
searchInput.addEventListener("input", () => {
  clearTimeout(searchTimer);
  searchTimer = setTimeout(() => {
    loadShows(searchInput.value.trim());
  }, 400);
});
```

Extra 2. Surprise me ~15 min

Teaches: `Math.floor(Math.random() * list.length)` picks a random position.

TASK

1. At the top: `const surpriseButton = document.querySelector("#surprise");`
2. Add an `async` click listener to it that waits for `getShowsPage()`, picks one random show, puts it in `state.shows` **as a list of one** (`[show]`), sets `state.genre` to `""` and calls `render()`.

A CORRECT ANSWER

```
// ... unchanged ...
const listCount = document.querySelector("#list-count");
const statusMessage = document.querySelector("#status");   // Day 8: "Loading..." and errors
const surpriseButton = document.querySelector("#surprise");

// ---------- State: everything on the page that can change ----------
// ... unchanged ...
credit.textContent = "Data from TVmaze";
document.querySelector(".site-footer p").append(" · ", credit);

// ---------- Surprise me: one random show from page 0 ----------
surpriseButton.addEventListener("click", async () => {
  const all = await getShowsPage();
  const index = Math.floor(Math.random() * all.length);
  state.shows = [all[index]];   // still a list, with one show in it
  state.genre = "";
  render();
});
```

Extra 3. How many shows? ~10 min

Teaches: Counts and labels that depend on the state are drawn in `render()`.

TASK

1. At the top: `const resultCount = document.querySelector("#result-count");`
2. In `render()`, after the genre filter, set its text to the number of shows in `list`, followed by " shows".

Click a genre chip and watch the number change.

A CORRECT ANSWER

```
// ... unchanged ...
const listCount = document.querySelector("#list-count");
const statusMessage = document.querySelector("#status");   // Day 8: "Loading..." and errors
const resultCount = document.querySelector("#result-count");

// ---------- State: everything on the page that can change ----------
// ... unchanged ...
  }

  resultCount.textContent = `${list.length} shows`;
  renderGrid(list);
  renderGenres();
// ... unchanged ...
```

Day 9: The show page

Views, details and My List

Today ShowShelf gets a page for every show, with its cast and seasons, and a page for My List.

Instead of separate HTML files, the page now has three **views**: three `<section>` elements, of which only one shows at a time. Open the **index.html (whole page)** tab to see them: `#home-view`, `#show-view` and `#mylist-view`. Switching views is one more thing in the state.

Your code box starts with the finished app from Day 8, tidied into sections. The element lookups for the new parts of the page are already at the top (look for **Day 9**), so you can focus on what they do. Today's new code goes at the very bottom, under the line that says *new on Day 9*.

This is your finished app. On Day 11 it becomes your own website, online. Take your time, and read the code you add as well as writing it.

Step 1. Three views, one page ~20 min

Teaches: Views in one page: `state.view` plus `hidden` in `render()`. `event.stopPropagation()` stops bubbling.

TASK

1. In `state`, add `view: "home",`.
2. At the top of `render()`, add three lines that hide every view except the one in `state.view` (`homeView`, `showView` and `myListView`).
3. Fill in the TODO in `openShow`.
4. In `createCard`, before `return card;`, add a click listener to the card that calls `openShow(show.id)`.
5. In `createListButton`, give the listener an `event` parameter and call `event.stopPropagation();` first.

Click a card: the home view disappears. (The show view is empty for now.) Press **Run again** to go back.

COMMON MISTAKES

- `homeView.hidden = state.view === "home";`: the wrong way round, so the home view hides exactly when it should show.
- `card.addEventListener("click", openShow(show.id))`: this calls `openShow` straight away for every card.
- Forgetting `event` in `(event) => { ... }`, so `event.stopPropagation()` fails.

A CORRECT ANSWER

```
// ... unchanged ...
// ---------- State: everything on the page that can change ----------
const state = {
  view: "home",   // which view is showing: "home", "show" or "mylist"
  shows: [],      // the shows on the home page, from TVmaze (in our shape)
  genre: "",      // the chosen genre ("" means all genres)
// ... unchanged ...
  card.append(poster, body);
  card.append(createListButton(show));
  card.addEventListener("click", () => {
    openShow(show.id);
  });
  return card;
}
// ... unchanged ...
  button.setAttribute("aria-pressed", isInMyList(show.id));   // "pressed" look, and for screen
readers
  button.setAttribute("aria-label", "My List: " + show.name);
  button.addEventListener("click", (event) => {
    event.stopPropagation();   // a click on the button is not a click on the card
    toggleMyList(show);
  });
// ... unchanged ...

function render() {
  // Show the view that matches state.view, and hide the other two
  homeView.hidden = state.view !== "home";
  showView.hidden = state.view !== "show";
  myListView.hidden = state.view !== "mylist";

  let list = state.shows;
  if (state.genre !== "") {
// ... unchanged ...
```

```
// ========== The show view and My List (new on Day 9) ==========

// ---------- The show view ----------

// Open the show view for one show (a click on a card calls this)
function openShow(id) {
  state.view = "show";
  render();
  window.scrollTo(0, 0);   // start at the top of the page
}
```

Step 2. One show, with cast and seasons ~15 min

Teaches: `/shows/ID?embed[]=cast&embed[]=seasons` gets a show with its cast and seasons; a function can return an object.

TASK

1. Fill in the TODO in `getShowById`: return an object with three properties, `show`, `cast` and `seasons`.
2. Make `openShow` an `async function`, and at its end add `const data = await getShowById(id);` and `console.log(data.show.name);`.

Click a card and look at the Console.

COMMON MISTAKES

- `data.embedded.cast`: the property starts with an underscore, `_embedded`.
- Forgetting `async` on `openShow`, so `await` gives an error.
- `toShow(data._embedded)` instead of `toShow(data)`.

A CORRECT ANSWER

```
// ... unchanged ...

// Open the show view for one show (a click on a card calls this)
async function openShow(id) {
  state.view = "show";
  render();
  window.scrollTo(0, 0);   // start at the top of the page
  const data = await getShowById(id);
  console.log(data.show.name);
}

// Ask TVmaze for one show, with its cast and seasons in the same answer
// getShowById(431) -> { show: {...our shape...}, cast: [...], seasons: [...] }
async function getShowById(id) {
  const response = await fetch(`${API}/shows/${id}?embed[]=cast&embed[]=seasons`);
  if (!response.ok) {
    throw new Error("TVmaze answered " + response.status);
  }
  const data = await response.json();
  return {
    show: toShow(data),
    cast: data._embedded.cast,
    seasons: data._embedded.seasons,
  };
}
```

Step 3. Fill in the show ~20 min

Teaches: Fill a view from an object with `textContent`; never use `innerHTML` for text from outside.

TASK

1. Fill in the four TODOs in `renderShow`.
2. In `openShow`, replace `console.log(data.show.name);` with `renderShow(data.show);`.

Click a few cards: each opens with its own name, years, summary and genres. Press **Run again** to go back (the back button comes later).

COMMON MISTAKES

- `showSummary.innerHTML = show.summary`: works, but opens the door to code hidden in the text.
- Writing TODO 2 below the network lines, so `=` wipes the network out again.
- Appending the badges without `replaceChildren()` first (it's given), so they pile up from show to show.

A CORRECT ANSWER

```
// ... unchanged ...
  window.scrollTo(0, 0);   // start at the top of the page
  const data = await getShowById(id);
  renderShow(data.show);
}

// ... unchanged ...
  };
}

// Fill the show view with one show (in our shape)
function renderShow(show) {
  showPoster.src = show.image ?? "poster.svg";
  showPoster.alt = "Poster for " + show.name;
  showName.textContent = show.name;

  showMeta.textContent = yearsLabel(show.premiered, show.ended);
  if (show.network !== "") {
    showMeta.textContent += " · " + show.network;   // += adds to the end of the text
  }

  if (show.rating === null) {
    showRating.textContent = "No rating yet";
  } else {
    showRating.textContent = "★ " + show.rating + " · " + ratingLabel(show.rating);
  }

  // textContent, never innerHTML: the summary is text from the internet
  showSummary.textContent = show.summary;
  if (show.summary === "") {
    showSummary.textContent = "No summary yet.";
  }

  // One badge per genre
  showGenres.replaceChildren();
  for (const genre of show.genres) {
    const badge = document.createElement("span");
    badge.className = "badge";
    badge.textContent = genre;
    showGenres.append(badge);
    showGenres.append(" ");   // a space between the badges
  }
}
```

Step 4. The cast ~15 min

Teaches: Reach nested data step by step (`member.person.name`) and build a list with a loop.

TASK

1. Fill in the TODO in `renderCast`, so each cast member becomes an `<li>` like "Bryan Cranston as Walter White".
2. In `openShow`, under `renderShow(data.show);`, add `renderCast(data.cast);`.

COMMON MISTAKES

- `member.name`: the name is inside `person`.
- Creating the `li` outside the loop, so only the last person shows.

A CORRECT ANSWER

```
// ... unchanged ...
  const data = await getShowById(id);
  renderShow(data.show);
  renderCast(data.cast);
}

// ... unchanged ...
  }
}

// The cast as a list: "Jennifer Aniston as Rachel Green"
function renderCast(cast) {
  castList.replaceChildren();
  for (const member of cast) {
    const item = document.createElement("li");
    item.textContent = member.person.name + " as " + member.character.name;
    castList.append(item);
  }
  if (cast.length === 0) {
    const empty = document.createElement("li");
    empty.textContent = "No cast information yet.";
    castList.append(empty);
  }
}
```

Step 5. The seasons table ~15 min

Teaches: A small helper function (`createCell`) removes repetition.

TASK

1. Fill in the TODO in `renderSeasons`: add three cells to the row, for the season number, the number of episodes (`episodeOrder`) and the year.
2. In `openShow`, under `renderCast(data.cast);`, add `renderSeasons(data.seasons);`.

COMMON MISTAKES

- `createCell(season.premiereDate)`: the whole date instead of the year.
- Appending the row to `castList` or to the `table` instead of `seasonsBody`.

A CORRECT ANSWER

```
// ... unchanged ...
  renderShow(data.show);
  renderCast(data.cast);
  renderSeasons(data.seasons);
}

// ... unchanged ...
  }
}

// One table cell (<td>) with some text
function createCell(text) {
  const cell = document.createElement("td");
  cell.textContent = text;
  return cell;
}

// The seasons as table rows: season number, episodes, year
function renderSeasons(seasons) {
  seasonsBody.replaceChildren();
  for (const season of seasons) {
    const row = document.createElement("tr");
    row.append(
      createCell(season.number),
      createCell(season.episodeOrder),
      createCell(yearOf(season.premiereDate))
    );
    seasonsBody.append(row);
  }
}
```

Step 6. The back button ~10 min

Teaches: A back button changes `state.view` and calls `render()`.

TASK

At the bottom, add a click listener to `backButton` that sets `state.view` to `"home"` and calls `render()`.

Open a show, then go back: the same shows are still there, with the same genre chosen.

COMMON MISTAKES

- Calling `loadShows("")` in the back button: it works, but asks TVmaze again for nothing.

A CORRECT ANSWER

```
// ... unchanged ...
  }
}

// ---------- Switching views ----------

// The back button in the show view goes home
backButton.addEventListener("click", () => {
  state.view = "home";
  render();
});
```

Step 7. Loading and errors ~15 min

Teaches: Loading and error states for the show view: a message before `await`, `try / catch` around it.

TASK

In `openShow`, after `window.scrollTo(0, 0);`:

1. Hide the details, `showDetails.hidden = true;`, and show `showStatus` with the text "Loading show...". Also remove its `error` class.
2. Put the `await` line and the three `render...` lines inside `try { }`. At the end of `try`, hide the status and show the details again.
3. In `catch (error) { }`, give `showStatus` a friendly error message and the class `error`.

COMMON MISTAKES

- Showing the details before the `await`, so the old show flashes.
- Leaving `renderShow` outside `try`, so an error still breaks the page.

A CORRECT ANSWER

```
// ... unchanged ...
  render();
  window.scrollTo(0, 0);   // start at the top of the page

  // While loading: hide the old show and say what's happening
  showDetails.hidden = true;
  showStatus.textContent = "Loading show…";
  showStatus.classList.remove("error");
  showStatus.hidden = false;

  try {
    const data = await getShowById(id);
    renderShow(data.show);
    renderCast(data.cast);
    renderSeasons(data.seasons);
    showStatus.hidden = true;
    showDetails.hidden = false;
  } catch (error) {
    // No internet, or TVmaze answered with an error
    showStatus.textContent = "Sorry, this show could not be loaded (" + error.message + "). Check
your internet connection and try again.";
    showStatus.classList.add("error");
  }
}

// ... unchanged ...
```

Step 8. Show not found ~10 min

Teaches: Treat 404 as "not found" (`return null`), and use an early `return` for special cases.

TASK

1. In `getShowById`, before the `response.ok` check: if `response.status` is `404`, `return null;`.
2. In `openShow`, straight after the `await` line: if `data` is `null`, set `showStatus` to "Show not found." and `return`.

COMMON MISTAKES

- Putting the 404 check after `if (!response.ok)`: the function throws first and never gets there.
- Forgetting `return;` after the message, so `renderShow(null.show)` crashes.

A CORRECT ANSWER

```
// ... unchanged ...
  try {
    const data = await getShowById(id);
    if (data === null) {
      showStatus.textContent = "Show not found. Go back and pick another one.";
      return;
    }
    renderShow(data.show);
    renderCast(data.cast);
// ... unchanged ...

// Ask TVmaze for one show, with its cast and seasons in the same answer
// getShowById(431) -> { show: {...our shape...}, cast: [...], seasons: [...] }, or null if there's
no such show
async function getShowById(id) {
  const response = await fetch(`${API}/shows/${id}?embed[]=cast&embed[]=seasons`);
  if (response.status === 404) {
    return null;   // TVmaze has no show with this id
  }
  if (!response.ok) {
    throw new Error("TVmaze answered " + response.status);
// ... unchanged ...
```

Step 9. My List remembers more ~20 min

Teaches: Store small objects in localStorage; `some` checks whether any item matches.

TASK

1. In `state`, load the list with the new function: `myList: loadMyList(),`
2. In `saveMyList`, use the key `"myList"` again.
3. In `isInMyList`: `return state.myList.some((item) => item.id === id);`
4. In `toggleMyList`, remove with `state.myList.filter((item) => item.id !== show.id)`, and push an object with the five properties `id`, `name`, `image`, `genres` and `rating`, taken from `show`.

COMMON MISTAKES

- Changing `push` but not `filter` (or `isInMyList`), so taking a show out stops working.
- `includes(id)` on a list of objects: always `false`, because it compares whole objects with a number.
- Forgetting a comma between the properties of the object.

A CORRECT ANSWER

```
// ... unchanged ...
  shows: [],      // the shows on the home page, from TVmaze (in our shape)
  genre: "",      // the chosen genre ("" means all genres)
  myList: loadMyList(),   // small objects for the shows in My List (see loadMyList below)
};

// ... unchanged ...
// Is this show in My List? Gives true or false.
function isInMyList(id) {
  return state.myList.some((item) => item.id === id);
}

// ... unchanged ...
function toggleMyList(show) {
  if (isInMyList(show.id)) {
    state.myList = state.myList.filter((item) => item.id !== show.id);
  } else {
    // Save what a card needs, so My List can be drawn without asking TVmaze
    state.myList.push({
      id: show.id,
      name: show.name,
      image: show.image,
      genres: show.genres,
      rating: show.rating,
    });
  }
  saveMyList();
// ... unchanged ...
// Save My List in the browser (localStorage keeps text only, so we turn it into JSON text)
function saveMyList() {
  localStorage.setItem("myList", JSON.stringify(state.myList));
}

// ... unchanged ...
  render();
});

// ---------- My List, saved as small objects ----------
// Each show in My List is saved with what a card needs, so the My List view
// can draw it without asking TVmaze:
//   { id: 431, name: "Friends", image: null, genres: ["Comedy", "Romance"], rating: 8.8 }

// Read My List from localStorage (the key is "myList")
```

```
function loadMyList() {
  const saved = JSON.parse(localStorage.getItem("myList")) ?? [];
  // On Day 7 this key held plain ids, like [2, 5]. Keep only the objects.
  return saved.filter((item) => typeof item === "object");
}
```

Step 10. The My List view ~20 min

Teaches: Reuse `createCard` for a second view; `preventDefault()` on links you handle yourself.

TASK

1. **TODO 1** in `renderMyList`: a `for...of` loop over `state.myList` that appends `createCard(show)` to `myListGrid`.
2. At the end of `render()`, call `renderMyList();`, so My List is drawn every time the state changes.
3. **TODO 2**: click listeners for `homeLink` (to the `"home"` view) and `myListLink` (to the `"mylist"` view), like the logo's.

Add a few shows, open **My List** in the header, and take one out again.

COMMON MISTAKES

- Forgetting `renderMyList()` in `render()`: the view only updates on a reload.
- Leaving out `event.preventDefault()`: in the Lab a notice appears; on the real website the page would reload.
- Looping over `state.shows` instead of `state.myList`.

A CORRECT ANSWER

```
// ... unchanged ...
  renderGenres();
  listCount.textContent = state.myList.length;
  renderMyList();
}

// ... unchanged ...
  return saved.filter((item) => typeof item === "object");
}

// ---------- The My List view ----------

// Draw the saved shows, with the same createCard as the home page
function renderMyList() {
  myListGrid.replaceChildren();
  for (const show of state.myList) {
    myListGrid.append(createCard(show));
  }
  myListEmpty.hidden = state.myList.length > 0;
}

// The logo goes home
logoLink.addEventListener("click", (event) => {
  event.preventDefault();   // don't follow the link: we switch views ourselves
  state.view = "home";
  render();
});

// The Home link goes home too
homeLink.addEventListener("click", (event) => {
  event.preventDefault();
  state.view = "home";
  render();
});

// The My List link opens the My List view
myListLink.addEventListener("click", (event) => {
  event.preventDefault();
  state.view = "mylist";
```

```
  render();
});
```

Step 11. My List from the show view ~15 min

Teaches: `state.currentShow` remembers the open show; one `render()` keeps every view in agreement.

TASK

1. In `state`, add `currentShow: null,`.
2. In `openShow`, right before `renderShow(data.show);`, remember the show: `state.currentShow = data.show;`. Right after `renderSeasons(...)`, call `renderShowListButton();`.
3. At the end of `render()`, call `renderShowListButton();`.
4. At the bottom, add a click listener to `showListButton` that calls `toggleMyList(state.currentShow);`.

Open a show, add it, then look at its card on the home page and in My List: all three agree.

COMMON MISTAKES

- Setting the button's text inside the click listener instead of in `render()`: it goes wrong when the show is added from its card.
- Adding the listener inside `openShow`: every opened show adds one more listener, and one click toggles several times.

NOTE

This is the last core step of the app students take home on Day 11. If time allows, ask them to try it online (the preview asks the real TVmaze when there's internet): real posters, and some shows with "No rating yet".

A CORRECT ANSWER

```
// ... unchanged ...
  genre: "",      // the chosen genre ("" means all genres)
  myList: loadMyList(),   // small objects for the shows in My List (see loadMyList below)
  currentShow: null,      // the show open in the show view (null when none is)
};

// ... unchanged ...
  listCount.textContent = state.myList.length;
  renderMyList();
  renderShowListButton();
}

// ... unchanged ...
      return;
    }
    state.currentShow = data.show;
    renderShow(data.show);
    renderCast(data.cast);
    renderSeasons(data.seasons);
    renderShowListButton();
    showStatus.hidden = true;
    showDetails.hidden = false;
// ... unchanged ...
  render();
});

// The My List button in the show view. Its text follows the state.
function renderShowListButton() {
  if (!state.currentShow) {
    return;   // no show is open yet
  }
  if (isInMyList(state.currentShow.id)) {
    showListButton.textContent = "✓ In My List";
```

```
  } else {
    showListButton.textContent = "+ Add to My List";
  }
  showListButton.setAttribute("aria-pressed", isInMyList(state.currentShow.id));
}

// Add the open show to My List, or take it out
showListButton.addEventListener("click", () => {
  toggleMyList(state.currentShow);
});
```

Extra 1. A link for every show ~20 min

Teaches: `location.hash` and the `"hashchange"` event give each view its own address.

TASK

1. **TODO**: call `showFromAddress` on every `"hashchange"` of `window`.
2. In `createCard`'s click listener, replace `openShow(show.id);` with `location.hash = "show/" + show.id;`.
3. In the back button's listener, replace its two lines with `location.hash = "";`.
4. In the logo, Home and My List listeners, add `location.hash = "";` after `event.preventDefault();`, so the address forgets the show.

NOTE

Inside the Lab, the preview's own address changes (you can't see it). On the real website from Day 11, the address bar shows `#show/431` and the browser's back button works.

A CORRECT ANSWER

```
// ... unchanged ...
  card.append(createListButton(show));
  card.addEventListener("click", () => {
    location.hash = "show/" + show.id;   // the hashchange listener opens the show
  });
  return card;
// ... unchanged ...
// The back button in the show view goes home
backButton.addEventListener("click", () => {
  location.hash = "";   // the hashchange listener goes home
});

// ... unchanged ...
logoLink.addEventListener("click", (event) => {
  event.preventDefault();   // don't follow the link: we switch views ourselves
  location.hash = "";
  state.view = "home";
  render();
// ... unchanged ...
homeLink.addEventListener("click", (event) => {
  event.preventDefault();
  location.hash = "";
  state.view = "home";
  render();
// ... unchanged ...
myListLink.addEventListener("click", (event) => {
  event.preventDefault();
  location.hash = "";
  state.view = "mylist";
  render();
// ... unchanged ...
  toggleMyList(state.currentShow);
});

// ---------- Extra: an address for every show, like index.html#show/431 ----------

// Show the view that the address asks for
function showFromAddress() {
  if (location.hash.startsWith("#show/")) {
    const id = Number(location.hash.slice(6));   // "#show/431" -> 431
    openShow(id);
  } else if (state.view === "show") {
```

```
    state.view = "home";                          // the address has no show any more
    render();
  }
}

window.addEventListener("hashchange", showFromAddress);

// Someone opened a link to a show, like index.html#show/431: open it straight away
if (location.hash.startsWith("#show/")) {
  showFromAddress();
}
```

Extra 2. Cast photos ~15 min

Teaches: `append` mixes elements and text; check for `null` before using an optional object.

TASK

In `renderCast`'s loop, when `member.person.image` is not `null`, create an `<img>` with `src` set to `member.person.image.medium`, an empty `alt`, and `width` 60, and append it to the `li` **before** the text. Then append the text with `append` instead of `textContent`.

A CORRECT ANSWER

```
// ... unchanged ...
  for (const member of cast) {
    const item = document.createElement("li");
    if (member.person.image !== null) {
      const photo = document.createElement("img");
      photo.src = member.person.image.medium;
      photo.alt = "";
      photo.width = 60;
      item.append(photo);
    }
    item.append(member.person.name + " as " + member.character.name);
    castList.append(item);
  }
// ... unchanged ...
```

Extra 3. The average rating of My List ~15 min

Teaches: Average: filter out `null`, add up in a loop, divide, `toFixed(1)`.

TASK

1. At the top: `const myListAverage = document.querySelector("#mylist-average");`
2. At the end of `renderMyList`: keep the shows whose rating isn't `null`. If there are none, empty the line. Otherwise add up their ratings and show "Average rating: " with the average, to one decimal.

A CORRECT ANSWER

```
// ... unchanged ...
const myListGrid = document.querySelector("#mylist-grid");
const myListEmpty = document.querySelector("#mylist-empty");
const myListAverage = document.querySelector("#mylist-average");

// ---------- State: everything on the page that can change ----------
// ... unchanged ...
  }
  myListEmpty.hidden = state.myList.length > 0;

  // The average rating of the shows that have one
  const rated = state.myList.filter((show) => show.rating !== null);
  if (rated.length === 0) {
    myListAverage.textContent = "";
  } else {
    let total = 0;
    for (const show of rated) {
      total = total + show.rating;
    }
    myListAverage.textContent = "Average rating: " + (total / rated.length).toFixed(1);
  }
}

// ... unchanged ...
```

Day 10: How real apps are built A taste of React and of servers

This morning the teacher showed you a real **React** project and a real **Express** server running on his laptop. This afternoon you try both ideas yourself, in small steps.

In a real project this code lives in files on your computer (React) or on a server (Express), and you'd install tools first: Node.js, and then React or Express with `npm`. The Lab lets you try the ideas without that.

- **Part 1, steps 1 to 6: React.** You rebuild the ShowShelf grid out of **components**: a card, a list of cards, a My List button and a search box.
- **Part 2, steps 7 to 12: a server.** You write the **routes** of a small My List server with Express. A ready-made page talks to it with `fetch`, so you see your server working straight away.

Today is a taste. The goal isn't to master React or Express in one afternoon, but to recognise the ideas when you meet them again. Everything you already know (functions, objects, arrays, `filter`, `fetch`) is used in both.

Step 1. Your first React component ~15 min

Teaches: A React component is a function that returns JSX; `createRoot(...).render(<App />)` draws it.

TASK

1. Write a component called `App` that returns a main heading: `<h1>ShowShelf</h1>`.
2. Under it, add the line that draws `<App />` inside `#root`.

COMMON MISTAKES

- A small letter: `function app()` and `<app />`. React treats lowercase tags as HTML elements.
- Forgetting the render line, so nothing appears.
- Quotes around the JSX: `return "<h1>ShowShelf</h1>";` shows the tags as text.

NOTE

If the preview stays empty, open the Console first: JSX mistakes come with a line number. The Lab translates the JSX into plain JavaScript before running it, so `import` lines don't work here (and aren't needed).

A CORRECT ANSWER

```
// App.jsx: your React code.
// React, ReactDOM and the list of shows (const shows = [...]) are ready to use.

function App() {
  return <h1>ShowShelf</h1>;
}

ReactDOM.createRoot(document.getElementById("root")).render(<App />);
```

Step 2. JavaScript values inside JSX ~10 min

Teaches: `{ }` in JSX shows the result of JavaScript; a component returns one outer element.

TASK

Make `App` return this structure, with the classes of the ShowShelf stylesheet:

1. an outer `<main className="container">`
2. inside it, a `<section className="hero">` with your `<h1>ShowShelf</h1>`
3. under the heading, a paragraph that says **12 shows to choose from**, where the number comes from `{shows.length}`

COMMON MISTAKES

- Returning two elements side by side without one element around them.
- Forgetting the `( )` after `return` on a multi-line JSX, so `return` returns nothing.
- `class=` instead of `className=` (React warns in the Console).

A CORRECT ANSWER

```
// ... unchanged ...

function App() {
  return (
    <main className="container">
      <section className="hero">
        <h1>ShowShelf</h1>
        <p>{shows.length} shows to choose from</p>
      </section>
    </main>
  );
}

// ... unchanged ...
```

Step 3. A ShowCard component with props ~20 min

Teaches: Props are a component's inputs: `<ShowCard show={...} />`, received as `function ShowCard({ show })`.

TASK

1. In `ShowCard`, replace the two `TODO` comments with an `<h3>` that shows `show.name`, and a `<p className="rating">` that shows a star and `show.rating`, like ★ **8.8**. (`{/* ... */}` is how you write a comment inside JSX.)
2. In `App`, under the hero section (still inside `<main>`), add `<div className="grid">` with one card inside: `<ShowCard show={shows[0]} />`.

COMMON MISTAKES

- `show="shows[0]"` in quotes: that passes the text "shows[0]", not the object.
- `<h3>show.name</h3>` without curly brackets: the words appear as they are.
- `{name}` instead of `{show.name}`: the prop is the whole show object.

A CORRECT ANSWER

```
// ... unchanged ...
        <p>{shows.length} shows to choose from</p>
      </section>

      <div className="grid">
        <ShowCard show={shows[0]} />
      </div>
    </main>
  );
// ... unchanged ...

ReactDOM.createRoot(document.getElementById("root")).render(<App />);

// One card for one show. Its prop (input) is `show`: one object from the list.
function ShowCard({ show }) {
  return (
    <article className="card">
      <img className="poster" src="poster.svg" alt="" />
      <div className="card-body">
        <span className="badge">{show.genres[0]}</span>
        <h3>{show.name}</h3>
        <p className="rating">★ {show.rating}</p>
      </div>
    </article>
  );
}
```

Step 4. A card for every show ~10 min

Teaches: `array.map((item) => <Comp key={item.id} ... />)` draws a list; each item needs a unique `key`.

TASK

In `App`, replace the single `<ShowCard show={shows[0]} />` with a card for **every** show: use `shows.map(...)`, and give each card `key={show.id}`.

COMMON MISTAKES

- `shows.map((show) => { <ShowCard show={show} /> })`: curly brackets without `return`, so no cards.
- `show={shows[0]}` inside the map: twelve cards of the same show.
- `key={show.name}` works here, but an id is safer: names can repeat.

A CORRECT ANSWER

```
// ... unchanged ...

      <div className="grid">
        {shows.map((show) => <ShowCard key={show.id} show={show} />)}
      </div>
    </main>
// ... unchanged ...
```

Step 5. Remember something with state ~20 min

Teaches: `const [value, setValue] = React.useState(start)` gives a component memory; calling the setter redraws it.

TASK

Give every card a My List button that switches between + and ✓:

1. At the top of `ShowCard` (before `return`), create state: `const [inList, setInList] = React.useState(false);`
2. Under the `<img>`, add a `<button>` with `className="list-btn"` and `aria-pressed={inList}`. When it's clicked, it calls `setInList(!inList)`.
3. The button's text is `{inList ? "✓" : "+"}`. (A normal `+` is fine too.)

`!inList` means "the opposite of inList": `true` becomes `false`, and `false` becomes `true`. Click a few buttons in the preview.

COMMON MISTAKES

- `onClick={setInList(!inList)}` without `() =>`: it runs while drawing, again and again ("Too many re-renders" in the Console).
- The `useState` line in `App` instead of `ShowCard`: all cards switch together.
- `onclick` with a small c: React ignores it and warns in the Console.

NOTE

Compare with the plain JavaScript version: there, the click handler had to find the button and change its text. Here the handler only changes state, and the JSX already says what each state looks like.

A CORRECT ANSWER

```
// ... unchanged ...
// One card for one show. Its prop (input) is `show`: one object from the list.
function ShowCard({ show }) {
  const [inList, setInList] = React.useState(false);

  return (
    <article className="card">
      <img className="poster" src="poster.svg" alt="" />
      <button className="list-btn" aria-pressed={inList} onClick={() => setInList(!inList)}>
        {inList ? "✓" : "+"}
      </button>
      <div className="card-body">
        <span className="badge">{show.genres[0]}</span>
// ... unchanged ...
```

Step 6. A search box that filters ~15 min

Teaches: A controlled input: `value={state}` and an `onChange` that sets the state; filter the data and map the result.

TASK

In `App`:

1. Create state for the search text: `const [search, setSearch] = React.useState("");`
2. In the hero, under the paragraph, add `<div className="search">` with a controlled `<input />`: `value={search}`, an `onChange` that calls `setSearch`, and a `placeholder` like "Search shows".
3. Before `return`, make a list of the shows whose name contains the search text (with `filter`, `toLowerCase` and `includes`), and `map` that list instead of `shows`.

Type `the` in the preview's search box.

COMMON MISTAKES

- `value={search}` without `onChange`: the box can't be typed in.
- Still mapping `shows` instead of the filtered list, so nothing changes.
- Using `onChange={setSearch}`: that saves the whole event, not the text. Use `(e) => setSearch(e.target.value)`.

NOTE

If a student asks why the My List ticks disappear when a card is filtered out and comes back: the card's state goes away with the card. In the full app, the list lives in `App` and is passed down as a prop. That's a good question to end the React part on.

A CORRECT ANSWER

```
// ... unchanged ...

function App() {
  const [search, setSearch] = React.useState("");
  const found = shows.filter((show) => show.name.toLowerCase().includes(search.toLowerCase()));

  return (
    <main className="container">
// ... unchanged ...
        <h1>ShowShelf</h1>
        <p>{shows.length} shows to choose from</p>
        <div className="search">
          <input
            type="search"
            placeholder="Search shows"
            value={search}
            onChange={(e) => setSearch(e.target.value)}
          />
        </div>
      </section>

      <div className="grid">
        {found.map((show) => <ShowCard key={show.id} show={show} />)}
      </div>
    </main>
// ... unchanged ...
```

Step 7. Your first server route ~15 min

Teaches: A route: `app.get("/path", (req, res) => { res.json(...); })`.

TASK

Under `// Your routes go here`, add a route for `GET /api/hello` that answers with an object like `{ message: "Hello from the server!" }`.

When it works, the top of the page shows what your server said. Then try the **Try a request** box at the bottom of the preview: press **Send**.

COMMON MISTAKES

- `res.send("Hello")` instead of `res.json({ message: ... })`: the answer is plain text, not an object.
- An address without the first `/`, or a typo like `/api/helo`.
- Putting the route inside the `{ }` of `app.listen(...)`. It works, but routes belong above it.

NOTE

The pretend server is only a stand-in: it understands the same code as Express for today's features. If students ask where Node.js is: it's what the teacher used this morning in the terminal (`npm run dev`).

A CORRECT ANSWER

```
// ... unchanged ...
// Your routes go here

app.get("/api/hello", (req, res) => {
  res.json({ message: "Hello from the server!" });
});

app.listen(3000, () => {
// ... unchanged ...
```

Step 8. Send My List ~10 min

Teaches: `res.json(array)` sends a list; for now the data lives in the server's memory.

TASK

1. Above `app.listen`, create `let myList` with at least two shows. Each show is an object with a number `id` and a `name`, like `{ id: 1, name: "Friends" }`.
2. Add a route for `GET /api/mylist` that answers with `myList`.

The page's list fills up with your shows.

COMMON MISTAKES

- Quotes around the ids: `id: "1"` is text, not a number.
- A missing comma between the objects in the array.
- `res.json("myList")` in quotes: that sends the word, not the list.

A CORRECT ANSWER

```
// ... unchanged ...
});

// My List lives in the server's memory
let myList = [
  { id: 1, name: "Friends" },
  { id: 2, name: "Dark" },
];

app.get("/api/mylist", (req, res) => {
  res.json(myList);
});

app.listen(3000, () => {
  console.log("ShowShelf API is ready");
// ... unchanged ...
```

Step 9. Add a show with POST ~15 min

Teaches: `app.post` and `req.body` receive data; `res.status(201).json(item)` answers "Created".

TASK

Above `app.listen`, add a route for `POST /api/mylist` that:

1. makes a new show object with `id: Date.now()` and `name: req.body.name`
2. adds it to `myList` with `push`
3. answers with status `201` and the new show

Then add a show with the form in the preview.

COMMON MISTAKES

- `res.json(item).status(201)`: the answer has already gone with 200 before the status is set.
- `name: req.body` (the whole body) instead of `req.body.name`.
- Forgetting `myList.push(item)`: the page says "Added", but the show never appears.

A CORRECT ANSWER

```
// ... unchanged ...
});

app.post("/api/mylist", (req, res) => {
  const item = { id: Date.now(), name: req.body.name };
  myList.push(item);
  res.status(201).json(item);
});

app.listen(3000, () => {
  console.log("ShowShelf API is ready");
// ... unchanged ...
```

Step 10. Say no to bad data ~10 min

Teaches: Validation: refuse bad data with `return res.status(400).json({ error: "..." })`.

TASK

At the top of your POST route, refuse a request without a name: answer `400` with `{ error: "..." }` and a message of your own. Don't forget the `return`.

Then press **Add** with an empty box: the page shows your message.

COMMON MISTAKES

- No `return`: the show is still added, and the Console warns that the server tried to answer twice.
- `if (req.body.name)` without the `!`: now the good shows are refused.
- The `if` placed after `myList.push(item)`: too late.

A CORRECT ANSWER

```
// ... unchanged ...

app.post("/api/mylist", (req, res) => {
  if (!req.body.name) {
    return res.status(400).json({ error: "Please send the name of a show" });
  }
  const item = { id: Date.now(), name: req.body.name };
  myList.push(item);
// ... unchanged ...
```

Step 11. Remove a show with DELETE ~20 min

Teaches: `app.delete("/api/mylist/:id", ...)`: `Number(req.params.id)`, 404 when it's missing, 204 when it's done.

TASK

Above `app.listen`, add a route for `DELETE /api/mylist/:id` that:

1. turns `req.params.id` into a number
2. answers `404` with an `error` if no show in `myList` has that id (use `find`)
3. otherwise removes the show with `filter` and answers `204`

Then press **Remove** on a show in the preview.

COMMON MISTAKES

- Forgetting `Number(...)`: `"2" === 2` is false, so every show is "not found".
- `myList.filter(...)` without `myList =` in front: the filter makes a new list, but nobody keeps it.
- `const myList`: then `myList = ...` crashes with "Assignment to constant variable".

A CORRECT ANSWER

```
// ... unchanged ...
});

app.delete("/api/mylist/:id", (req, res) => {
  const id = Number(req.params.id);
  const found = myList.find((show) => show.id === id);
  if (!found) {
    return res.status(404).json({ error: "That show is not in My List" });
  }
  myList = myList.filter((show) => show.id !== id);
  res.status(204).end();
});

app.listen(3000, () => {
  console.log("ShowShelf API is ready");
// ... unchanged ...
```

Step 12. Read the request log ~10 min

Teaches: Status codes: 2xx success, 4xx a wrong request, 5xx a server problem; a server's `console.log` goes to its log.

TASK

1. In the preview, add a show, add an empty name, remove a show, and send `GET /api/nothing` with **Try a request**. Watch which codes appear in the Console.
2. In your POST route, right before the answer, add a `console.log` that prints the name of the new show, like `console.log("Added:", item.name);`. Add a show again, and find your message in the Console.

COMMON MISTAKES

- Putting the `console.log` in the GET route instead of POST.
- `console.log("item.name")` in quotes: it prints the words, not the show's name.

NOTE

A good discussion to end the day: which part of the ShowShelf you know is the page, and which part could be a server? (The page draws; TVmaze and My List are data a server could keep.) If there's time, let students break a route on purpose (`throw new Error("Oops")`) to see a 500.

A CORRECT ANSWER

```
// ... unchanged ...
  const item = { id: Date.now(), name: req.body.name };
  myList.push(item);
  console.log("Added:", item.name);
  res.status(201).json(item);
});
// ... unchanged ...
```

Extra 1. Shows from TVmaze with useEffect ~20 min

Teaches: `React.useEffect(() => { ... }, [])` runs once after the first draw: a good place for `fetch`.

TASK

In `App`:

1. Create state `tvShows` that starts as an empty array: `React.useState([])`.
2. Add a `React.useEffect` that fetches `https://api.tvmaze.com/shows?page=0`, turns every show into our shape with `data.map(toShow)`, and stores the result with `setTvShows`.
3. Filter `tvShows` instead of `shows`.

Without internet, the Lab answers with 12 saved sample shows, so the cards look the same as before, but now they come from `fetch`.

COMMON MISTAKES

- No `[]` at the end of `useEffect`: the page fetches again after every draw.
- `setTvShows(data)` without `toShow`: React can't show `rating` because it's an object.

NOTE

Online, TVmaze answers with about 240 shows, so the grid gets long. That's expected. The check replaces TVmaze's answer with three known shows for a moment, to see that the cards really follow the data.

A CORRECT ANSWER

```
// ... unchanged ...
function App() {
  const [search, setSearch] = React.useState("");
  const [tvShows, setTvShows] = React.useState([]);

  // Runs once, after the first draw
  React.useEffect(() => {
    async function load() {
      const response = await fetch("https://api.tvmaze.com/shows?page=0");
      const data = await response.json();
      setTvShows(data.map(toShow));
    }
    load();
  }, []);
  const found = tvShows.filter((show) => show.name.toLowerCase().includes(search.toLowerCase()));

  return (
    <main className="container">
// ... unchanged ...
  );
}

// TVmaze's shows look different from ours. This turns one of them into our shape.
function toShow(tv) {
  return {
    id: tv.id,
    name: tv.name,
    genres: tv.genres,
    rating: tv.rating.average,
  };
}
```

Extra 2. Ask for one show ~10 min

Teaches: `GET /api/mylist/:id` answers with one item, or 404.

TASK

Add a route for `GET /api/mylist/:id` that answers with the show that has that id, or `404` with an `error` if there's none.

Test it with **Try a request**: `GET /api/mylist/1`, then `GET /api/mylist/99`.

A CORRECT ANSWER

```
// ... unchanged ...
});

app.get("/api/mylist/:id", (req, res) => {
  const id = Number(req.params.id);
  const found = myList.find((show) => show.id === id);
  if (!found) {
    return res.status(404).json({ error: "That show is not in My List" });
  }
  res.json(found);
});

app.listen(3000, () => {
  console.log("ShowShelf API is ready");
// ... unchanged ...
```

Extra 3. Change a status with PATCH ~20 min

Teaches: `app.patch` changes part of an item; check allowed values with `includes`.

TASK

1. Give every show a `status`: add `status: "want"` to the shows in `myList` and to the new show in your POST route. The page then shows a menu next to each show.
2. Add a route for `PATCH /api/mylist/:id` that answers `400` if `req.body.status` isn't `"want"`, `"watching"` or `"watched"`, `404` if the id isn't in the list, and otherwise changes the show's status and answers with the show.

Then change a status with the menu in the preview.

A CORRECT ANSWER

```
// ... unchanged ...
// My List lives in the server's memory
let myList = [
  { id: 1, name: "Friends", status: "want" },
  { id: 2, name: "Dark", status: "want" },
];

// ... unchanged ...
    return res.status(400).json({ error: "Please send the name of a show" });
  }
  const item = { id: Date.now(), name: req.body.name, status: "want" };
  myList.push(item);
  console.log("Added:", item.name);
// ... unchanged ...
});

const STATUSES = ["want", "watching", "watched"];

app.patch("/api/mylist/:id", (req, res) => {
  if (!STATUSES.includes(req.body.status)) {
    return res.status(400).json({ error: "status must be want, watching or watched" });
  }
  const found = myList.find((show) => show.id === Number(req.params.id));
  if (!found) {
    return res.status(404).json({ error: "That show is not in My List" });
  }
  found.status = req.body.status;
  res.json(found);
});

app.listen(3000, () => {
  console.log("ShowShelf API is ready");
// ... unchanged ...
```

Day 11: Make it yours and publish

Personalise, download, put online, demo

The last day. This morning's lecture went back over the whole course. This afternoon, ShowShelf becomes **your** website: your name, your colours, a feature or two of your own, and an address on the internet that you can send to anyone.

The app is the one you finished on Day 9. Its files are read-only today (open the tabs above the code box to read them). You write in three new files instead:

- **site.js**: your site's name, a sentence about it, and your name (steps 1 and 2)
- **my-theme.css**: your colours, corners and font (steps 3 and 4)
- **extra.js**: new features that use the app's own functions (steps 5 to 7)

Then you download your website as one file, put it online (steps 8 and 9), and prepare a **2-minute demo** for the end of the day (step 10). Aim to finish step 9 with at least 30 minutes to spare.

What next? Keep practising a little every week. freeCodeCamp has free, step-by-step courses, and MDN Web Docs explains every HTML element, CSS property and JavaScript method. A great next project: build ShowShelf again from scratch with a different free API, for example one for music or for books.

Step 1. Tour your app ~20 min

Teaches: A website is several files loaded in order; settings live in one object (`SITE`).

TASK

1. Open the **index.html (whole page)** tab. Find the four `<script src="...">` lines at the bottom. Which file loads last?
2. Open **app.js**. Find the `state` object and the function `openShow`. You'll use both later.
3. Near the bottom of app.js, find **Your site's details**. Which fields of `SITE` does it use?
4. Back in **site.js**, change `name` to a name for your site, like "Couch Club" or "Ana's Shelf". Watch the logo.

COMMON MISTAKES

- Single quotes around a name with an apostrophe: `'Ana's Shelf'` ends the text too early. Use double quotes.
- Deleting the comma at the end of the line, or a quote.
- Typing in a read-only tab and wondering why nothing changes.

NOTE

Give students a few minutes to read app.js properly: steps 5 to 7 use `state`, `openShow`, `render` and `afterRender`. The order of the `<script>` lines explains why extra.js can use everything in app.js.

A CORRECT ANSWER

```
// ... unchanged ...

const SITE = {
  name: "Ana's Shelf",
  tagline: "Search real shows from TVmaze and keep your watchlist.",
  owner: "Your name",
// ... unchanged ...
```

Step 2. A tagline and your name ~10 min

Teaches: The tagline and "Made by" come from `SITE.tagline` and `SITE.owner`.

TASK

In `SITE`:

1. Change `tagline` to your own sentence about your site.
2. Change `owner` to your name.

Scroll down in the preview to see the footer.

COMMON MISTAKES

- Putting the comma inside the quotes: `owner: "Ana Novak,"` (it works, but the footer shows the comma).
- A double quote inside the tagline: `"The "best" shows"` ends the text early.

A CORRECT ANSWER

```
// ... unchanged ...
const SITE = {
  name: "Ana's Shelf",
  tagline: "The shows Ana loves, and the ones she's saving for later.",
  owner: "Ana Novak",
};
```

Step 3. Your colours ~20 min

Teaches: A later stylesheet overrides an earlier one; text needs enough contrast with its background.

TASK

Inside `:root { }`, under the comment, set:

1. `--color-accent`
2. `--color-bg`
3. `--color-surface`
4. one more colour of your choice from the list (for example `--color-accent-dark`, a darker shade of your accent)

The check also measures the contrast of your text and buttons.

COMMON MISTAKES

- Writing the new lines inside the comment, before the `*/`, so they're ignored.
- One dash instead of two: `-color-bg`.
- A dark `--color-bg` with the default dark text: the check reports low contrast.

NOTE

The contrast numbers follow the web accessibility guidelines (WCAG): 4.5 to 1 for normal text, 3 to 1 for large or bold text like buttons. Light yellow or pale accents usually fail the button test: that's a good conversation about why.

A CORRECT ANSWER

```
/* ... unchanged ... */
     And one that isn't a colour:
       --radius               how round the corners of the cards are */
  --color-accent: #0a7d5a;
  --color-accent-dark: #075c42;
  --color-bg: #f1f7f4;
  --color-surface: #ffffff;
}
```

Step 4. Corners and a font ~15 min

Teaches: `--radius` sets the corners; `font-family` on `body` changes the whole site's font.

TASK

1. Inside `:root { }`, add `--radius` with a new value, rounder (like `20px`) or sharper (like `0px`).
2. Under the `:root` rule, add a `body { }` rule with a `font-family` of your choice, ending with a general family (`serif` or `sans-serif`).

COMMON MISTAKES

- `--radius: 20;` without `px`: the browser ignores it.
- The `body` rule inside the `:root { }` brackets.
- A font name with spaces and no quotes, then a typo: the browser quietly uses the backup.

A CORRECT ANSWER

```
/* ... unchanged ... */
  --color-bg: #f1f7f4;
  --color-surface: #ffffff;
  --radius: 20px;
}

body {
  font-family: "Trebuchet MS", Arial, sans-serif;
}
```

Step 5. A "Surprise me" button ~20 min

Teaches: Add features from a separate file by using the app's `state` and functions.

TASK

In extra.js:

1. Create a `button` with the text "Surprise me" and the class `button-secondary`.
2. Append it to the hero: `document.querySelector("#home-view .hero")`.
3. On click: pick a random show from `state.shows` and call `openShow` with its `id`.

Try it after a search too: it should only pick from the shows you see.

COMMON MISTAKES

- `openShow(state.shows[index])`: the whole show instead of its `id`, so TVmaze can't find it.
- `Math.random() * 12`: works until a search loads fewer shows.
- Naming the button `button` or `grid`: "has already been declared".

NOTE

If a student's Console says "has already been declared", they reused a name from app.js. Any new name fixes it.

A CORRECT ANSWER

```
// extra.js: your own features. This file runs last, so everything in app.js is ready:
//   state          state.shows (the shows loaded now), state.myList, state.view ...
//   render()       draws the whole page from the state
//   openShow(id)   opens the show view for one show
//   toggleMyList(show), saveMyList(), homeView, grid ... and more.
// Open the app.js tab to see them all.

// ---------- Surprise me: open a random show ----------
const surpriseButton = document.createElement("button");
surpriseButton.className = "button-secondary";
surpriseButton.textContent = "Surprise me";
document.querySelector("#home-view .hero").append(surpriseButton);

surpriseButton.addEventListener("click", () => {
  if (state.shows.length === 0) {
    return;   // no shows loaded (yet)
  }
  const index = Math.floor(Math.random() * state.shows.length);
  openShow(state.shows[index].id);
});
```

Step 6. Top rated first ~15 min

Teaches: `sort((a, b) => b.rating - a.rating)` puts the highest first; change the state, then `render()`.

TASK

In extra.js, under your Surprise me code:

1. Create a button with the text "Top rated first" and the class `chip`, and append it to the toolbar: `document.querySelector(".toolbar")`.
2. On click: sort `state.shows` from the highest `rating` to the lowest, then call `render()`.

COMMON MISTAKES

- `a.rating - b.rating`: lowest first.
- Forgetting `render()`: the state is sorted but the page isn't redrawn.
- `state.shows = state.shows.sort(...)` works too, but isn't needed: `sort` changes the array itself.

A CORRECT ANSWER

```
// ... unchanged ...
  openShow(state.shows[index].id);
});

// ---------- Top rated first: sort the grid by rating ----------
const sortButton = document.createElement("button");
sortButton.className = "chip";
sortButton.textContent = "★ Top rated first";
document.querySelector(".toolbar").append(sortButton);

sortButton.addEventListener("click", () => {
  state.shows.sort((a, b) => b.rating - a.rating);   // highest rating first
  render();
});
```

Step 7. A personal touch ~15 min

Teaches: A hook (`afterRender`) lets extra code run at the right moment in someone else's code.

TASK

Choose **one** (or more):

1. **A welcome line** with your name in the hero, like "Welcome! This shelf belongs to Ana." (`SITE.owner` has your name.)
2. **A count line** in the hero, like "Shows loaded: 12", that changes after every search. Set its text in `afterRender`, from `state.shows.length`.
3. **A badge** on the cards of shows in My List. In `afterRender`, loop over the cards: a card's round button has `aria-pressed` "true" when its show is in My List (see `createListButton` in app.js).

COMMON MISTAKES

- Spelling the hook differently (`afterrender`, `afterRendering`): app.js never calls it.
- Setting the count once at the top: it says 0, because the shows haven't loaded yet.
- Two functions called `afterRender`: only the last one counts.

NOTE

Any of the three choices passes. The badge is the hardest; point stronger students at it.

A CORRECT ANSWER

```
// ... unchanged ...
  render();
});

// ---------- A personal touch: how many shows are loaded ----------
const countLine = document.createElement("p");
countLine.className = "meta";
document.querySelector("#home-view .hero").append(countLine);

// app.js calls afterRender at the end of every render(), so the line stays up to date
function afterRender() {
  countLine.textContent = "Shows loaded: " + state.shows.length;
}
```

Step 8. Download your website ~15 min

Teaches: A website can be one `index.html` file; `<meta name="description">` is the sentence search engines show.

TASK

1. In `SITE`, add a `description`: one sentence about your site, at least 20 characters.
2. Press **Check**. It makes sure the page has no errors and that your name and the site's name are set.
3. Press **Download my website**. The file goes to your **Downloads** folder. (If there's already an `index.html` there, your browser may call it `index (1).html`. Rename it to `index.html`.)
4. Open your Downloads folder and **double-click** `index.html`: your site opens in the browser, with real data and real posters from TVmaze. It needs internet now, and My List starts empty there.

COMMON MISTAKES

- Forgetting the comma at the end of the `owner` line before the new `description` line.
- Downloading twice and opening the old `index.html` instead of `index (1).html`.
- Opening the file without internet: the page says the shows could not be loaded (the Lab's sample data isn't in the file).

NOTE

Some browsers ask where to save; tell students to choose Downloads or the Desktop and remember where. If a student only sees the code when opening the file, it was opened in a text editor: right-click, Open with, and choose a browser.

A CORRECT ANSWER

```
// ... unchanged ...
  tagline: "The shows Ana loves, and the ones she's saving for later.",
  owner: "Ana Novak",
  description: "My TV show finder: search real shows, see their cast and seasons, and keep a
watchlist.",
};
```

Step 9. Put it online ~25 min

Teaches: Hosting puts your files on a web server; deploying is uploading them.

TASK

Option A: Netlify Drop

1. On your computer, make a new folder, for example `my-showshelf`, and move your `index.html` into it.
2. Open app.netlify.com/drop. If it asks you to sign up or log in, make a free account: you need one so your site stays online.
3. Drag the **folder** from your computer onto the page and drop it.
4. Wait a few seconds. Netlify shows your site's address, ending in `.netlify.app`. Open it.

Option B: GitHub Pages

1. Sign up or log in at github.com.
2. Make a new repository: **+** (top right), **New repository**. Name it, for example `showshelf`, choose **Public**, and press **Create repository**.
3. On the new page, click the link **uploading an existing file**, drag in your `index.html`, and press **Commit changes**.
4. Open **Settings**, then **Pages** (left menu). Under "Build and deployment", choose **Deploy from a branch**, then the branch `main` and the folder `/ (root)`, and press **Save**.
5. After a minute or two, your site is at `https://USERNAME.github.io/REPO/` (your user name, then the repository's name). Reload the Pages settings to see the link.

Then: open your site, copy its address from the address bar, and paste it on the first line of the box.

COMMON MISTAKES

- Dropping the `index.html` file instead of the folder on Netlify Drop, or a folder where the file is called `index (1).html`.
- Pasting the repository address (`github.com/...`) instead of the site address (`....github.io/...`).
- A private repository: GitHub Pages needs it to be public on a free account.

NOTE

Both options need an email address for the account. A student who doesn't want an account can skip this step and give the demo from the downloaded file (with internet). GitHub Pages can take a few minutes the first time; let students start step 10 while they wait. To update a site later: on Netlify, open the site and drag the folder onto its Deploys page; on GitHub, upload the new `index.html` into the same repository.

A CORRECT ANSWER

```
https://ana-novak-showshelf.netlify.app
```

Step 10. Prepare your demo ~10 min

Teaches: A short demo: what it does, what you're proud of, what's next.

TASK

In the box, write **three lines**, each at least a full sentence:

1. What your site does.
2. The part you're proudest of, and why.
3. What you'd add next if you had another week.

Then practise once, out loud, with the site open.

COMMON MISTAKES

- One long line with all three answers: press Enter between them.
- Planning to demo from the Lab instead of the real site.

NOTE

Keep demos to 2 minutes with a visible timer. Have a backup: if the network is slow, students can demo from the downloaded file or from the Lab preview.

A CORRECT ANSWER

```
Ana's Shelf finds any TV show and keeps a watchlist of the ones I want to see.
I'm proudest of the Surprise me button, because it helped me choose tonight's show.
Next I'd add a page for each actor, with all the shows they're in.
```

Extra 1. A dark theme button ~20 min

Teaches: `classList.toggle` switches a class; `localStorage` remembers the choice.

TASK

In extra.js:

1. Create a button with the text "Dark mode" and the class `chip`, and append it to the header's `.nav`.
2. On click: toggle the class `dark` on `document.body`, then save `"dark"` or `"light"` in `localStorage` under the key `"theme"`.
3. At the start (outside the listener): if the saved theme is `"dark"`, add the class `dark` to `document.body`.

NOTE

The dark colours are fixed in the page (the `body.dark` rule), so students only write the JavaScript. Their own `my-theme.css` colours still apply in light mode.

A CORRECT ANSWER

```
// ... unchanged ...
  countLine.textContent = "Shows loaded: " + state.shows.length;
}

// ---------- Extra: a dark theme button ----------
const themeButton = document.createElement("button");
themeButton.className = "chip";
themeButton.textContent = "Dark mode";
document.querySelector(".nav").append(themeButton);

// Start dark if that was the last choice
if (localStorage.getItem("theme") === "dark") {
  document.body.classList.add("dark");
}

themeButton.addEventListener("click", () => {
  document.body.classList.toggle("dark");
  if (document.body.classList.contains("dark")) {
    localStorage.setItem("theme", "dark");
  } else {
    localStorage.setItem("theme", "light");
  }
});
```

Extra 2. Clear My List ~15 min

Teaches: `confirm()` asks OK or Cancel and gives `true` or `false`.

TASK

In extra.js:

1. Create a button with the text "Clear My List" and the class `button-secondary`, and append it to the hero of the My List view: `document.querySelector("#mylist-view .hero")`.
2. On click: if `confirm(...)` says OK, set `state.myList` to an empty array, then call `saveMyList()` and `render()`.

Add a few shows, open **My List**, and try both Cancel and OK.

A CORRECT ANSWER

```
// ... unchanged ...
  countLine.textContent = "Shows loaded: " + state.shows.length;
}

// ---------- Extra: clear My List, after asking ----------
const clearButton = document.createElement("button");
clearButton.className = "button-secondary";
clearButton.textContent = "Clear My List";
document.querySelector("#mylist-view .hero").append(clearButton);

clearButton.addEventListener("click", () => {
  if (confirm("Remove every show from My List?")) {
    state.myList = [];
    saveMyList();
    render();
  }
});
```

Extra 3. An About section ~15 min

Teaches: Build a section in JavaScript: create elements, `append` them together, then add them to the page.

TASK

In extra.js:

1. Create a `section` with the class `show`.
2. Inside it, put an `h2` that says "About this site" and a `p` with a few sentences about you and how you built the site.
3. Append the section to `homeView`.

A CORRECT ANSWER

```
// ... unchanged ...
  countLine.textContent = "Shows loaded: " + state.shows.length;
}

// ---------- Extra: an About section ----------
const about = document.createElement("section");
about.className = "show";
const aboutTitle = document.createElement("h2");
aboutTitle.textContent = "About this site";
const aboutText = document.createElement("p");
aboutText.textContent = "I'm " + SITE.owner + ". I built this site in the Web Dev in 11 Days course,
starting from zero: HTML, CSS, JavaScript and the free TVmaze API.";
about.append(aboutTitle, aboutText);
homeView.append(about);
```